databricks

Big Book of Data Engineering

Contents

Introduction to Data Engineering in the AI Era 3

Guidance and Best Practices 13

From PDF to Insights: Autonomous Document Intelligence at Scale With Lakeflow and Agent Bricks 14

Auto Loader With File Events — Simplified File Discovery at Scale 26

A Practitioner's Guide to Real-Time Mode on Spark Declarative Pipelines 35

Spark Declarative Pipelines "How-To" Series, Part 1: How to Save Results Into a Table 43

Stop Hand-Coding Change Data Capture Pipelines 48

Why Data Engineering Needs to Become End-to-End Declarative 58

Building a Near Real-Time Application With Zerobus Ingest and Lakebase 63

Petabyte Scale With Zerobus Ingest: Download the Code and Ingest the Milky Way 70

Try the New Zerobus Rescue Column: Choose-Your-Own-Adventure Schema Management (Beta) 72

On The Agentic Roadmap 77

What's new in Genie Code at Data + AI Summit 2026 78

Introducing Genie ZeroOps: Put Your Data and AI Operations on Autopilot 83

Case Studies 87

Panasonic: Panasonic accelerates enterprise BI with Lakeflow 88

NAB: How NAB's journey to 100% declarative pipelines is helping data flow like electricity 91

Coinbase: Databricks powers real-time fraud detection at Coinbase 94

AccuWeather: Transforming weather forecasting with Lakeflow Jobs 97

Toyota: Toyota uses Zerobus Ingest for real-time factory data 101

Daikin: How Daikin Applied Americas builds consistent data pipelines at scale with Genie Code 104

01 Introduction

Introduction to Data Engineering in the AI Era

As organizations shift toward AI-native workflows, the demand for fresh, trusted enterprise data has never been higher. Meanwhile, most data engineering teams are still hand-authoring the pipelines feeding these models. What happens when pipeline creation volume outpaces the people managing them? While coding agents and AI tools have accelerated and improved traditional software development processes, data engineering is just starting to realize these same transformative benefits to help automate complex pipeline creation.

As AI workloads scale, they require a foundation of continuous, trusted high-quality data. However, the engineers tasked with delivering this data are often trapped maintaining a fragmented stack of legacy ETL tools. This tool sprawl creates deep silos that are incredibly difficult to integrate and govern. Instead of architecting the data foundation of the future, highly skilled practitioners are bogged down by the daily operational burden of keeping rigid pipelines afloat.

From predictive analytics to real-time applications and agentic workflows, everything starts with good data. But automated pipelines are only as valuable as the infrastructure supporting them.

Overcoming this bottleneck means rethinking how data is moved and processed, eliminating the gaps caused by tool sprawl and adopting a unified, agentic data engineering foundation.

This book uses practical guidance, useful patterns, best practices and real-world examples to help you understand how data engineers can meet the challenges of this new era. Read on to learn how the Databricks Data + AI Platform can support your journey.

databricks

What is agentic data engineering?

Agentic data engineering is the shift from manually hand-coding and monitoring pipelines to utilizing autonomous AI agents that understand your enterprise context to help author, operate and troubleshoot your data workflows.

AI agents can't function in a vacuum, though. They require complete end-to-end context to ensure safety, security and accuracy. To meet this standard, agentic data engineering requires a unified foundation.

Lakeflow now covers the entire data lifecycle across three key areas:

1. **Unified data engineering:** Connect to enterprise applications, databases, and real-time event streams without the manual burden of managing fragmented, siloed message buses. Seamlessly transform your data (filter, standardize, and aggregate) into a medallion architecture of Bronze, Silver, and Gold across both batch and streaming within a single foundation.
2. **Agentic development:** Move beyond hand-coded pipelines. Context-aware AI agents can help author, optimize, and manage your data transformations, significantly accelerating pipeline delivery and reducing development overhead.
3. **Autonomous operations**: Replace blind scheduling and manual triage with a control plane that understands actual data readiness. Native AI agents continuously monitor production assets in the background to help orchestrate complex dependencies, detect silent failures, trace root causes across schema changes, and safely propose verified fixes.

Challenges of data engineering in the AI era

Data engineers who build and maintain ETL pipelines and the data infrastructure that underpins analytics and AI workloads face specific challenges in this fast-moving, agentic landscape:

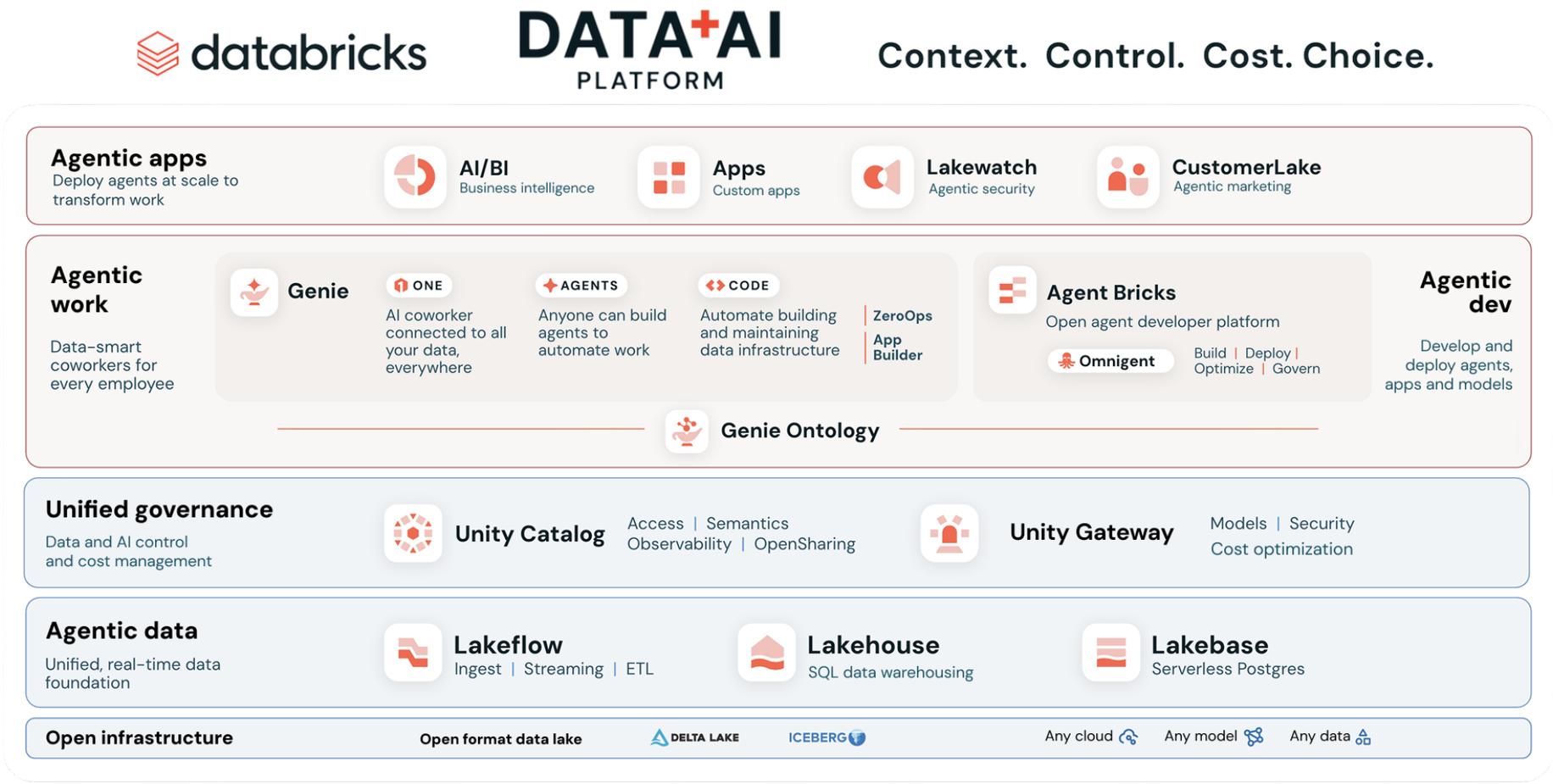
- **Generic coding agents fall short:** While they can generate pipeline code, they lack essential context, platform semantics, and governance. An agent can write flawless Python and still fail if it doesn't know which upstream definition of 'revenue' to use. Agentic data engineering requires purpose-built agents on a unified data and AI foundation.
- **Tool sprawl and fragmented data stacks:** Most companies rely on multiple external tools and services or custom in-house solutions to manage their ETL processes. This creates specialized, siloed teams, incredibly complex integrations and an operational bottleneck that blocks AI innovation.
- **The maintenance burden:** Data teams report spending over 50% of their time fighting fires rather than building. Failures are often silent — such as upstream schema changes or bad data propagating through a dependency chain. To fix this, your AI operations agent can't be a generic external coding tool. The agent must live directly within your platform and have access to your production data.
- **Dual-engine complexity for real-time data:** Achieving ultra-low-latency streaming has traditionally forced data teams to manage complex architectures. This often requires them to maintain a second specialized engine (such as Apache Flink) alongside their batch processing engines.
- **Blind orchestration:** Legacy orchestrators (like Airflow) often rely on time-based cron schedules, which are essentially guesses at when data is ready. This leads to processing stale data or wasting compute resources on workflows that fail due to unmet dependencies.

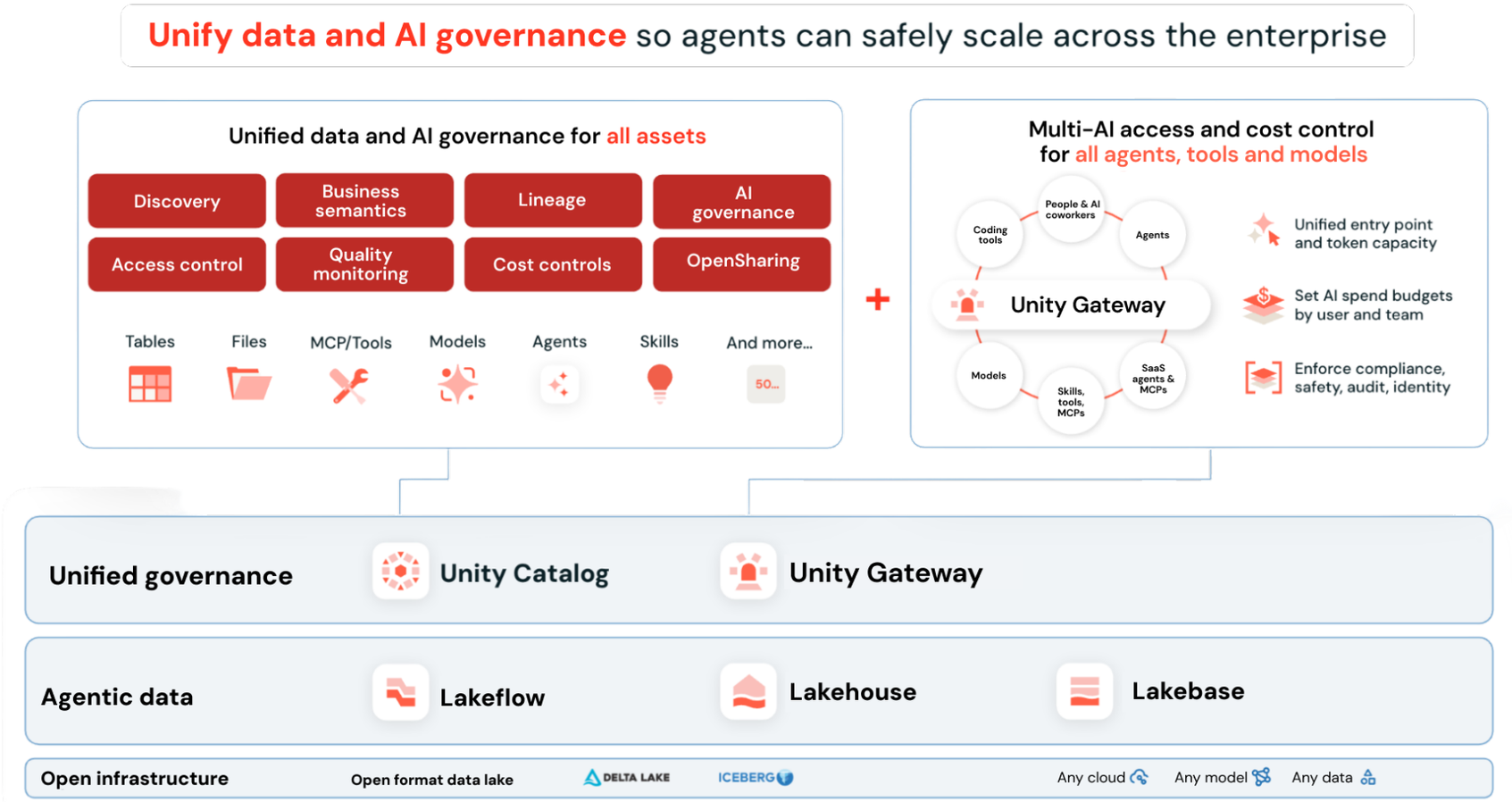
Unifying the data architecture provides the end-to-end context required to power agentic data engineering. This means shifting the burden of manual pipeline maintenance to AI. To execute on this vision, organizations need a platform built from the ground up to unify data and AI.

The Databricks Data + AI Platform

The Databricks mission is to democratize data and AI so organizations can use their unique data to build or fine-tune their own machine learning and generative AI models, producing new insights that drive business innovation.

The **Databricks Data + AI Platform** provides an open, unified foundation for all data, analytics and AI. At its core, the Databricks Platform eliminates silos by unifying your entire architecture with Unity Catalog, which provides a single governance model for managing permissions, auditing and column-level lineage. Since agents require complete context to act safely, unified governance and lineage are exactly what make agentic automation possible.



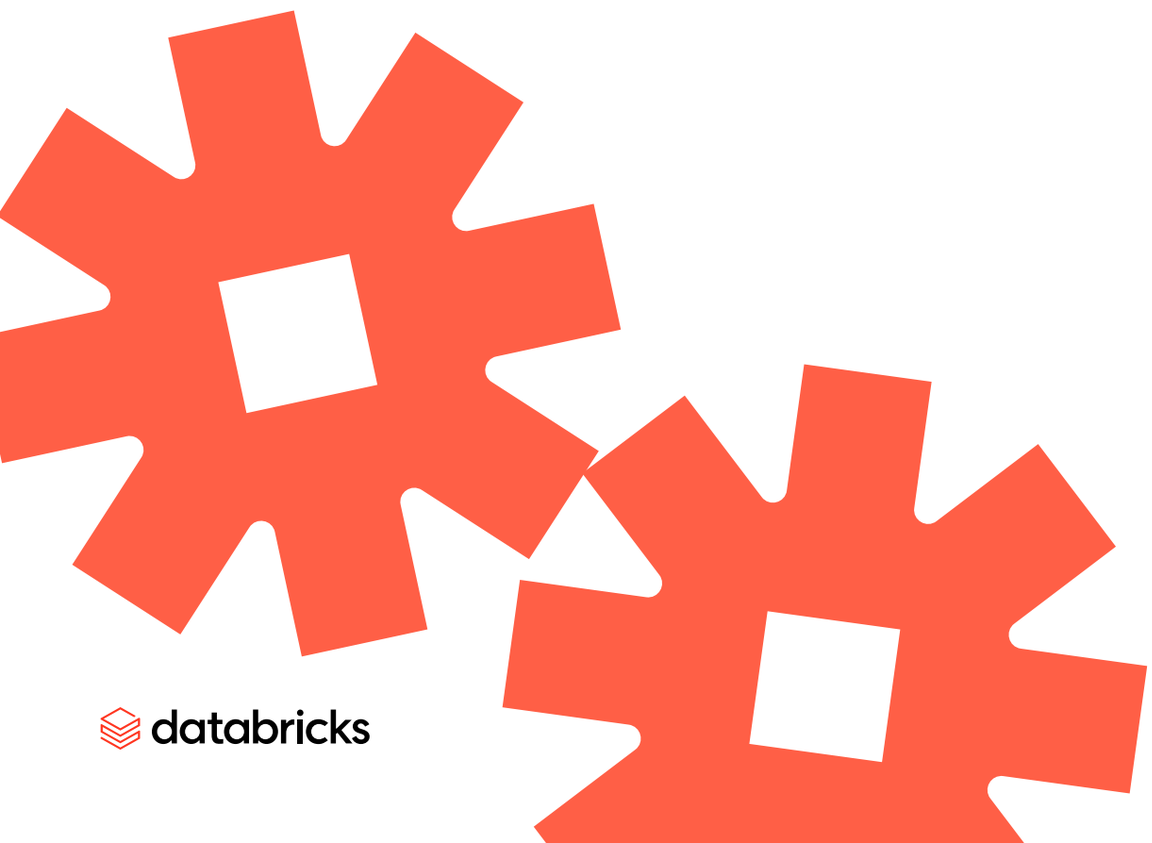


In addition, this foundation seamlessly integrates with operational systems. With built-in capabilities like Databricks Lakebase (a fully managed Postgres database), the platform automatically syncs your lakehouse data, features and models directly into an operational database. This ensures your downstream applications and AI agents have low-latency access to the data they need, completely eliminating the need for engineers to build and maintain fragile reverse-ETL pipelines.

UNIFIED DATA ENGINEERING WITH DATABRICKS LAKEFLOW

To solve the challenges of the fragmented data stack, organizations must move away from disjointed tools and adopt a cohesive strategy. Lakeflow is the end-to-end data engineering solution built natively on the Databricks Platform to simplify pipeline development and management across the entire data lifecycle.

Lakeflow serves as the engine that builds a unified data foundation for AI. Because Lakeflow unifies ingestion, transformation and orchestration under Unity Catalog, it creates a single, governed source of truth. To execute on this unified foundation, teams use Databricks Genie, a suite of context-aware AI agents designed to help author, operate and troubleshoot data workflows. Genie is integrated into the Lakeflow experience.



This is exactly the context Databricks Genie needs to understand your business and execute agentic work. Keeping everything on a single platform eliminates tool sprawl and gives your AI agents the context required to do the heavy lifting.

AGENTIC DEVELOPMENT AND OPERATIONS WITH DATABRICKS GENIE

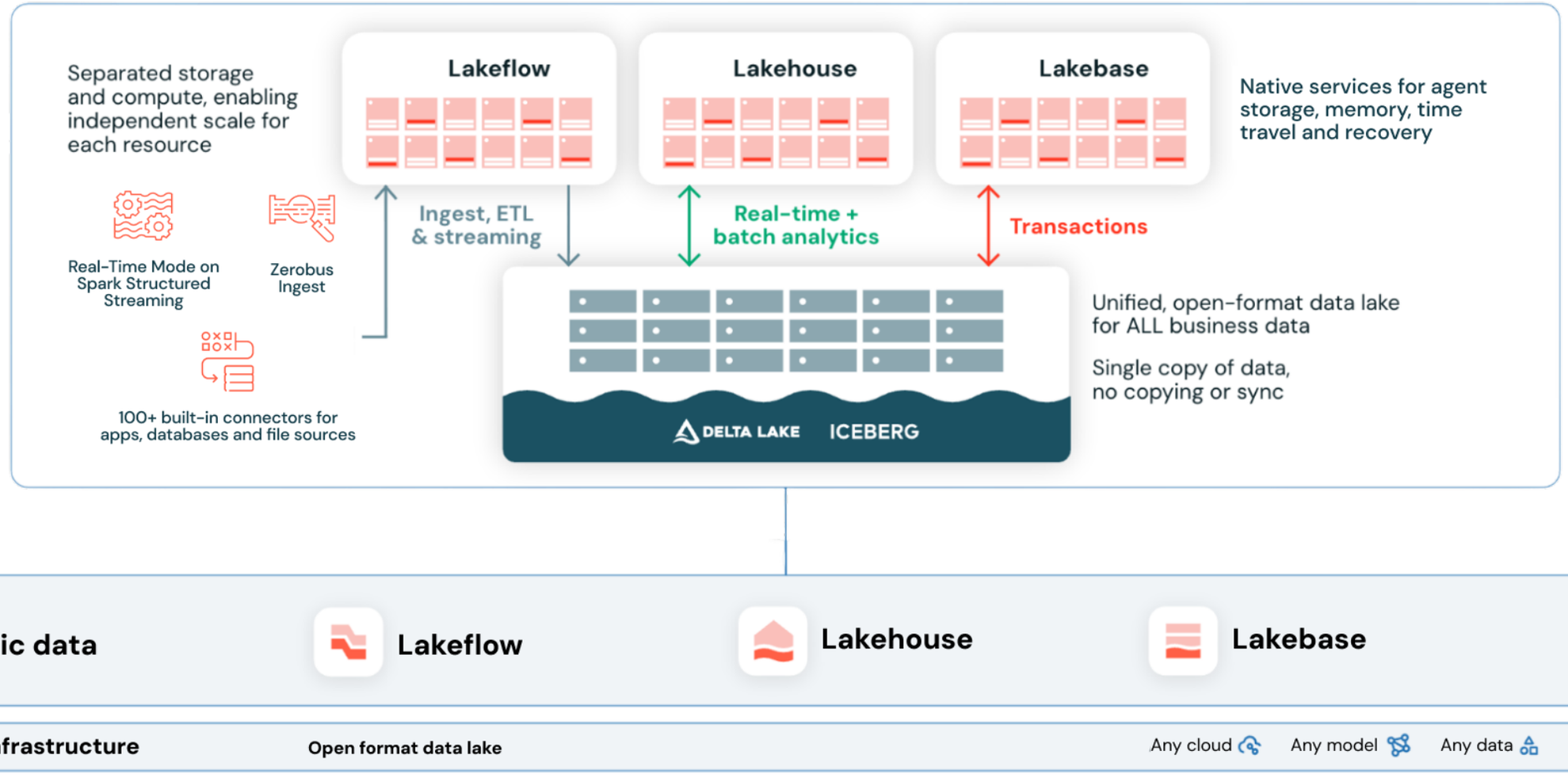
The era of agentic data engineering is already here.

- **Build with Genie Code:** Unlike generic coding assistants, it is purpose-built for data engineering. Genie Code operates with the full context of your data schema, lineage, and governance policies. Whether you're building ingestion connectors, writing Python or SQL, or orchestrating complex jobs, it understands your specific pipeline structure.
- **Operate with Genie ZeroOps:** Recently announced, Genie ZeroOps is a background agent built into the Databricks Platform that autonomously monitors production assets. Since it runs inside the platform, ZeroOps uses Unity Catalog lineage to track failures back to their root cause. Unlike external coding agents, ZeroOps validates every proposed fix against real data in an isolated sandbox. Nothing is applied to production without human-in-the-loop approval.

Ultimately, a unified data foundation is what makes agentic AI possible. With a deep, governed enterprise context, you can effectively deploy autonomous agents to help drive business impact. Lakeflow delivers unified agentic data engineering by bringing ingestion, transformation, and orchestration together on a single platform. Then, purpose-built AI agents help you build, operate, and continuously improve your data pipelines.

Deploy a unified, agent-ready data infrastructure

Best-in-class data ingest, warehousing, analytics and transactions from a single data lake



DATA INGESTION WITH LAKEFLOW CONNECT AND ZEROBUS INGEST

Lakeflow Connect securely brings all your data into the Data + AI Platform to fuel downstream AI and BI. It eliminates the need for brittle third-party ingestion tools by offering over 100 native, fully managed connectors. You can incrementally ingest data from enterprise applications (Salesforce, Workday, ServiceNow), databases (SQL Server) and unstructured file sources (SharePoint, Google Drive) using a simple UI or API.

For high-volume event data, Zerobus Ingest changes how organizations handle telemetry and IoT streams. It provides Kafka-free ingestion directly to the lakehouse, delivering near real-time writes (under five seconds) and massive throughput (up to 10GB/s per table). By supporting gRPC, REST APIs and native SDKs, Zerobus allows your data producers to stream data directly to the Databricks Platform without the operational nightmare of managing a stand-alone message bus.

UNIFIED BATCH AND STREAMING WITH APACHE SPARK™ DECLARATIVE PIPELINES

Apache Spark™ Declarative Pipelines (SDP) is a declarative ETL framework that allows teams to focus on business logic rather than infrastructure. Simply define the transformations you want in SQL or Python, and the pipeline automatically handles task orchestration, compute management, change data capture (Auto CDC) and data quality monitoring. Reliable data pipelines are now made easy with SDP.

Crucially, SDP bridges the gap between batch and streaming. And now with the introduction of Real-Time Mode, SDP achieves end-to-end streaming latencies as low as five milliseconds. This allows data teams to deliver ultra-low-latency analytics on a single platform, eliminating the high cost and complexity of spinning up and maintaining a separate serving engine like Apache Flink.

NO-CODE DATA PREP AND EXPLORATION WITH LAKEFLOW DESIGNER

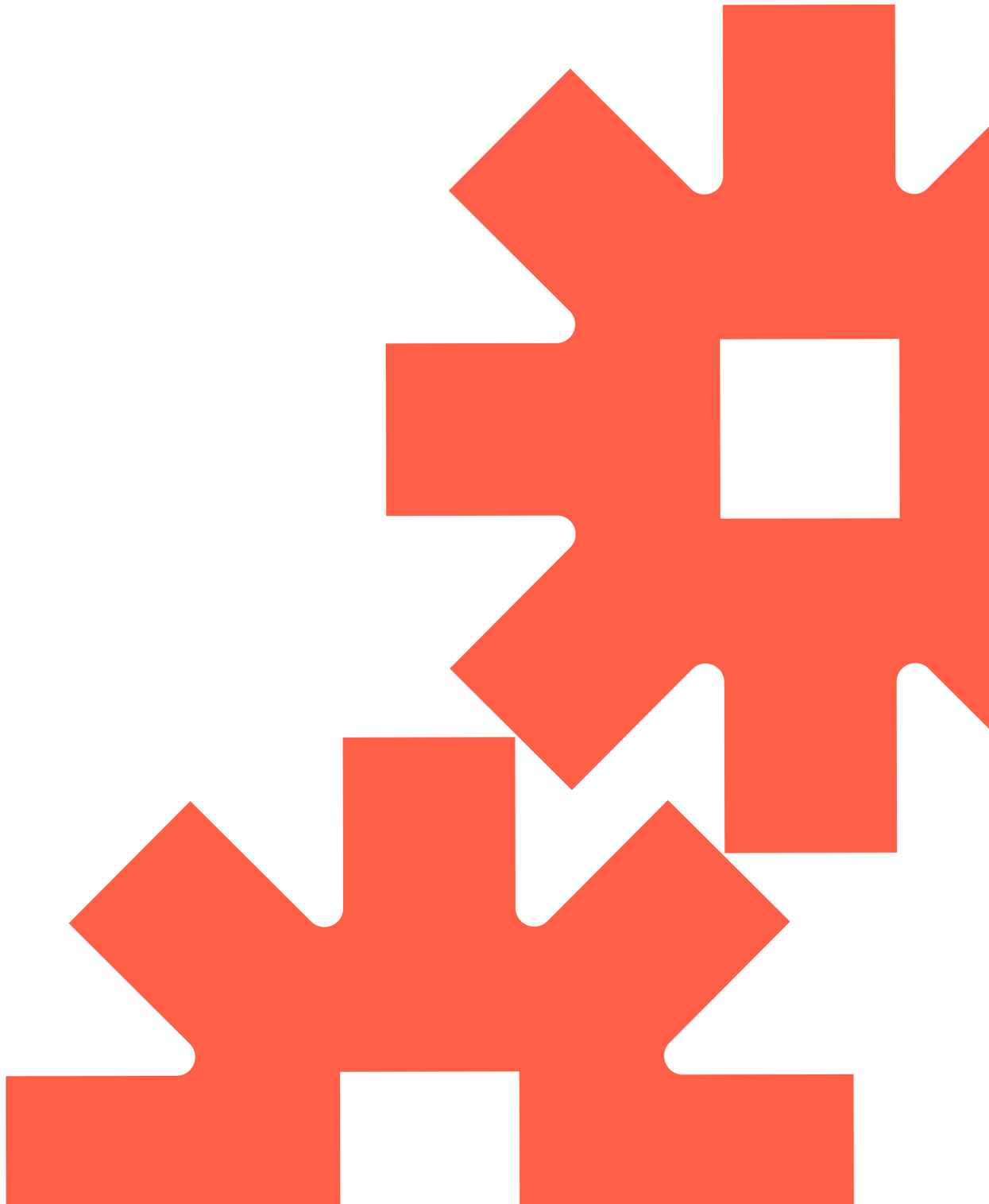
To truly democratize data engineering, business analysts and nontechnical users need access to reliable tooling. Lakeflow Designer provides an AI-first, drag-and-drop visual canvas for building data pipelines with natural language prompts.

Unlike traditional no-code tools that lock you into proprietary formats, every visual data prep built in Lakeflow Designer natively generates production-ready code under the hood. This ensures zero translation loss between teams. A business analyst can design the pipeline visually, and a data engineer can immediately review, refine and optimize the underlying code without switching context.

DATA-AWARE ORCHESTRATION WITH LAKEFLOW JOBS

Lakeflow Jobs replaces fragmented legacy orchestrators by providing a native, managed orchestration service for any data, analytics or AI workflow. Moving beyond basic time-based scheduling, Lakeflow Jobs enables *data and context-aware orchestration*. Workflows can be triggered based on actual data readiness and contracts, ensuring you never process stale data or waste compute resources.

It also supports advanced control flows, including conditional execution, loops and modular orchestration using CI/CD asset bundles. For architectures that extend beyond the Databricks Platform, Lakeflow Jobs provides external orchestration capabilities, allowing you to seamlessly trigger Snowflake jobs, fire custom REST APIs or manage Slack alerts from a single, unified control plane.



Conclusion

As organizations strive to build AI-native applications, data engineering is a critical focal point. With Databricks Lakeflow and the Data + AI Platform, data engineers are empowered to leave behind the fragmented data stack of legacy ETL. By adopting an agentic data foundation, teams can automate their operations, eliminate silos and focus on delivering the trusted, real-time data that makes AI possible.

In the next sections of the ***Big Book of Data Engineering***, we describe best practices and end-to-end use cases for working with Databricks Lakeflow, drawn from real-world examples. From data ingestion and real-time streaming to autonomous operations, you'll learn how to apply proven patterns to build the data infrastructure of the future.

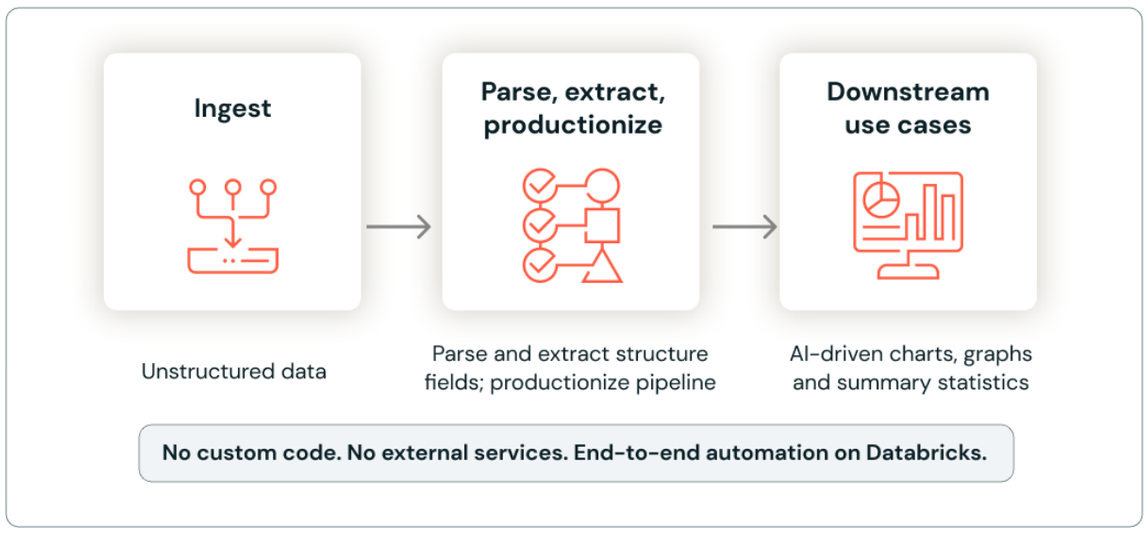
In this eBook, you'll learn:

- **Eliminate tool sprawl:** Simplify your data architecture by natively integrating enterprise sources without relying on complex message buses or separate serving engines
- **Scale pipeline delivery:** Build and deploy reliable streaming and batch pipelines at scale, significantly reducing the cost and complexity of traditional ETL processes
- **Run pipelines autonomously:** Replace blind, time-based scheduling with smart, data-aware workflows that trigger based on actual data readiness
- **Automate data operations:** Leverage built-in AI agents to automatically detect silent failures, safely verify fixes in isolated sandboxes, and transition away from brittle legacy stacks

02 Guidance and Best Practices

From PDF to Insights: Autonomous Document Intelligence at Scale With Lakeflow and Agent Bricks

By Giselle Goicochea and Jason Ping



Every organization has critical information trapped in PDFs and unstructured documents: forms, reports, records, filings. Historically, turning those files into usable data has meant manual data entry, brittle OCR scripts or single-purpose tools that don't scale.

On Databricks, you can treat documents like any other data source. Ingest them reliably, enrich them with AI, productionize the workflow and expose the results to business users, all in one place.

This section gives you a step-by-step pattern for building a production-grade Intelligent Document Processing (IDP) pipeline using Lakeflow Connect, Lakeflow Spark Declarative Pipelines, Lakeflow Jobs, and Document Intelligence. You can learn more about the broader architecture and strategy in Building with Databricks Document Intelligence and Lakeflow.

Rather than managing a complex web of disconnected tools, this solution uses Lakeflow to create a simple, unified environment for document intelligence. We take raw, unstructured documents from your source and transform them into production-ready insights, all within the Databricks Platform.

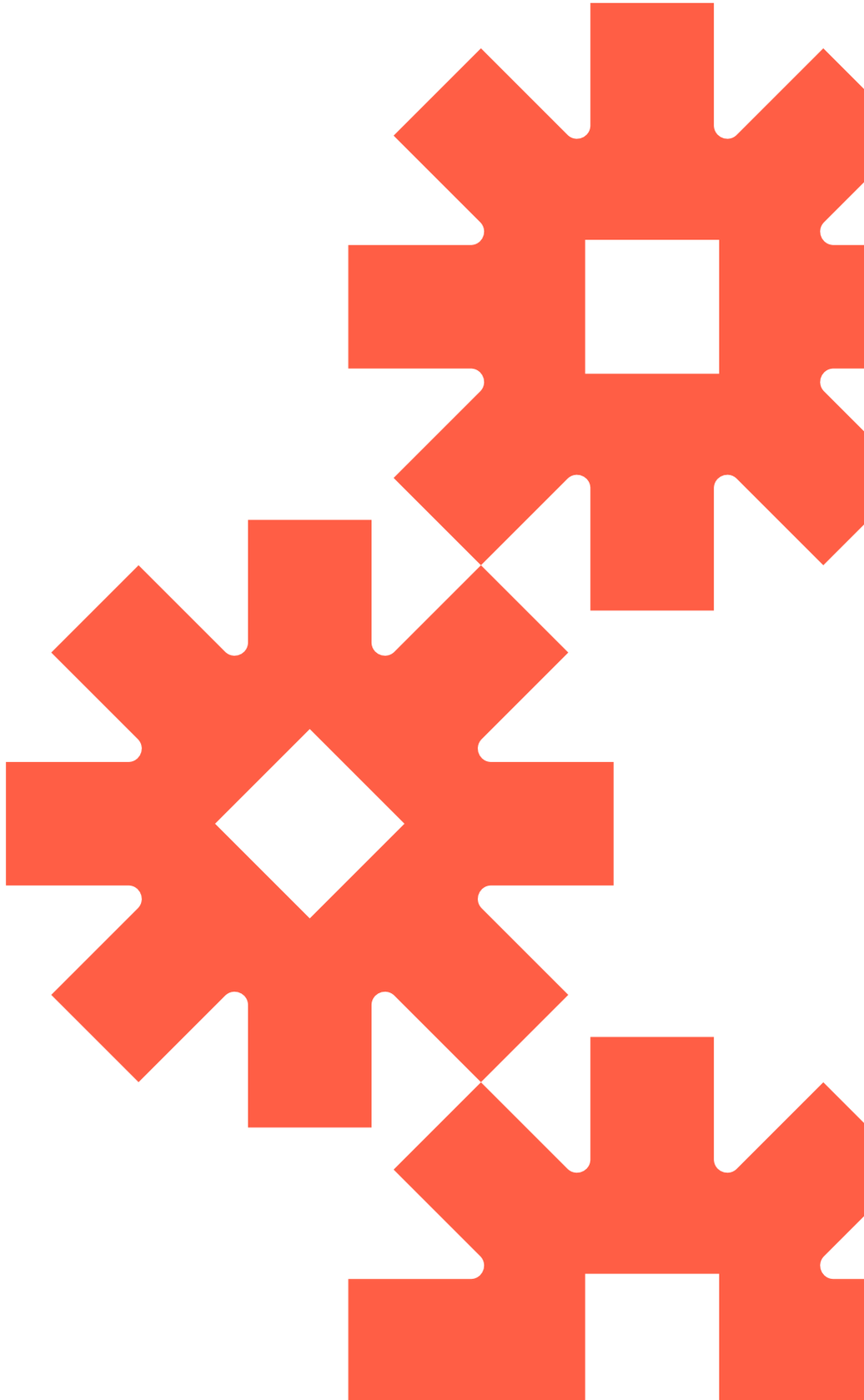
You can watch the video walkthrough here, or follow the step-by-step guide below.

Step 1: Select source documents

The starting point is a set of PDFs sitting in a SharePoint document library: the kind of unstructured files every organization accumulates, filled out by hand, varying in format and difficult to process programmatically.

In the companion video on the previous page, we use aircraft maintenance log books as a concrete example. However, the same pattern applies to any document type: invoices, claims forms, engineering specs, medical records, inspection reports or any other PDF your teams produce and store.

These are the documents we'll be pulling into our pipeline to extract rich insights using Lakeflow and Document Intelligence. What makes this set particularly challenging is their variability, as they contain a mix of handwritten and printed text, tables with inconsistent layouts and key information buried in different locations across every file. While traditional OCR tools often struggle with this level of complexity, the combination of a managed pipeline and native AI functions allows us to handle these challenges reliably and at scale.



Step 2: Ingest with Lakeflow Connect

Lakeflow Connect provides two ways to ingest documents from SharePoint: a wizard-driven UI and a SQL API. Both create a fully managed, incremental ingestion pipeline.

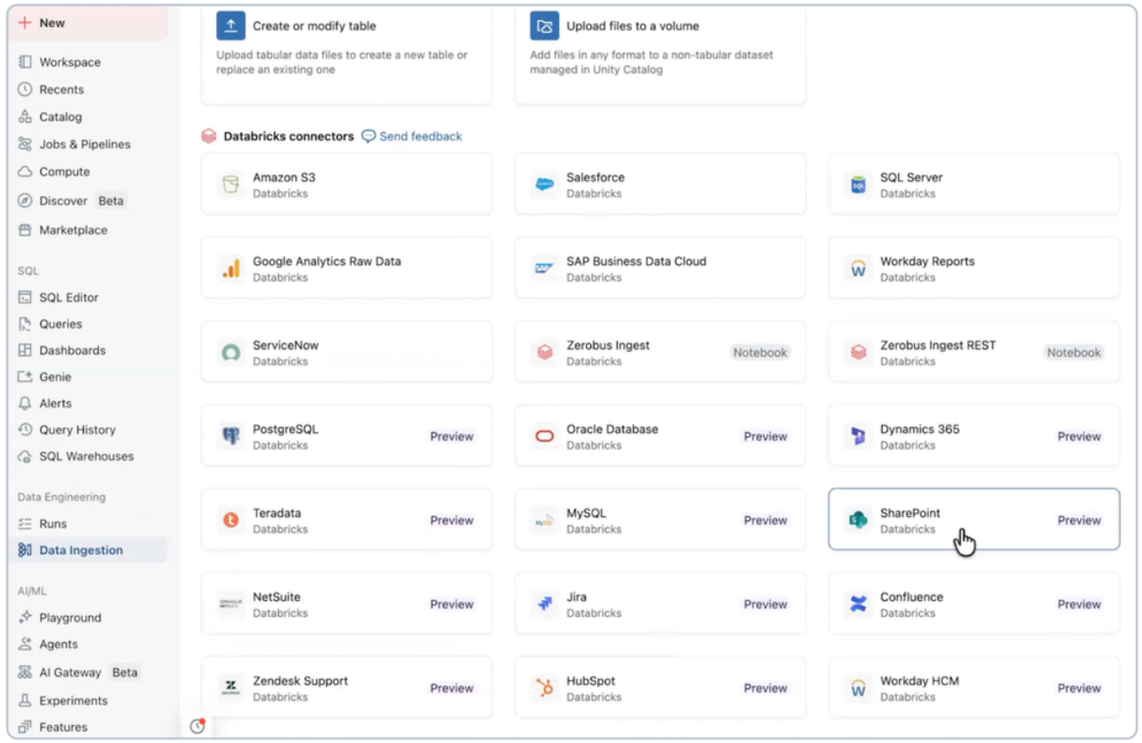
INGESTION UI

From the Databricks data ingestion page, select 'SharePoint' as the source. The wizard walks you through five steps:

- **Connection:** Select an existing Unity Catalog connection to your SharePoint environment, or create a new one. You'll need your SharePoint site ID, tenant ID, client ID and client secret. These credentials are stored securely in Unity Catalog.
- **Ingestion setup:** Name the pipeline and select 'unstructured data' as the data type (since you're working with PDFs).
- **Select source:** Provide the URL to your SharePoint document library. Browse and select the files you want to ingest.
- **Destination:** Choose the destination catalog, schema, and table name where the files will land.
- **Schedule:** Set a cadence for the pipeline to check for new or modified files. A daily schedule is typical.

That's the entire setup. No incremental logic scripts, no building connectors from scratch. Lakeflow Connect manages all of it.

Note: *The ingestion wizard UI is being rolled out and may not be available in all workspaces yet. If you don't see it, use the SQL API on the next page.*



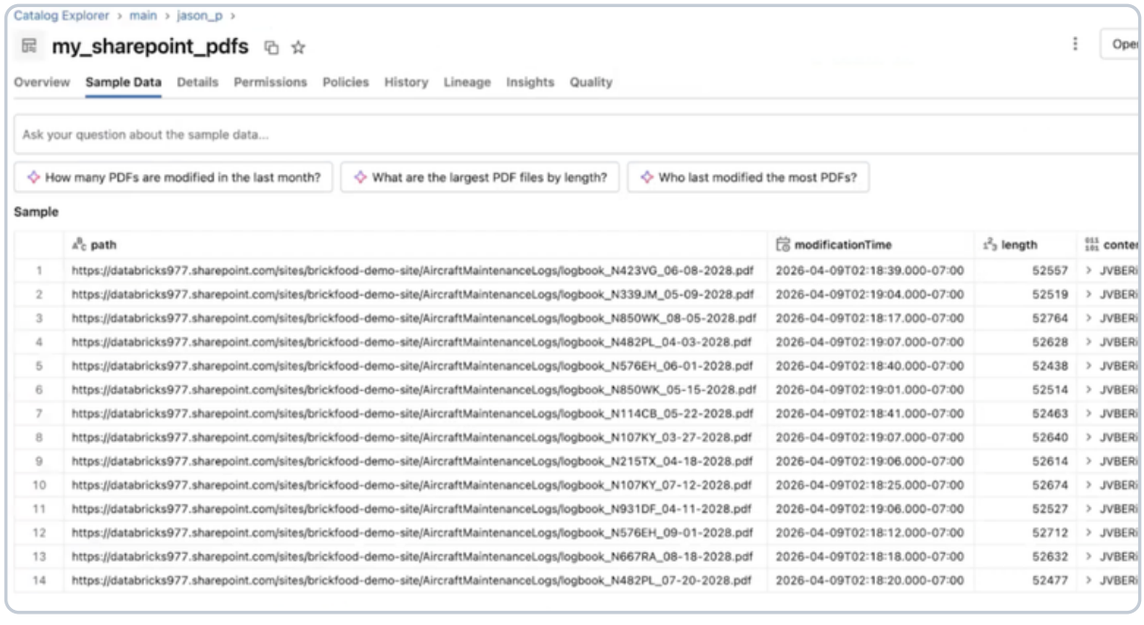
INGESTION SQL API

You can also define the ingestion pipeline directly in SQL within a Lakeflow Spark Declarative Pipeline. Just provide the URL to your folder of documents and reference an existing SharePoint Unity Catalog connection. If you don't have a connection yet, follow these steps to create one.

```
CREATE OR REFRESH STREAMING TABLE main.idp_demo.sharepoint_pdfs
AS SELECT *, _metadata FROM STREAM read_files(
  "https://mytenant.sharepoint.com/sites/MySite/Shared%20Documents",
  format => "binaryFile",
  `databricks.connection` => "my_sharepoint_conn",
  pathGlobFilter => "*.pdf"
);
```

The `STREAMING TABLE` keyword means the pipeline is incremental: each run only picks up files that are new or modified since the last run. The `binaryFile` format preserves the raw PDF content as binary data. The `_metadata` column captures additional file-level information from the source.

THE RESULT



The output is a table where each row represents one document. Key columns:

- `path:` the file path from SharePoint
- `modificationTime:` when the file was last updated
- `length:` file size in bytes
- `content:` the raw binary content of the PDF itself

Inspect the ingested table in Catalog Explorer to confirm what landed. You'll see one row per document, with the binary content and metadata described above.

Step 3: Parse and extract with Document Intelligence

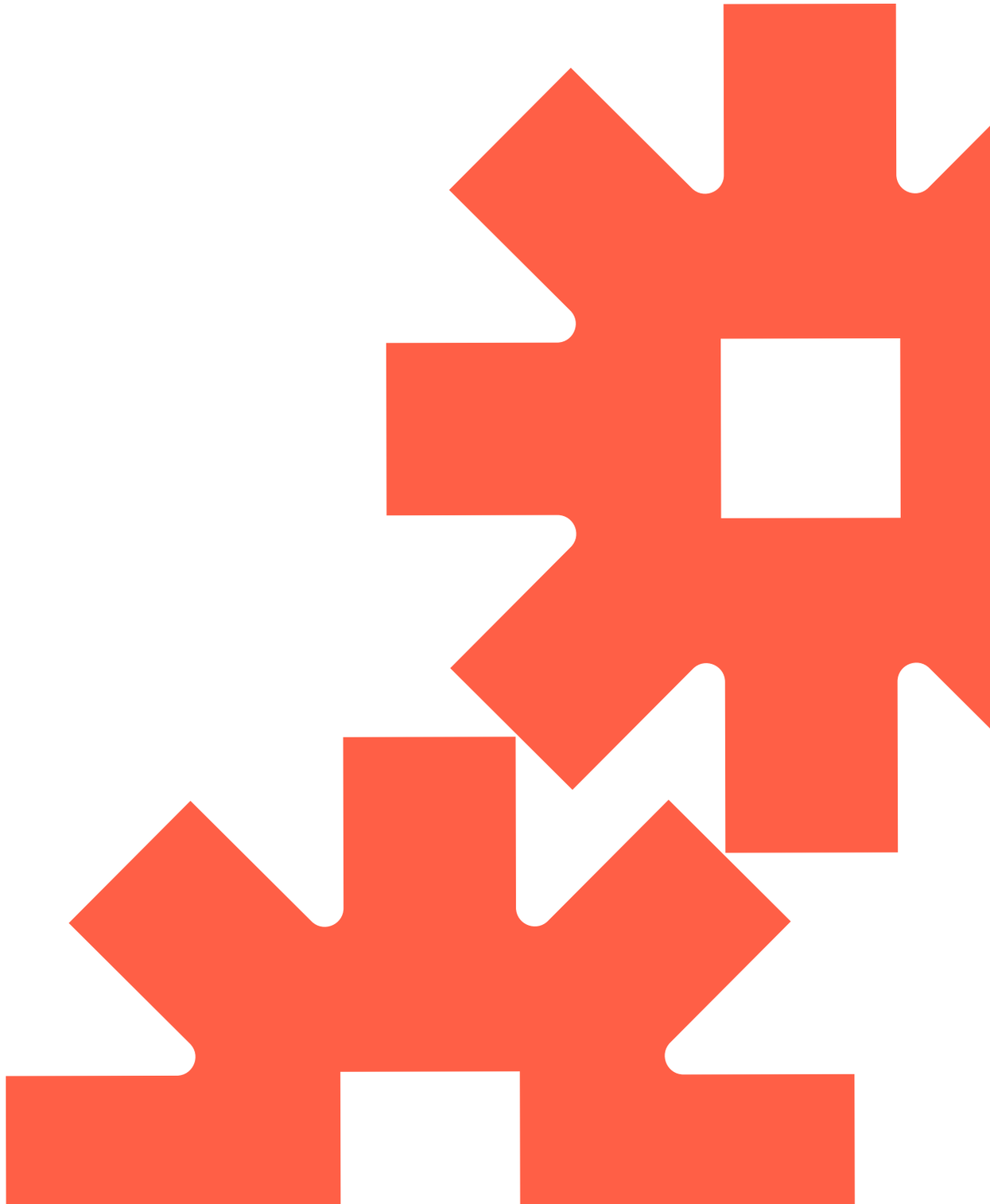
Document Intelligence on Databricks is powered by two AI Functions: `ai_parse_document` (converts raw PDFs into structured content) and `ai_extract` (pulls out specific fields you define). You can work with these functions in two ways:

1. **Document Intelligence UI:** A visual interface for configuring, previewing and iterating on your extraction. Define your schema in natural language, see results per document and refine until the output looks right. When you're done, Document Intelligence generates the `ai_parse_document` and `ai_extract` SQL for you automatically.
2. **SQL API:** Call `ai_parse_document` and `ai_extract` directly in SQL for full control over the pipeline logic.

The typical workflow is to start in the UI, get the extraction working, then continue iterating on the generated SQL (for example, to validate programmatically against ground truth data or to customize the pipeline logic).

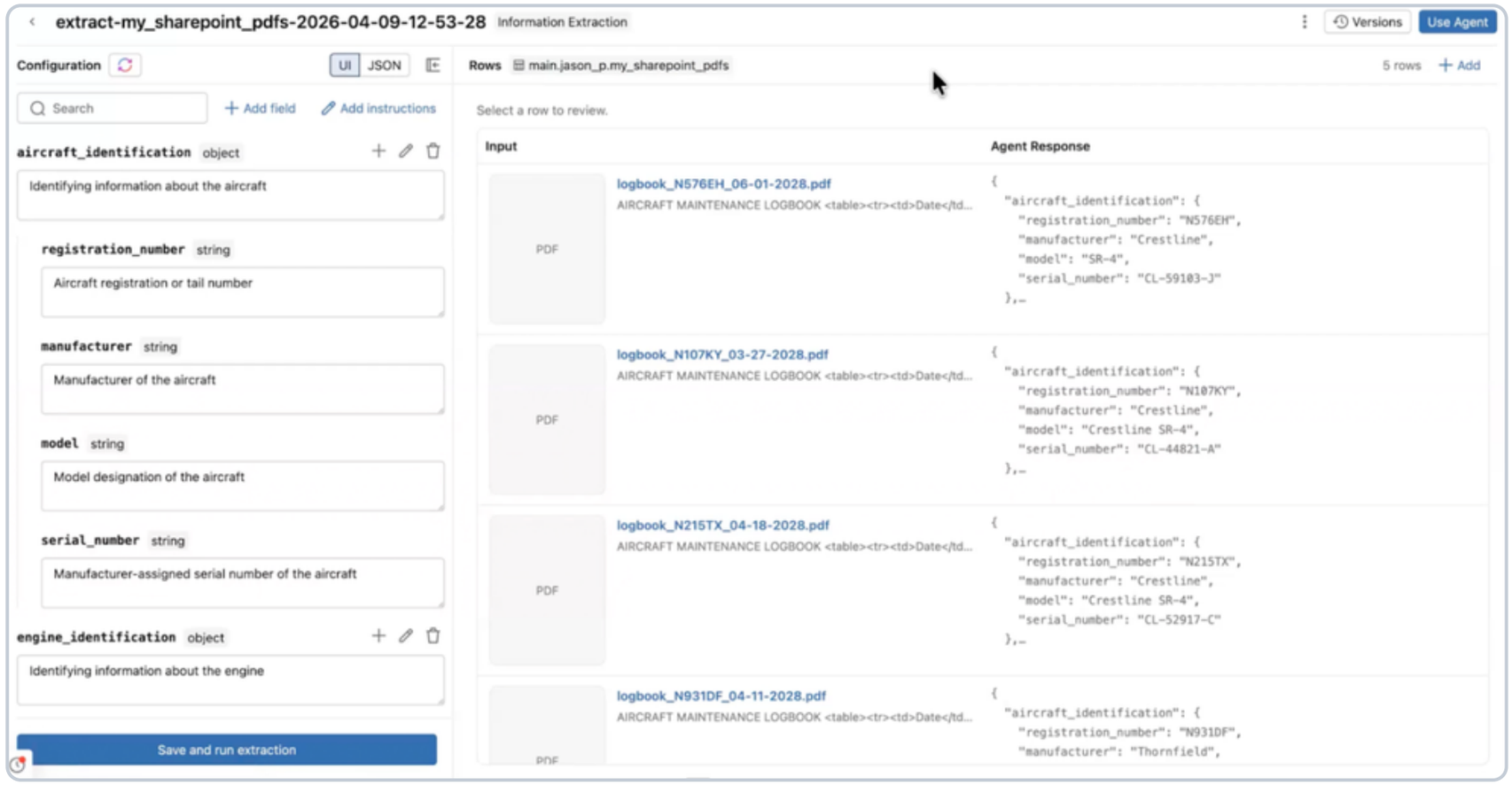
DOCUMENT INTELLIGENCE (UI)

1. Navigate to **Agents** in the Databricks sidebar and click **Create Agent.**
2. Filter by **unstructured** and select **Information Extraction.**
3. Select your ingestion table and choose the `content` column (the binary PDF data).
4. Click **Create Agent** to spin up the extraction environment.



1. Describe the information you want to extract in natural language (for example, "the date the work was performed, the name of the person who performed it, the total duration and any relevant identifiers"). Click **Generate Schema** and the agent converts your description into a structured extraction schema.
2. **Preview results** for each document. The agent shows the parsed PDF content alongside the extracted fields. Click through multiple files to verify consistency. Provide feedback to the agent to intelligently auto-update your schema and iterate on quality.

Once satisfied, productionize with a single click (covered in Step 4).



AI FUNCTIONS (SQL API)

Whether you're working with SQL generated by Document Intelligence or writing it from scratch, the code is straightforward.

Parse documents with `ai_parse_document`:

```
CREATE OR REFRESH STREAMING TABLE main.idp_demo.documents_parsed
TBLPROPERTIES (
  'delta.feature.variantType-preview' = 'supported'
)
AS
SELECT
  path,
  ai_parse_document(content, MAP('version', '2.0')) AS parsed_
content
FROM main.idp_demo.sharepoint_pdfs;
```

`ai_parse_document` takes the raw binary content of each PDF and research-backed techniques to read through the document: handwriting, printed text, tables, figures, mixed layouts. It returns a structured `VARIANT` output containing the full parsed content, page-level information and element-level details (text, tables, section headers, figures, captions, bounding boxes and more). One function call, any document format.

Extract fields with `ai_extract`:

```
CREATE OR REFRESH STREAMING TABLE main.idp_demo.documents_extracted
TBLPROPERTIES (
  'delta.feature.variantType-preview' = 'supported'
)
AS
SELECT
  path,
  ai_extract(
    parsed_content,
    '{
      "field_1": {"type": "string", "description": "Description of
field 1"},
      "field_2": {"type": "string", "description": "Description of
field 2"},
      "field_3": {"type": "number", "description": "Description of
field 3"},
      "field_4": {"type": "number", "description": "Description of
field 4"}
    }'
  ) AS extracted
FROM main.idp_demo.documents_parsed;
```

`ai_extract` takes the `VARIANT` output from `ai_parse_document` and applies a JSON schema you define. Replace the placeholder fields above with your actual extraction targets (dates, names, durations, identifiers or any fields relevant to your documents). The function returns a structured `VARIANT` with a `response` object containing the extracted values for each document.

Quick exploration (chained in a single query):

For ad-hoc exploration before productionizing, you can chain both functions in one query:

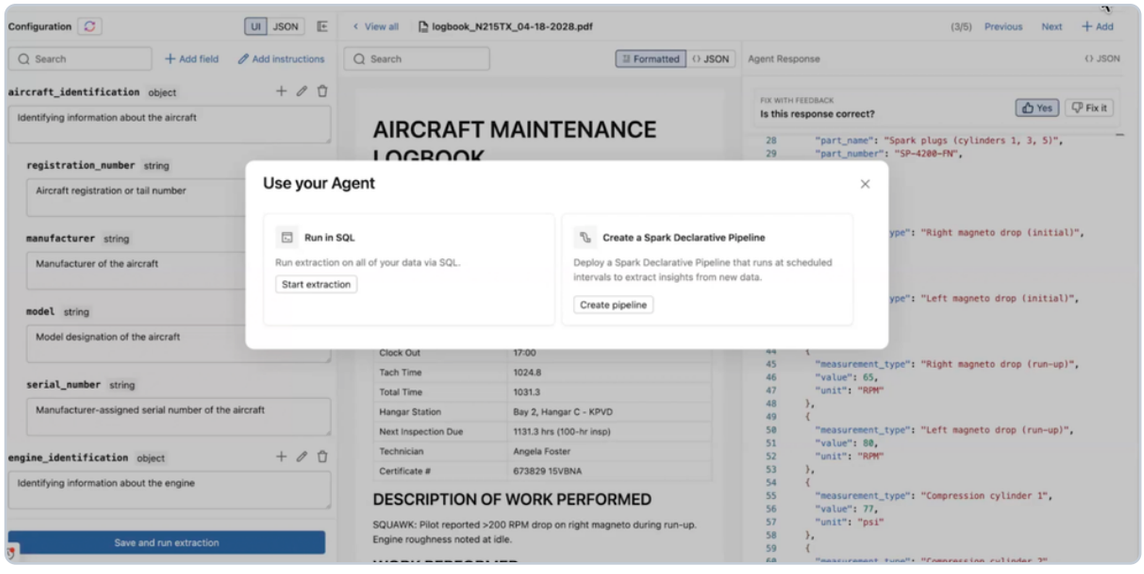
```
WITH parsed AS (
  SELECT
    path,
    ai_parse_document(content, MAP('version', '2.0')) AS parsed_
content
  FROM main.idp_demo.sharepoint_pdfs
)
SELECT
  path,
  ai_extract(
    parsed_content,
    '{
      "field_1": {"type": "string", "description": "Description of
field 1"},
      "field_2": {"type": "string", "description": "Description of
field 2"},
      "field_3": {"type": "number", "description": "Description of
field 3"}
    }'
  ) AS extracted
FROM parsed;
```

This is useful for testing your schema against a few documents before committing to a full pipeline.

Step 4: Productionize with a Lakeflow Spark Declarative Pipeline

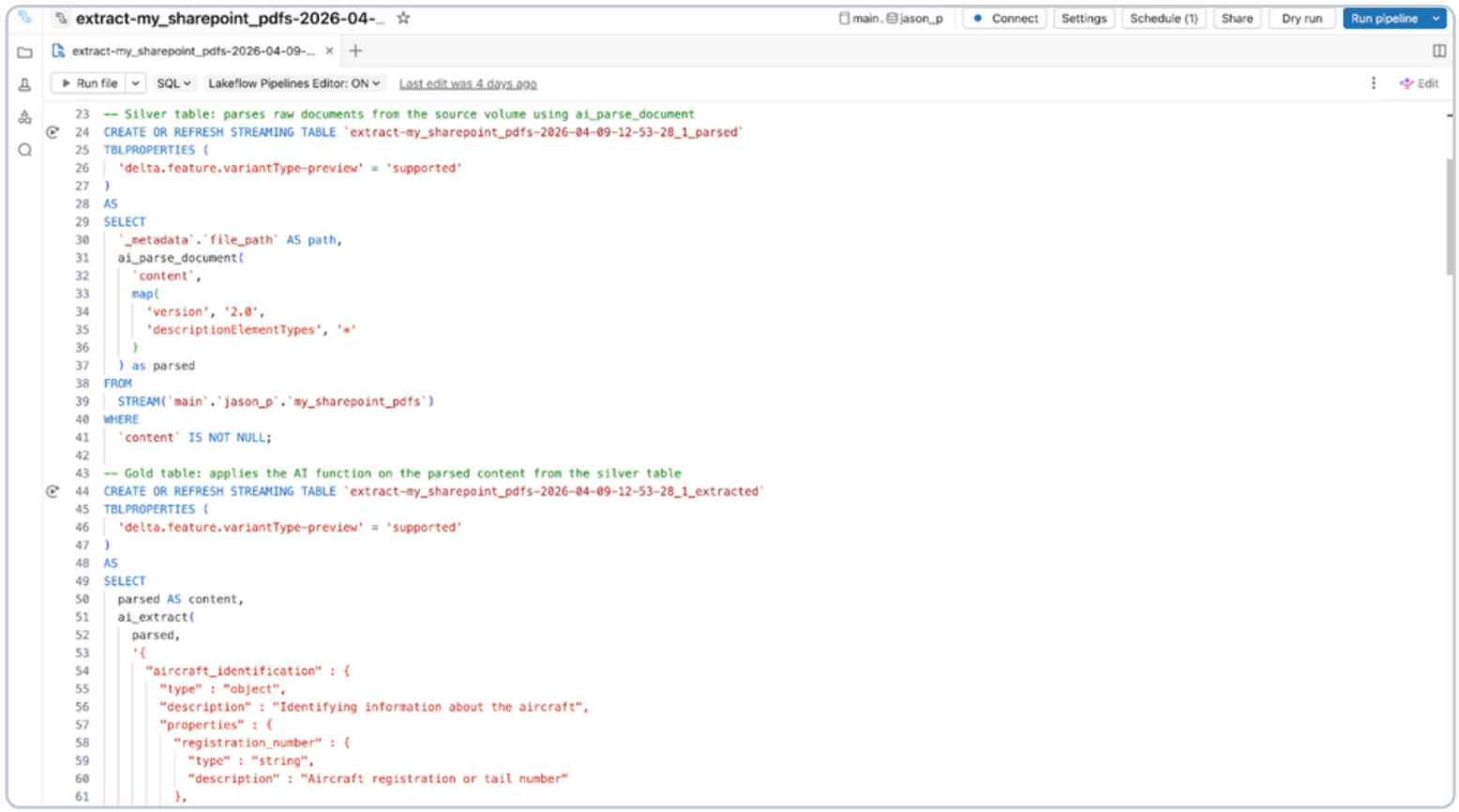
Whether you used Document Intelligence or wrote the SQL yourself, the next step is automation. This is where Spark Declarative Pipelines (SDP) really shine. Instead of manually writing hundreds of lines of procedural logic, you can focus on writing your business logic in SQL and SDP will automatically convert that into a managed production-grade pipeline. SDP handles the heavy lifting with efficient serverless compute and incremental processing by default while providing built-in observability.

FROM DOCUMENT INTELLIGENCE



If you used the Document Intelligence UI in Step 4, click Use Agent and select Lakeflow Spark Declarative Pipeline. Databricks auto-generates a complete pipeline with the SQL already written for you. No additional work needed.

FROM CODE



If you wrote the SQL yourself, those `CREATE OR REFRESH STREAMING TABLE` statements from Step 3 are already pipeline-ready. Place them in a Lakeflow Spark Declarative Pipeline, and you have a production pipeline.

WHAT THE PIPELINE DOES

Either way, the pipeline contains two STREAMING TABLEs:

- **Silver table** (`documents_parsed`)**:** Reads from the ingestion streaming table and runs `ai_parse_document` to convert raw binary PDFs into structured content.
- **Gold table** (`documents_extracted`)**:** Reads from the silver table and runs `ai_extract` to pull out the specific fields defined in your schema.

Both tables are incremental. Each time the pipeline runs, it checks for new rows in the ingestion table (new PDFs streamed in from SharePoint) and only processes those. Documents that have already been parsed are not re-parsed. Documents that have already been extracted are not re-extracted. This is what makes the pattern cost-effective at scale: you are not re-running AI models over documents you've already processed.

The result is a gold table with clean, structured data extracted from your PDFs, stored as a standard table in Unity Catalog that plugs into anything downstream.

Step 5: Use cases and dashboard

The gold table is a governed table with structured data extracted from your unstructured documents. What you build on top of it depends on your use case:

- **Chatbots and assistants (such as Databricks Genie or Databricks Knowledge Assistant):** Let teams ask natural language questions about the documents.
- **AI agents or agentic workflows:** Automate compliance checks, routing or classification based on extracted fields.
- **Search:** Build search applications over parsed document content.
- **Unified analytics:** Combine extracted document data with your existing structured data for cross-source reporting.
- **Dashboards:** Visualize extracted metrics for operational teams.

In the video on page 14, we show an AI/BI Dashboard as one example. The dashboard is built on top of the gold table and surfaces visualizations (activity over time, workload distribution, compliance metrics), all powered by data extracted from those original PDFs. Because it's an AI/BI Dashboard, users can ask questions in natural language without needing to know SQL.

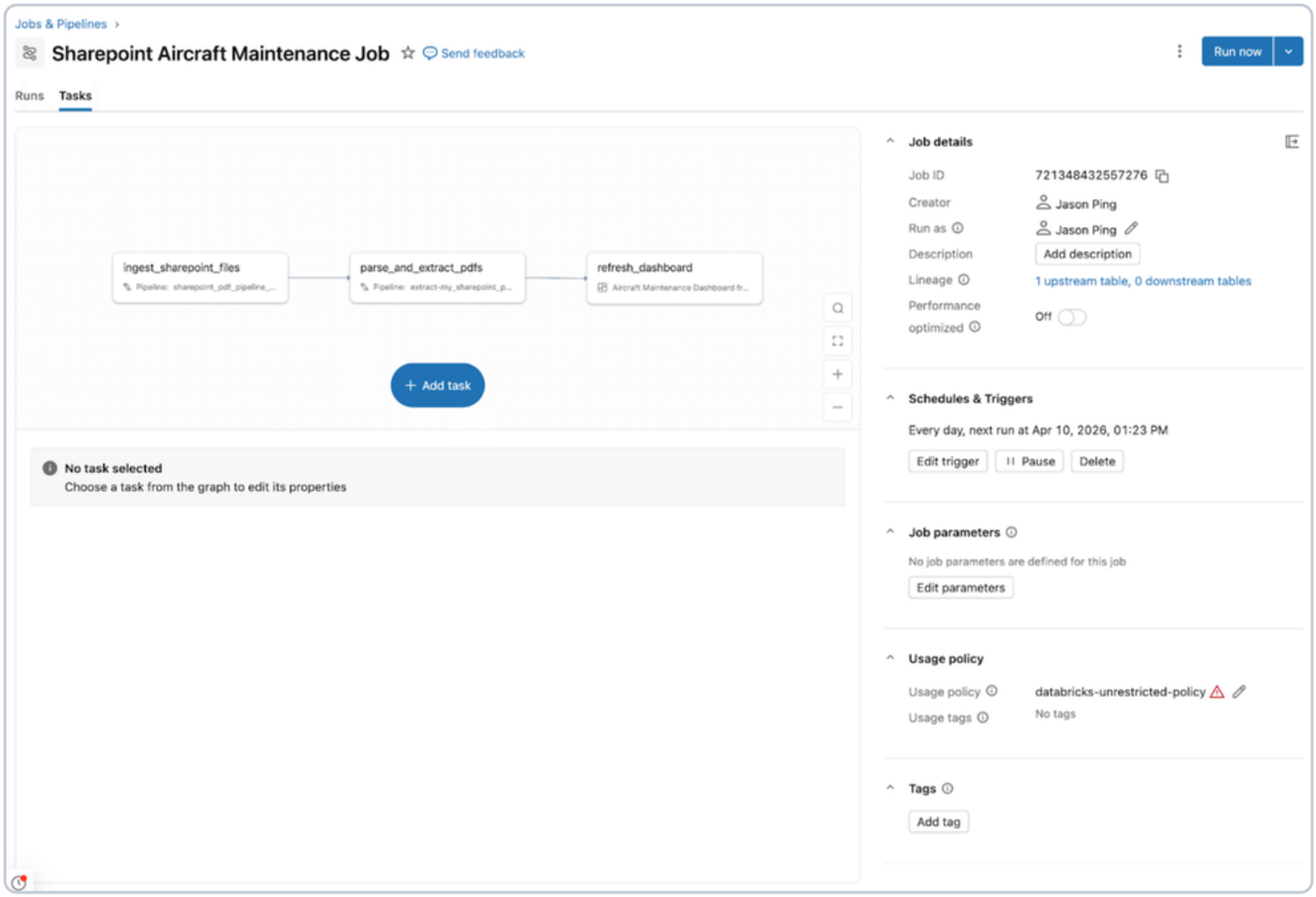
The dashboard is just one starting point. The extracted data table is the foundation.

Step 6: Orchestrate with Lakeflow Jobs

The final step is tying everything together into one automated workflow using Lakeflow Jobs.

Create a job with three tasks chained in sequence:

1. **Ingest:** Run the Lakeflow Connect pipeline. It checks SharePoint for new or modified files and incrementally ingests them.
2. **Parse and Extract:** Run the Spark Declarative Pipeline. It parses the newly ingested documents and extracts the defined fields.
3. **Refresh Dashboard:** Update the AI/BI Dashboard so it reflects the latest data.



Each time this job runs, the entire chain executes end-to-end. If no new files have arrived in SharePoint, the ingestion step completes quickly and nothing downstream re-processes. If ten new PDFs landed, only those ten flow through parsing and extraction.

Set a daily trigger, configure failure notifications so the right people know if something breaks, and that's it. Set it and forget it. No custom orchestration code, no fragile scripts to maintain.

Conclusion

Let's review the full end-to-end flow of the solution, which automates everything from ingestion to final insights:

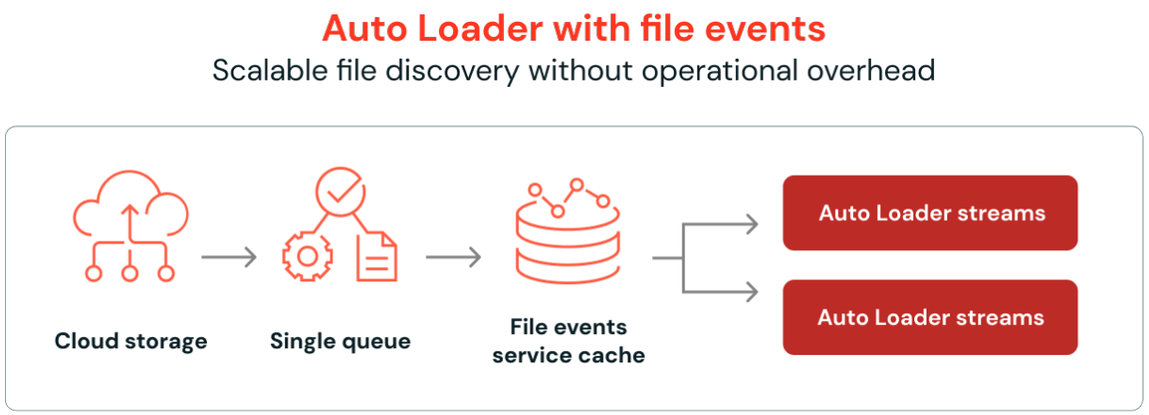
4. Select unstructured documents from SharePoint as the source.
5. Ingest using Lakeflow Connect. Simple UI, fully managed, incremental ingestion.
6. Parse and extract structured fields using AI Functions and Document Intelligence. Interactive preview, no external parsing tools.
7. Productionize into a Spark Declarative Pipeline. Auto-generated or hand-written, fully incremental at every step.
8. Build downstream use cases: dashboards, chatbots, agents, search, analytics.
9. Orchestrate the entire flow with Lakeflow Jobs. One automated workflow, one daily trigger.

No custom ingestion code. No external parsing services. No manual orchestration. The entire pipeline, from PDF to dashboard, runs on Databricks.

To get started, check out the Lakeflow Connect SharePoint documentation, the ai_parse_document reference, and the ai_extract reference. Start with a single document type, prove the pattern, and expand from there.

Auto Loader With File Events — Simplified File Discovery at Scale

By Murali Talluri and Yogesh Singh



Summary

- Learn how Auto Loader with file events simplifies cloud storage ingestion by eliminating the need to choose between directory listing simplicity and classic notifications performance.
- Discover the architectural differences and benefits of File Events compared to classic notifications, including simplifying permissions, mitigating cloud provider limits and automatically managing resources.
- Learn how to enable File Events, when to migrate and best practices to achieve optimal file discovery with file events.

Databricks customers rely on Auto Loader to access cloud storage data at scale. Now, file events are taking Auto Loader to the next level — delivering the speed and efficiency of incremental, notification-based processing alongside simplified setup, automated lifecycle management and seamless scalability. The result is faster ingestion, less operational overhead and infrastructure that scales as your data grows.

Challenges with cloud storage ingestion

Historically, Databricks customers have chosen between two modes of Auto Loader. Directory listing is simple to set up but can be expensive and slow for high file volumes, since it requires repeatedly scanning the cloud storage directory to identify any new files. Classic notifications are more efficient, but they require permissions for setting up notification resources (like SQS or Event Grid), in addition to those granted via Unity Catalog.

Cloud providers also limit the number of event notifications per single storage container. Since each Auto Loader job with classic notifications requires its own queue/subscription, you are capped at 100 jobs on AWS/GCP and 500 on Azure. Finally, customers must manually manage the lifecycle of these resources as streams are created or deleted.

To orchestrate these pipelines, many customers use file arrival triggers, which automatically start the pipeline's job when new files arrive in cloud storage. However, file arrival triggers have historically relied on directory listing, so they could only support cloud storage directories with fewer than 10,000 objects.

Introducing Auto Loader with file events

File events eliminate these hurdles by managing cloud notification infrastructure for you—providing the simplicity of directory listing with the performance of classic notifications. File events also enable scalable file arrival triggers, allowing you to automatically run jobs based on cloud storage directories of any size.

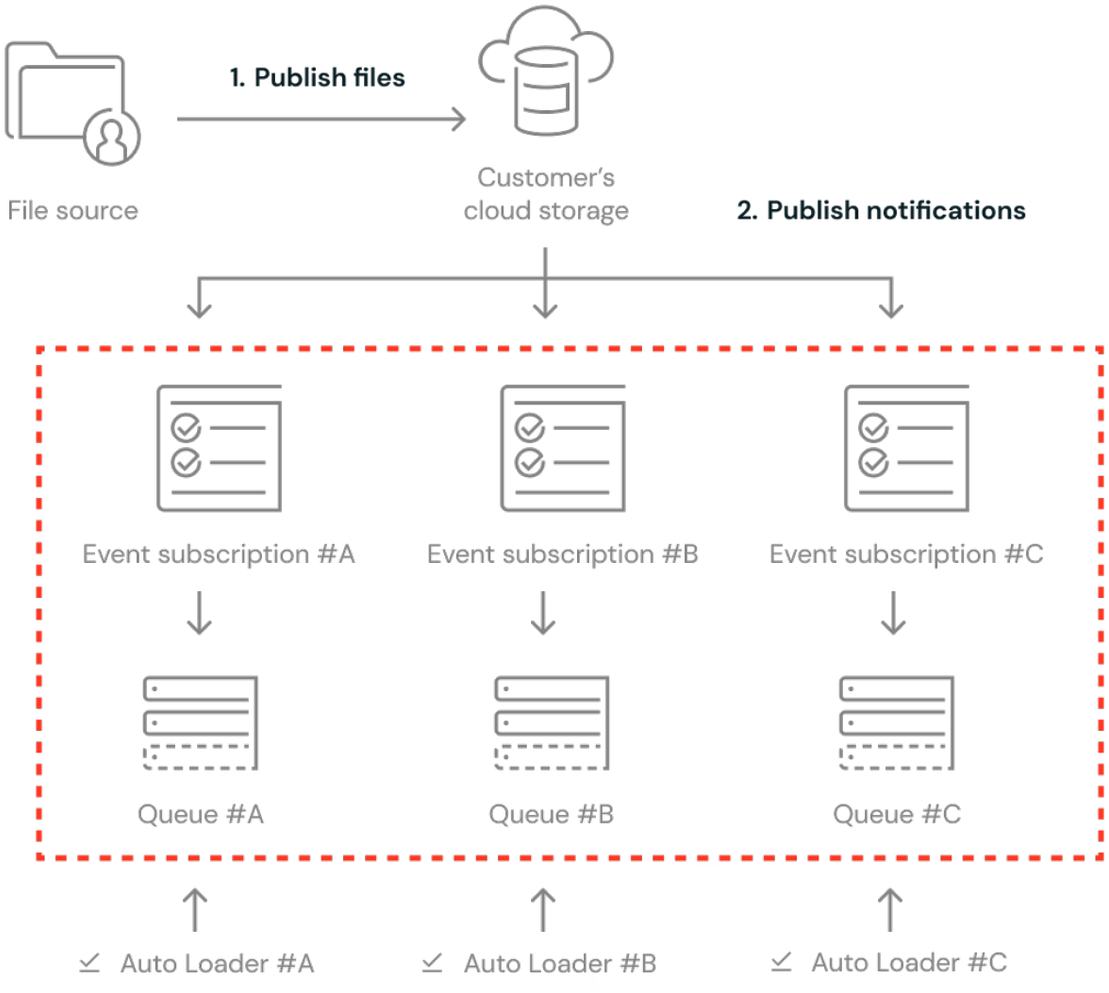
When file events are enabled on an external location, a Databricks-managed service creates notification resources, then reads and caches notifications. Auto Loader incrementally discovers new files from this cache using an efficient API that resumes from the last processed file.

Because this service uses permissions granted via Unity Catalog, users don't have to configure notification infrastructure, tune complex parameters, or manage queue and subscription lifecycles. And by using a single queue for the entire external location, rather than for each Auto Loader stream, it's easier to avoid per-bucket notification limits.

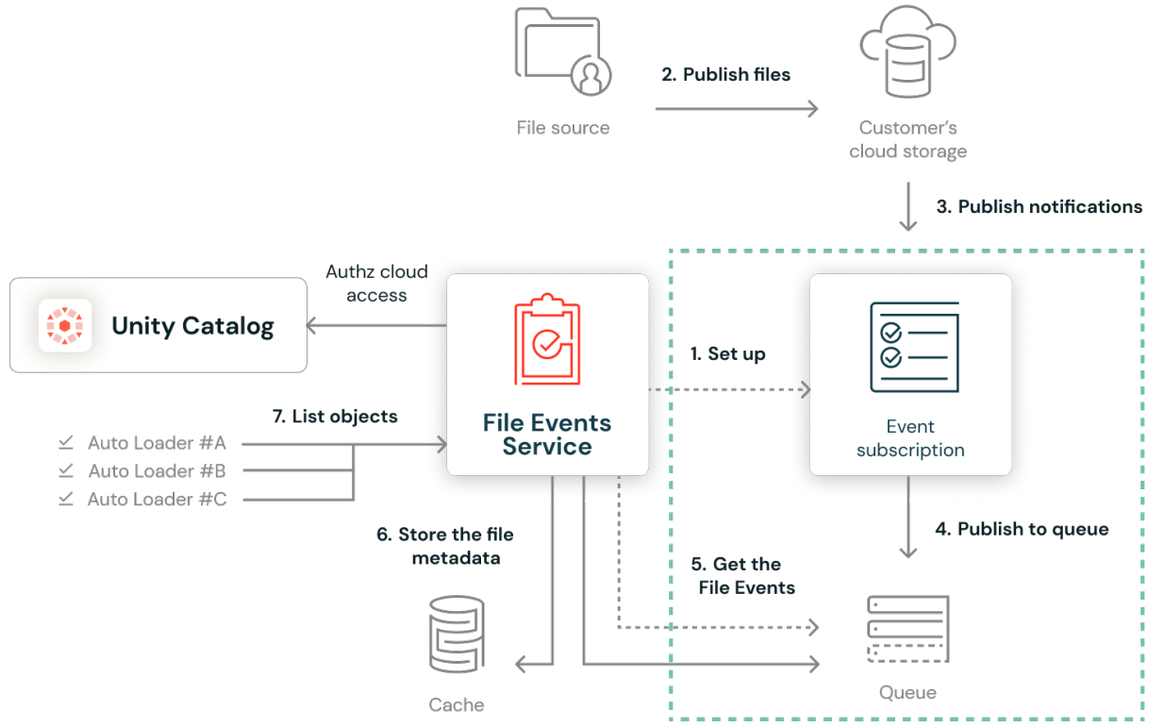
High-level architectural design comparison

HIGH-LEVEL AUTO LOADER ARCHITECTURE WITH CLASSIC NOTIFICATIONS

With classic notifications, each Auto Loader stream or job requires a separate subscription and Queue. This setup is required even if all these streams point to the same storage container or bucket.



HIGH-LEVEL AUTO LOADER ARCHITECTURE WITH FILE EVENTS



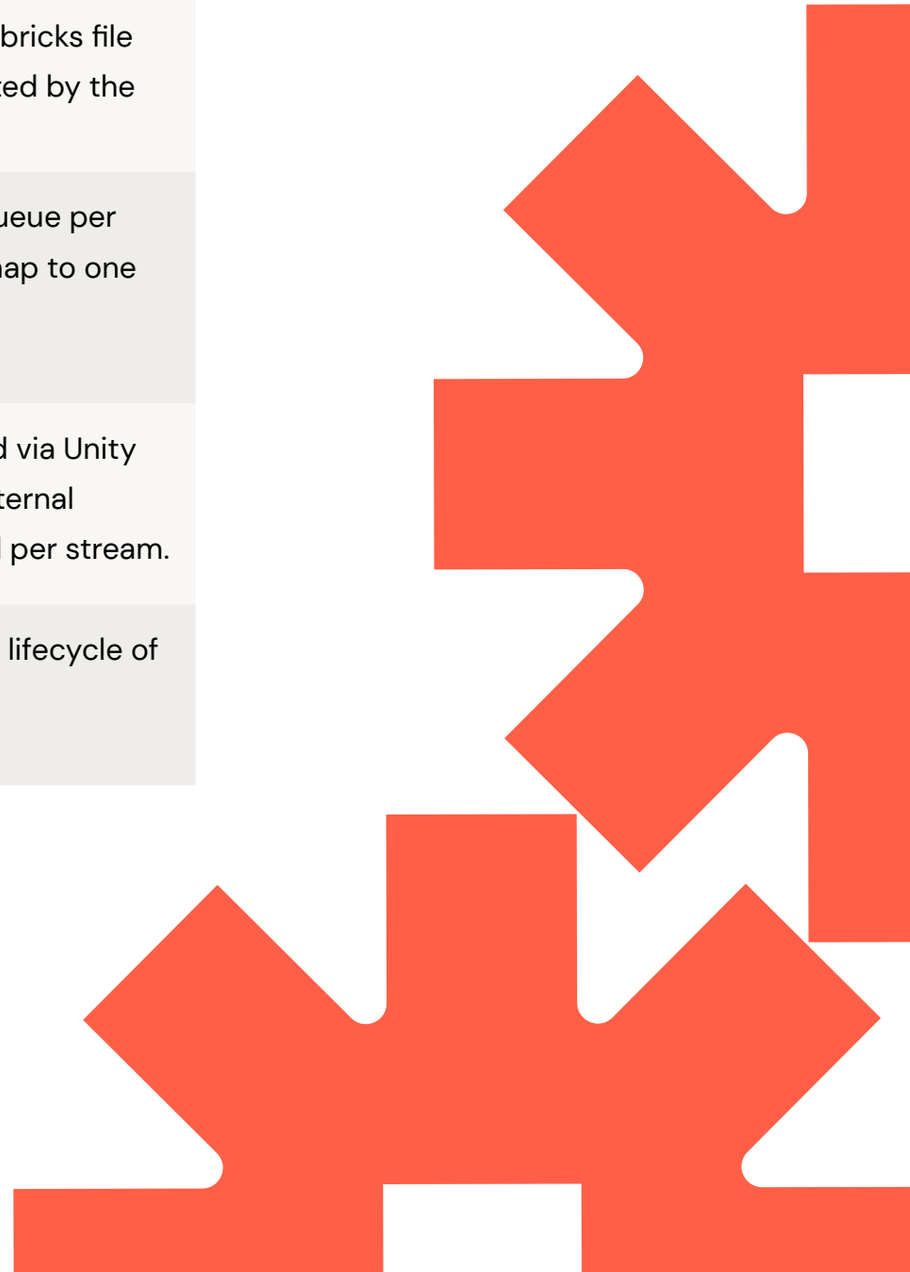
File events architecture deep-dive

When you enable `cloudFiles.useManagedFileEvents`, the system automates the heavy lifting of ingestion.

1. **Set up:** When you enable file events on an external location, Databricks creates the necessary cloud subscriptions and queues.
   a. All new external locations have file events enabled by default. When enabling file events on the external location, the storage credential backing that location should have permissions to create cloud subscriptions and queues. If the required permissions are not granted, the user will receive an error. Refer here for the required permissions.
   b. For simplicity, Databricks recommends using "managed" queues (where the user gives permissions to storage credentials to set everything up). However, you can also bring your own queue — in which case the storage credential should have permissions to read from the queue.
2. **Publish files:** This is ingesting files that need to be processed, into your cloud storage location.
3. **Publish notifications:** Enabling file events automatically creates the subscription service (SNS or Event Grid or Pub/Sub) that gets the notifications published to.
4. **Publish to queue:** Events from subscription service get published to Queue.
5. **Get file events:** The Databricks file events service reads messages from Queue.
6. **Store file metadata:** The messages read from the queue gets cached.
7. **List objects:** When an Auto Loader stream is run with cloudFiles.useManagedFileEvents set to true, it incrementally reads events from the file events cache and processes them.
   a. If the Auto Loader stream is running for the first time, it does directory listing on the load path to cache the list of existing files. It runs incrementally thereafter.
   a. To ensure no files are missed in the future, it also periodically performs directory listing on the external location. (Since this is managed automatically, the Auto Loader setting cloudFiles.backfillInterval is no longer needed, so this config is ignored.)

Comparison between classic notifications and file events

Feature	Classic Notifications	File Events
Cloud Infrastructure	Each Auto Loader stream sets up its own dedicated cloud notification resource (e.g., an SNS topic and SQS queue per stream).	Databricks sets up a single notification resource (queue/subscription) per external location.
Notification Service	Auto Loader streams read directly from their dedicated cloud queue.	Auto Loader streams read from a Databricks file events service cache, which is populated by the single cloud queue.
Cloud Resource Limits	Limited by cloud provider caps on the number of notification resources per storage container (e.g., 100/500 per container/account). One stream maps to one queue.	Avoids cloud limits by using a single queue per external location. Many streams can map to one queue.
Permissions	Requires elevated permissions for each stream to create and manage its dedicated cloud notification resources.	Requires elevated permissions granted via Unity Catalog's storage credential for the external location. No extra permissions needed per stream.
Lifecycle Management	Users often have to manually manage notification resources (queues/subscriptions) when streams are deleted or fully refreshed.	Databricks automatically manages the lifecycle of the cloud notification resources.



In Practice: How Auto Loader with File Events supports customers

PERFORMANCE AND SIMPLICITY

- **Performance meets simplicity:** You no longer need to decide between the efficiency of classic notifications and the simplicity of directory listing. Auto Loader with File Events offers both.
- **Eliminate costly/slow directory listing:** If you're using Auto Loader in directory listing mode today, you should see significant performance improvements by migrating to File Events. This is because Auto Loader stream doesn't need to regularly perform directory listing on the load path to discover new files.
- **Zero tuning required for file discovery:** You no longer need to tune the various settings regarding file discovery (e.g., in classic notifications, fetchParallelism, backfillInterval, etc.).

PERMISSIONS

- **No extra permissions per stream:** The service principal or user who runs the Auto Loader stream for the first time doesn't need to have permissions to create subscriptions/queue. They should just have permissions on the input data path to read from.

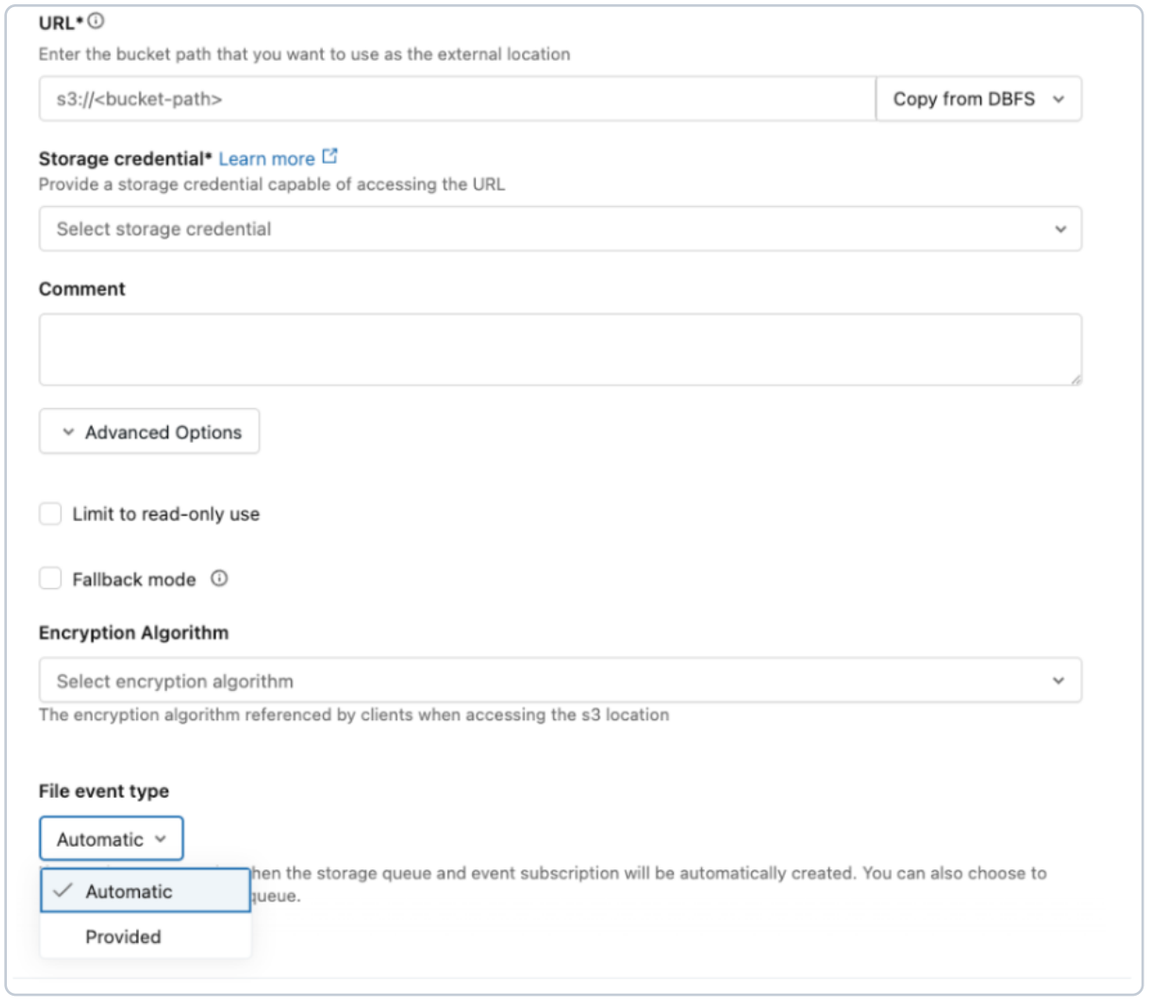
OPERATIONAL OVERHEAD:

- **One queue per external location, not per stream:** Unlike classic notifications, you no longer need to create a queue for each Auto Loader stream; this helps you avoid hitting notification limits on storage containers. We set up just one queue and storage event subscription per external location.
- **Automatic cloud resource lifecycle management:** You no longer need to worry as much about the lifecycle of event subscriptions and queues that are created in the cloud (e.g., when an Auto Loader stream is deleted or fully refreshed). These resources are deleted automatically (the storage credential has permissions) once file events are disabled on the external location.

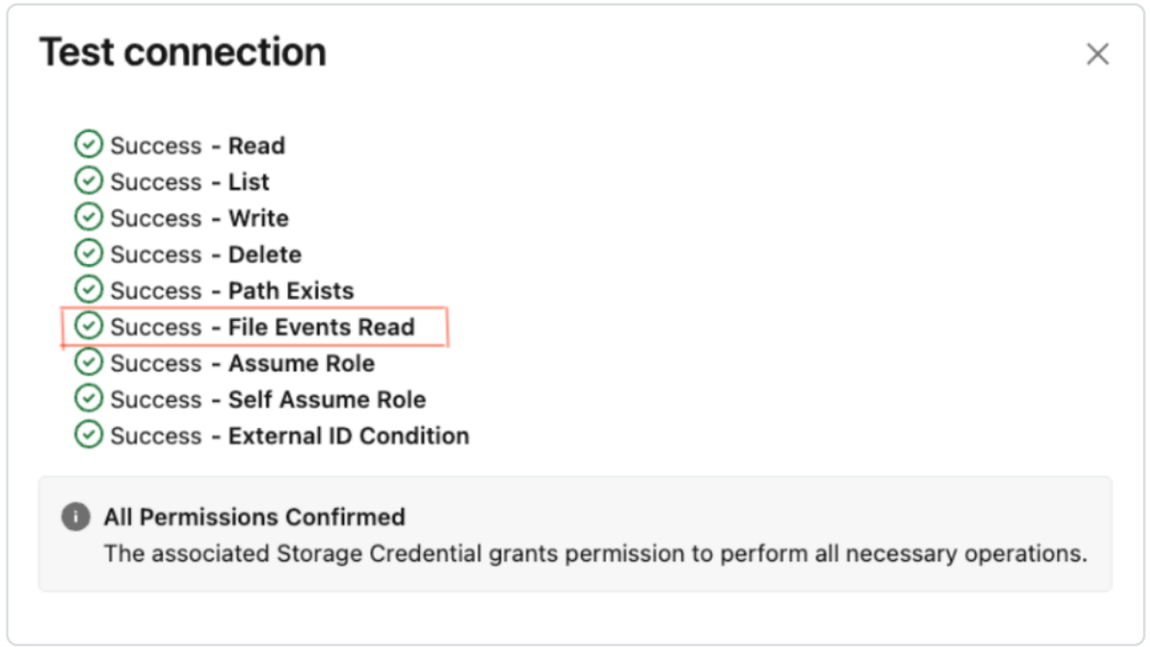
Refer here for how to migrate from classic notifications to file events.

Setting up File Events: Quick guide

- All new external locations have file events enabled by default. For existing external locations, you can go to the edit page or create a new external location and select "**Advanced Options**."



- Select the **File Event Type**. If you'd like Databricks to set up subscriptions and events for you, then choose "**Automatic**" (recommended). If you've configured a queue yourself, use "**Provided**." Click create.
- Lastly, after a few seconds, click "**Test connection**" in the external location page to confirm that file events have been enabled successfully.



Examples

Here is an example of an Auto Loader stream using file events:

```
stream_df = (spark.readStream.format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("cloudFiles.maxFilesPerTrigger", 5000)
 .option("cloudFiles.schemaLocation", f'{chekpoint_path}/schema')
 .option("cloudFiles.useManagedFileEvents", True)
 .load(f'{volume_path}')
 .writeStream
 .queryName("file-events-stream")
 .option("checkpointLocation", f"{chekpoint_path}")
 .trigger(processingTime='30 seconds')
 .table(f"{target_table}")
)
```

If you're using Spark Declarative Pipelines today and already have a pipeline with a STREAMING TABLE, update it to include the cloudFiles.useManagedFileEvents option.

```
CREATE OR REFRESH STREAMING TABLE <table-name>
AS SELECT <select clause expressions>
    FROM cloud_files("abfss://path/to/external/location/or/volume",
                     "<format>",
                      map(
                           ...
                          "cloudFiles.useManagedFileEvents", "True",
                          ...))
```

Setting this as spark conf:

```
spark.conf.set("spark.databricks.cloudFiles.useManagedFileEvents",
"true")
```

Best practices

Create volumes for multiple subdirectories in the same external location

- If you have multiple subdirectories under the external location, and if you have multiple Auto Loader streams consuming from each of these sub-directories, create and use volumes for each of these subdirectories for faster file discovery with file events. Refer to: Use volumes for optimal file discovery
- Note: If you have just one Auto Loader stream consuming from the entire external location, this does not apply.

Use file arrival triggers to optimize your ingestion jobs

- You can trigger your Auto Loader job to run continuously, using file arrival triggers, or on a schedule.
- File arrival triggers are often the best choice because they run the job only when new files appear in cloud storage—enabling truly event-driven orchestration to complement your incremental ingestion.
- File events make file arrival triggers significantly more performant:

	File Events NOT Enabled	File Events Enabled
File tracking limit	Up to 10k files on the directory	Unlimited
# File arrival triggers per workspace	Maximum of 50	Unlimited

Conclusion

- If you are using Auto Loader with directory listing today, we recommend migrating to file events.
- If you are using Auto Loader with classic notifications today with < 2,000 files ingested per second, we recommend migrating to file events.
- File events are generally available on AWS, Azure and GCP.

A Practitioner's Guide to Real-Time Mode on Spark Declarative Pipelines

By Anant Pingle and Jerry Peng

Build a sub-second operational dashboard on live streaming data, starting with a live Kafka stream.

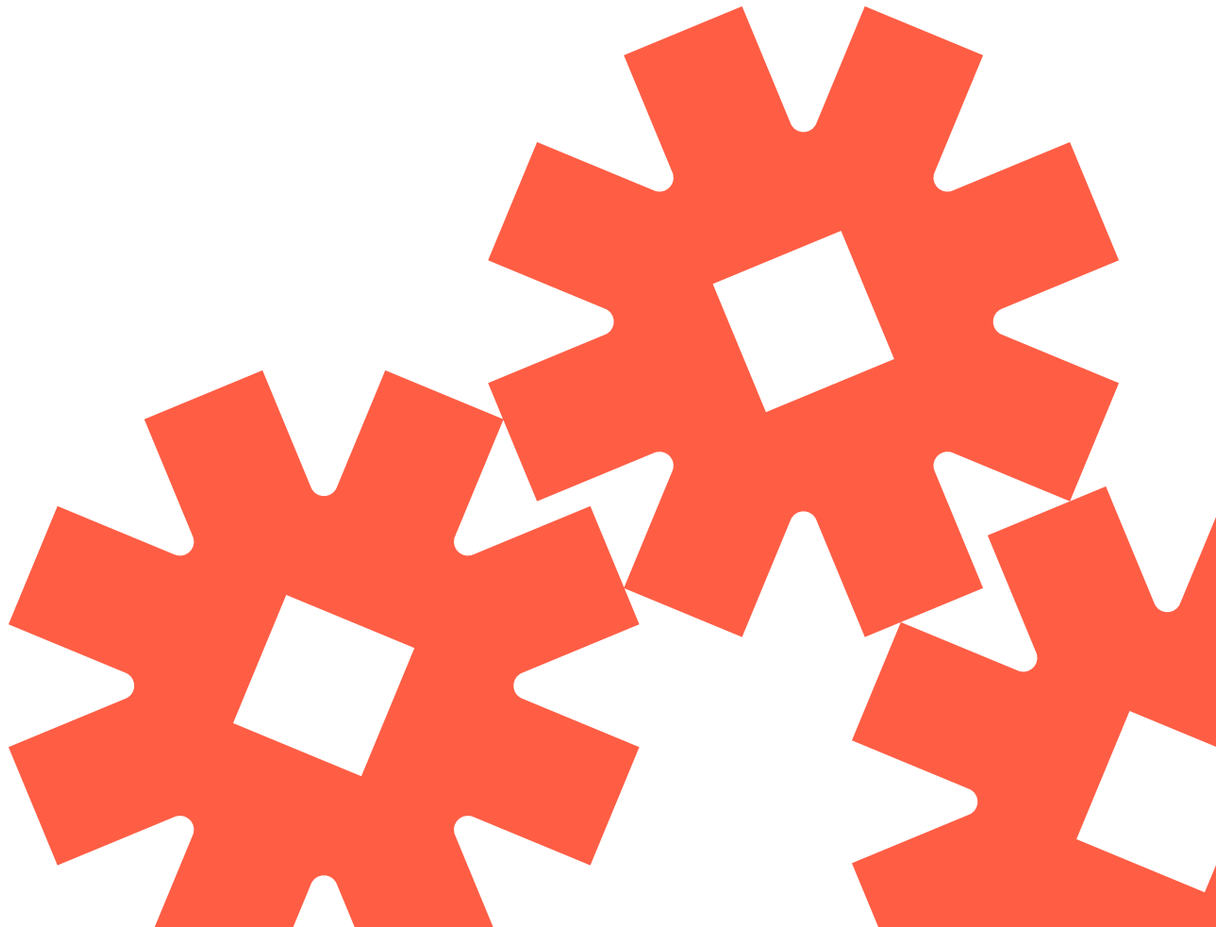
This section is a full end-to-end walkthrough for practitioners on how to build real-time applications using Apache Spark™ Declarative Pipelines, Lakebase and Databricks Apps. In June 2026, Databricks announced exciting news for anyone building streaming pipelines: **Real-Time Mode (RTM) for Spark Declarative Pipelines is now in Public Preview.** This means you can build continuous pipelines with end-to-end latencies as low as five milliseconds, without the complexity and cost of managing separate engines.

With Spark Declarative Pipelines, you simply **declare the flows you want, and the framework keeps them running correctly:** handling orchestration, checkpoints, retries and state so your code focuses on business logic rather than streaming plumbing.

Spark Declarative Pipelines (SDP) did not previously offer **operational, sub-second latency.** The default execution mode is excellent for high-throughput ETL workloads that can tolerate latencies ranging from a few seconds to minutes, but not for in-the-loop workloads where a decision must land in tens of milliseconds, such as fraud scoring, live personalization or real-time alerting. For those use cases, teams were forced to add a specialized engine, such as Apache Flink, to meet their needs, which inherited a whole stack to run alongside it.

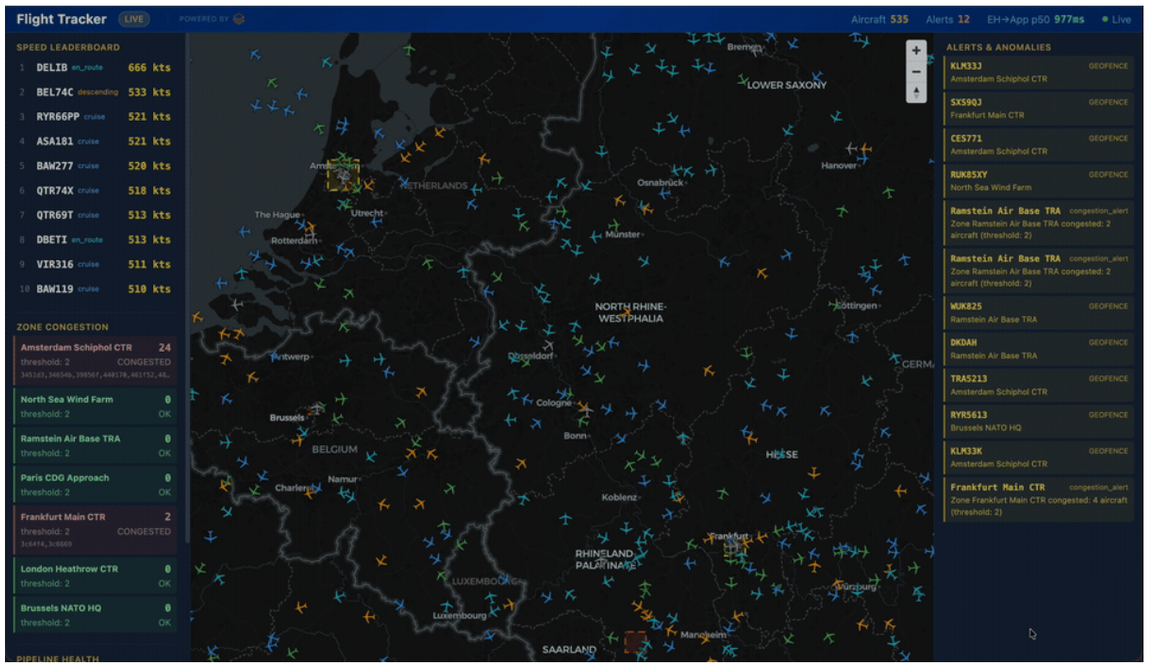
Real-Time Mode eliminates this added complexity. It targets p99 latency in the tens to low hundreds of milliseconds, and now you get it *inside* the declarative model: the same sinks-and-flows authoring surface, on serverless. You keep "declare what, not how" and gain real-time speed in one engine.

In this section, we're diving deep into what this new capability means for practitioners. To show RTM on SDP in action, we'll walk through a demo that tracks aircraft positions, demonstrating how you can easily build, monitor and scale sub-second operational workloads inside the Spark Declarative Pipelines you already know.



Demo: An end-to-end operational pipeline from Kafka to Lakebase

In this demo, live aircraft positions stream into **Apache Kafka**. A **Spark Declarative Pipeline** running with **Real-Time Mode** reads the stream, enriches each position (a status column, an alert flag, a zone label) and counts how many aircraft are inside each monitored zone at once, raising an alert when a zone gets crowded. Both outputs are written to **Databricks Lakebase** (managed Postgres), which a **Databricks App** reads to render a live map. This is an end-to-end flow: a plane moves, and the dot on the map moves. The aviation details are just the use case; what we are really showcasing is the code.



What we built: An end-to-end operational pipeline from Kafka to Lakebase

The programming model: sinks + update flows

Real-Time Mode pipelines are authored with the `pyspark.pipelines` API (imported as dp). Two primitives do the work, and notice what is *absent*: no `writeStream`, no `awaitTermination`, no checkpoint paths. You declare; the framework runs it.

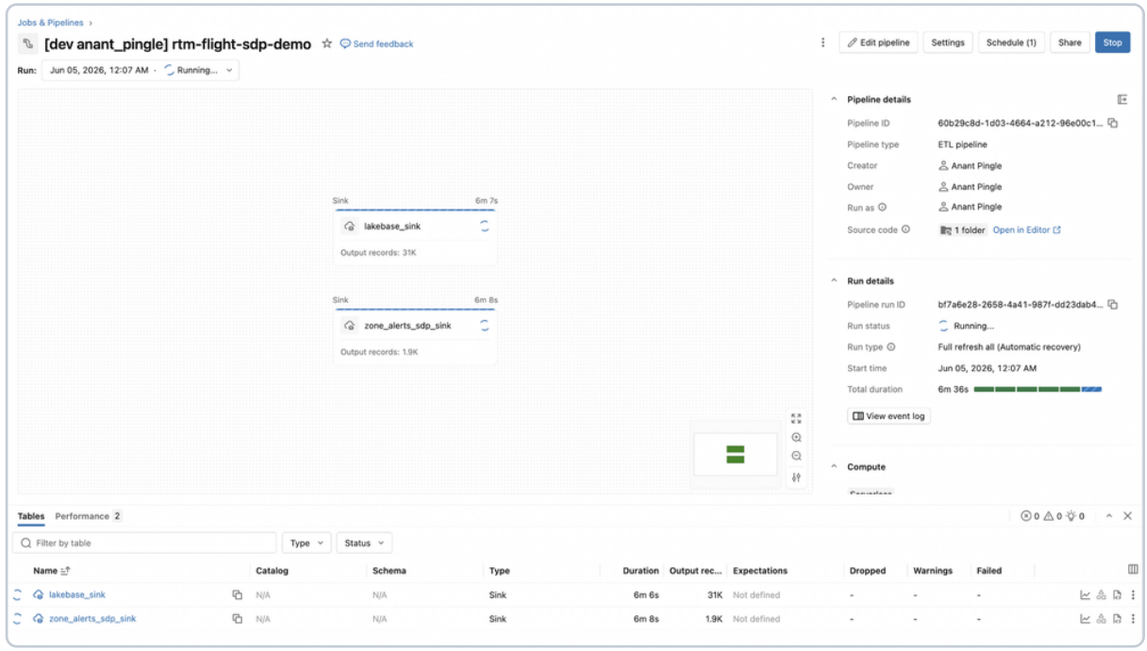
1. **Declare an external sink.** Real-Time Mode is built for operational delivery, so rather than landing results in a table for analytics, you write straight to the operational system your application reads from. Here we write to Databricks Lakebase through a native external sink:

```
from pyspark import pipelines as dp

# Native Lakebase sink (Private Preview). The configuration API
might change
# before GA, so the exact options are omitted here.
dp.create_sink(
    name="lakebase_sink",
    format=...,      # native Lakebase sink (Private Preview)
    options={...},   # connection + target table
)
```

2. **Define an update flow that targets the sink.** An `@dp.update_flow` returns a streaming DataFrame, and its rows are routed to the named sink. The flow is where you turn RTM *on,* through the flow-level Spark config `pipelines.trigger: "RealTime"`. The next two sections show two flows that do exactly this, one stateless and one stateful.

   Three things to internalize:

   - **pipelines.trigger: "RealTime"** is the switch. Combined with the pipeline-level `spark.databricks.streaming.realTimeMode.enabled`, this moves the flow onto the continuous engine.
   - **pipelines.trigger.interval is *not* a micro-batch size.** In RTM the batch is long-running, and data is processed as it arrives. The interval governs how often state and source offsets are checkpointed, not how often results appear.
   - The pipeline itself is configured `serverless: true`, `continuous: true`, on the `PREVIEW` channel.



That is the whole shape: **declare sinks, write flows, flip the trigger.** Everything that used to be plumbing (checkpoints, recovery, execution order) stays the framework's job. You stay in business logic.

The stateless flow: enrich each event

The first flow reads rows from Kafka, parses the JSON, drops incomplete rows and adds a few derived columns: a flight-phase label, an alert flag for emergency transponder codes and a zone label. It is ordinary DataFrame code, and Real-Time Mode does not change how you write it. The essence, with the Kafka boilerplate trimmed:

```
@dp.update_flow(
    name="positions_flow",
    target="lakebase_sink",                          # the Lakebase
sink declared above
    spark_conf={"pipelines.trigger": "RealTime",  # turn RTM on
                "pipelines.trigger.interval": "5 minutes"},
)
def positions_flow():
    return (
        spark.readStream.format("kafka")
            .option("subscribe", "raw-flights")
            # ... bootstrap servers + SASL auth ...
            .load()
            .select(from_json(col("value").cast("string"), RAW_
SCHEMA).alias("d"))
            .select("d.*")
            .filter(col("lat").isNotNull() & col("icao24").
isNotNull())
            # a few stateless transformations
            .withColumn("flight_phase",
                when(col("vert_rate") >  500, "climbing")
                .when(col("vert_rate") < -500, "descending")
                .when(col("baro_alt") > 30000, "cruise")
                .otherwise("en_route"))
            .withColumn("squawk_alert",
                when(col("squawk").isin("7500", "7600", "7700"),
"emergency"))
            .withColumn("zone", zone_label())     # which geofence
box, if any
    )
```

Nothing here is RTM-specific. The same DataFrame you would write for a micro-batch flow runs unchanged. The only thing that made it real-time was the pipelines.trigger: "RealTime" line in the decorator.

Going stateful: `transformWithState` in RTM

The second flow answers a question you cannot answer from any single record: *how many aircraft are inside each monitored zone at the same time, and when does a zone become too crowded?* A zone here is just a geographic box, like a sector an air-traffic controller watches. Answering this means remembering which aircraft are currently in each zone, so it needs state. Real-Time Mode supports the Arbitrary Stateful Processing API, `transformWithState`, through the `StatefulProcessor` class.

We built this as a **single keyed stateful operator**: one shuffle, one operator, one hop on the real-time path. The processor, `ZoneCounter`, is keyed by zone and keeps a `MapState` mapping each aircraft id to the last time we saw it in that zone. Each incoming position refreshes that timestamp. A recurring timer then sweeps out any aircraft we have not seen for a while (it has left the zone or gone quiet) and re-emits the updated count, so a busy zone that empties out clears on its own. The essence of the operator and how it is wired in:

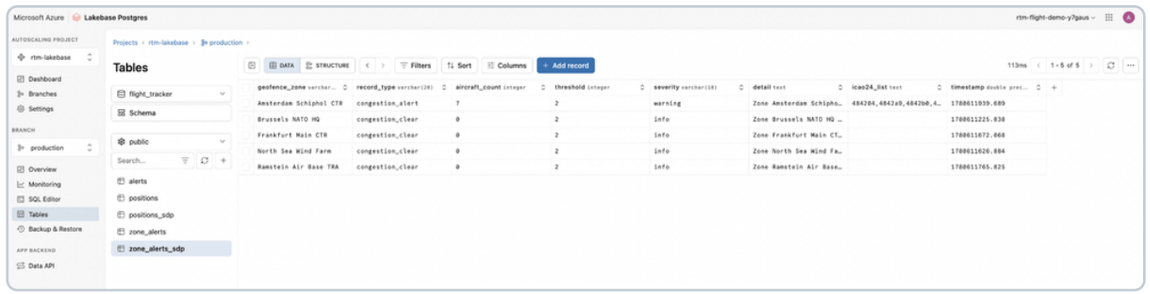
```
class ZoneCounter(StatefulProcessor):
    def init(self, handle):
        self.handle    = handle
        self._aircraft = handle.getMapState("aircraft", KEY_SCHEMA,
VAL_SCHEMA)  # icao24 -> last seen
        self._timer    = handle.getValueState("sweepTimer", TIMER_
SCHEMA)

    def handleInputRows(self, key, rows, timerValues):
        now_ms = timerValues.getCurrentProcessingTimeInMs()
        for r in rows:                          # called once per row
in RTM
            self._aircraft.updateValue(Row(icao24=r.icao24),
Row(last_ms=now_ms))
        active = self._sweep(now_ms)            # drop aircraft not
seen within the TTL
        self._ensure_timer(now_ms)              # keep exactly one
pending sweep timer
        yield self._alert_row(key[0], active, now_ms / 1000.0)

    def handleExpiredTimer(self, key, timerValues,
expiredTimerInfo):
        now_ms = timerValues.getCurrentProcessingTimeInMs()
        active = self._sweep(now_ms)         # re-emit the refreshed
count
        if active:
            self._ensure_timer(now_ms)
        yield self._alert_row(key[0], active, now_ms / 1000.0)

# Wire the operator into a second flow: filter to in-zone positions,
group by
# zone, run ZoneCounter, and write the alerts to a (second) Lakebase
table.
@dp.update_flow(
    name="zone_congestion_flow",
    target="zone_alerts_sink",                    # a second
Lakebase sink, declared like the first
    spark_conf={"pipelines.trigger": "RealTime",  # turn RTM on
                "pipelines.trigger.interval": "5 minutes"},
)
```

```
Lakebase sink, declared like the first
    spark_conf={"pipelines.trigger": "RealTime",  # turn RTM on
                "pipelines.trigger.interval": "5 minutes"},
)
def zone_congestion_flow():
    in_zone = (
        spark.readStream.format("kafka")
            .option("subscribe", "raw-flights")
            # ... bootstrap servers + SASL auth ...
            .load()
            .select(from_json(col("value").cast("string"), RAW_
SCHEMA).alias("d"))
            .select("d.*")
            .withColumn("zone", zone_label())
            .filter(col("zone").isNotNull())      # only in-zone
positions feed the counter
            .select("zone", "icao24", "poller_ts")
    )
    # one shuffle: groupBy(zone) -> one stateful operator ->
congestion alerts
    return (
        in_zone.groupBy("zone")
               .transformWithState(
                   statefulProcessor = ZoneCounter(),
                   outputStructType  = ALERT_SCHEMA,
                   outputMode        = "update",
                   timeMode          = "processingTime",
               )
    )
```

Two patterns here are worth lifting into your own code:

Idempotent timers. `_ensure_timer` registers a sweep timer only if one is not already pending. Because `handleInputRows` is invoked per row in RTM, a naive "register a timer on every row" would pile up thousands of redundant timers. Keeping exactly one pending timer in a `ValueState` is the safe pattern. When a zone empties, the timer stops re-arming, and the next arriving position re-arms it.

Staleness-based eviction. An aircraft that crosses out of a zone simply stops appearing in that zone's stream; there is no "leave" event to react to. So the sweep evicts any aircraft not seen within a short TTL and re-emits the count on a recurring processing-time timer. This keeps the entire computation in a single stateful operator on the real-time path, exactly the kind of design that plays to Real-Time Mode's strengths.



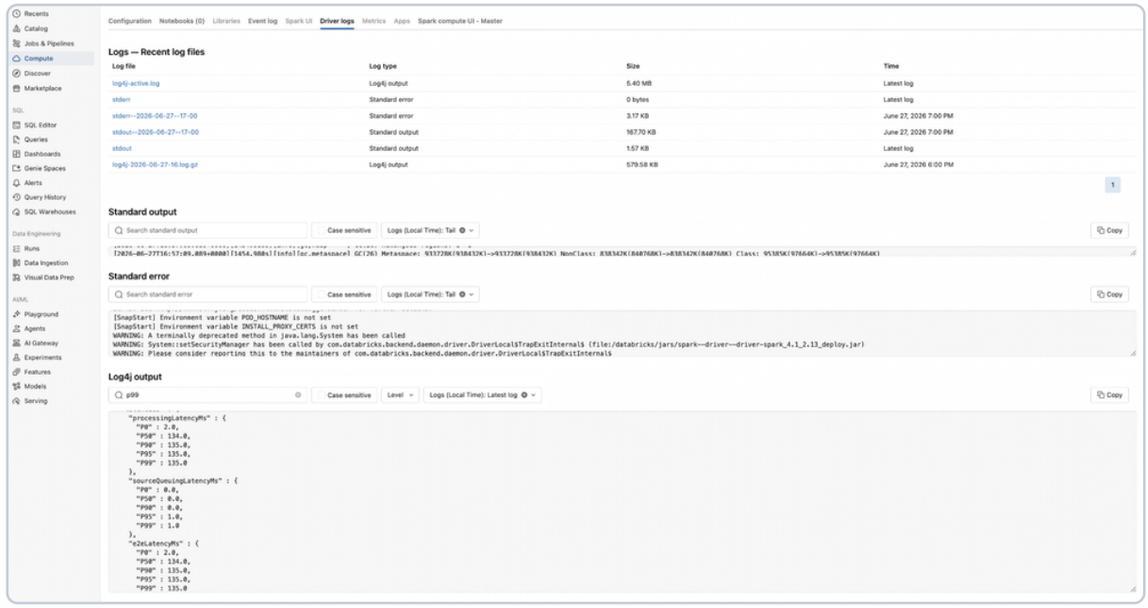
Measuring latency and monitoring

Real-Time Mode reports per-record latency at p50, p90, p95, and p99 (docs). Three numbers matter:

- `processingLatencyMs`: from reading a record to writing it downstream.
- `sourceQueuingLatencyMs`: from message-bus write (e.g. Kafka log-append) to first read.
- `e2eLatencyMs`: from message-bus write to writing downstream.

These are emitted in the streaming query progress, which the pipeline writes to its driver logs as JSON on each reporting interval. Search the driver log for `latencies` and you will find a block like this, from a run of our demo pipeline:

```
"latencies": {
  "processingLatencyMs": {
    "P0": 2,
    "P50": 134,
    "P90": 135,
    "P95": 135,
    "P99": 135
  },
  "sourceQueuingLatencyMs": {
    "P0": 0,
    "P50": 0,
    "P90": 0,
    "P95": 1,
    "P99": 1
  },
  "e2eLatencyMs": {
    "P0": 2,
    "P50": 134,
    "P90": 135,
    "P95": 135,
    "P99": 135
  }
}
```



`e2eLatencyMs` covers the engine path from the message bus to the query's downstream write. Watch the percentiles, not the average; Real-Time Mode delivers, particularly at p99.

Getting started with Real-Time Mode on SDP

1. **Create the pipeline.** A serverless Lakeflow Spark Declarative Pipeline on the `PREVIEW` channel with `continuous: true`. Set `spark.databricks.streaming.realTimeMode.enabled: true`
2. **Point it at Apache Kafka.** Your streaming source is a Kafka topic; read it with `spark.readStream.format("kafka")` inside the flow.
3. **Declare a sink and a flow.** `dp.create_sink(...)` for your destination, then an `@dp.update_flow(target=...)` with `pipelines.trigger: "RealTime"`.
4. **Add state only where you need it,** with `transformWithState` and the `StatefulProcessor` class. The idempotent-timer and staleness-eviction patterns above keep stateful logic in a single operator on the real-time path.
5. **Measure** with the latency percentiles before and after, so you can prove the win.

The full flight-tracker pipeline (both flows and the Lakebase sink) is on GitHub.

Conclusion

Real-Time Mode delivers sub-second, continuous processing to Spark Declarative Pipelines without sacrificing the declarative model. You keep the authoring surface you already use, declare what, *not* how, and gain real-time speed all on one engine. The flight tracker shows the whole arc: a stateless enrichment flow, a stateful operator with a real-time-aware timer pattern, and external delivery to an operational store. Try it on a workload where seconds are too slow, and watch your p99.

Ready to break the microbatch barrier? Check out these resources:

- Real-Time Mode docs
- Breaking the micro-batch barrier: the architecture of Real-Time Mode
- Why data engineering needs to become end-to-end declarative
- Demo: "Getting started with Real-Time Mode"

The external sink that writes to Databricks Lakebase in this demo is currently in Private Preview. Its configuration API might change before general availability, which is why the sink's exact options are omitted from the code above. If you'd like to try the native sink while it's in Private Preview, reach out to Databricks through your account team.

Spark Declarative Pipelines "How-To" Series, Part 1: How to Save Results Into a Table

By Aleksandra Chashchina

Introduction

Lakeflow Spark Declarative Pipelines (SDP) is a framework designed for building scalable, maintainable ETL pipelines. Everyone should want to make the transition from traditional imperative ETL, like standard Spark or Pandas, to a declarative approach because it saves time and effort and removes headache.

By adopting a declarative approach, you focus exclusively on **business logic**. SDP automatically handles the heavy lifting: recomputing, retries, persistence and state management — eliminating the redundant boilerplate code.

Let's walk through an example.

Consider a common task: ingesting Change Data Capture (CDC) logs to maintain a Slowly Changing Dimension (SCD) Type 1 table. You need to upsert records based on a key, handle deletes and ensure only the latest version of a record is kept.

Spark Imperative code

```
from pyspark.sql import DataFrame
from pyspark.sql.functions import col, struct, max_by

def upsert_to_delta(micro_batch_df, batch_id):
    # Deduplicate within the micro-batch to get the latest record
per key
    (micro_batch_df
     .groupBy("key")
     .agg(max_by(struct("*"), col("ts")).alias("row"))
     .select("row.*")
     .createOrReplaceTempView("updates"))

    # Execute the MERGE logic
    # Note: Replace the ellipses with your actual column mapping
    micro_batch_df.spark_session.sql("""
        MERGE INTO cdc_data_raw t
        USING updates s
        ON s.key = t.key
        WHEN MATCHED AND s.is_delete = true THEN
            UPDATE SET DELETED_AT = now()
        WHEN MATCHED THEN UPDATE SET
            A = CASE WHEN s.ts > t.ts THEN s.a ELSE t.a END,
            B = CASE WHEN s.ts > t.ts THEN s.b ELSE t.b END,
            -- ... for every column ...
            ts = CASE WHEN s.ts > t.ts THEN s.ts ELSE t.ts END
        WHEN NOT MATCHED THEN INSERT *
    """)

# Start the stream
(cdcData.writeStream
    .foreachBatch(upsert_to_delta)
    .outputMode("append")
    .start()
)
```

Spark Declarative Pipelines

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

@dp.view
def users():
    return spark.readStream.table("cdc_data.users")

dp.create_streaming_table("target")

dp.create_auto_cdc_flow(
    target = "target",
    source = "users",
    keys = ["key"],
    sequence_by = col("ts"),
    apply_as_deletes = expr("is_deleted = True")
)
```

In an imperative approach, this would require a complex, multi-line `MERGE` statement and manual state management. In SDP, the `AUTO CDC` flow handles the mechanics automatically. You focus only on the business logic. **Bingo!**

Of course, `AUTO CDC` is just one building block of the framework. There are a few other simple patterns to keep in mind to get the most out of your pipelines. Let's start with the fundamentals: **how to save results into a table in SDP.**

Part 1. How to save results into a table with Spark Declarative Pipelines

In the traditional Spark world, we typically end every script with a "command." We transform our data, and then we tell Spark exactly what to do with it: `df.write.saveAsTable("sales")`.

When you first switch to Lakeflow SDP, you might get blocked. You've written your transformation, you're ready to persist the data, and you reach for your trusty `.write.save()`. **Except, it's not there.** You have the data, but how do you actually make it land in a table?

In fact, in SDP, *explicitly calling a "write" action isn't needed*.

THE SHIFT: FROM ACTION TO DECLARATION

The shift is simple: in an imperative world, you **send** data to a table. In a declarative world, you **define** a table as the result of a function or a SQL query (SDP supports both Python and SQL).

Instead of manually coding steps to follow, you simply use a decorator. Note that, in Python SDP, every table declaration is a Python function which returns a Spark DataFrame:

The Imperative Way

```
(spark.readStream.table("raw_sales")
    .filter("amount > 0")
    .writeStream .option("checkpointLocation","s3://bucket/
checkpoints/sales_report")
    .outputMode("append")
    .toTable("sales_report")
)
```

The SDP Way

```
from pyspark import pipelines as dp

@dp.table
def sales_report():
    return (spark.readStream
              .table("raw_sales")
              .filter("amount > 0")
            )
```

Consider a more complex example to illustrate a daily aggregation over sales data:

The Imperative Way

This approach forces you to step outside the main logic flow into a "helper" function, manually managing the sink and checkpoints.

```
# The foreachbatch function
def process_daily_chunk(batch_df, batch_id):
    # Perform aggregation within the batch.
    # Note: this would NOT handle late data.
    daily_summary = (batch_df
        .groupBy("store_id", "processing_date")
        .agg({"sales_amount": "sum"})
    )

    # Manually append to the target table
    (daily_summary.write.mode("append")
         .saveAsTable("daily_sales_stats"))

# The main stream management
(spark.readStream
    .table("raw_transactions")
    .writeStream
    .foreachBatch(process_daily_chunk)
    .option("checkpointLocation", "s3://bucket/checkpoints/daily_
stats")
    .start()
)
```

The SDP Way

The framework handles the "how." You only define the "what." Because you used `@dp.materialized_view`, SDP knows to handle the state and incremental updates automatically.

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

@dp.materialized_view
def daily_sales_stats():
    # You define the transformation;
    # SDP handles the execution under the hood.
    # Note: late data is handled automatically.
    return (
        spark.read.table("raw_transactions")
        .groupBy("store_id", "processing_date")
        .agg(F.sum("sales_amount").alias("total_sales"))
    )
```

By adding `@dp.table` or `@dp.materialized_view`, you are telling the framework: "This function represents a physical table. Make it so." Note that, by default, the name of the table is inferred from the function name.

If you're wondering, "Wait, when do I use a `@dp.table` versus a `@dp.materialized_view`?" - don't worry! I've used both in the examples above, and it's perfectly normal if the distinction feels blurry right now. They look similar, but they handle data quite differently under the hood.

Key takeaway

As you start your journey with SDP, perform a regular "self-check" on your code:

"Am I focusing on the business logic, or am I getting into implementation details?"

If you find yourself figuring out save modes or `foreachbatch` semantics, you're likely still thinking imperatively. When you let the framework handle the "how," you know you're using SDP correctly.

Conclusion

SDP provides a standardized, managed framework for ETL pipelines. It automatically handles the **checkpoints**, **storage paths** and **boilerplate code**, giving more room to focus on business logic and outcomes.

Stop Hand-Coding Change Data Capture Pipelines

How AutoCDC automates CDC and slowly changing dimensions

By Matt Jones, Zoé Durand, Phoebe Weiser, Bilal Aslam and Ray Zhu

Change data capture (CDC) and slowly changing dimensions (SCD) are foundational to modern analytics and AI workloads. Teams rely on them to keep downstream tables accurate as operational data changes — whether that means maintaining a current view of the business or preserving full historical context.

Yet in practice, CDC pipelines are often some of the most painful pipelines to build and operate. Teams routinely hand-roll complex `MERGE` logic to handle updates, deletes, and late-arriving data: layering on staging tables, window functions, and sequencing assumptions that are difficult to reason about, and even harder to maintain as pipelines evolve. While LLMs can make this code faster to produce, they don't reduce the complexity of getting it right or keeping it correct over time — they can generate code, but they don't understand your data.

This is exactly what AutoCDC was designed to solve. Instead of encoding this logic directly in every pipeline, teams declare the desired semantics and the platform implements them. Genie Code can then build on this foundation to generate pipelines that are correct by design.

FORTUNE 500

> "I tried AutoCDC from Snapshots in Python and was amazed at how four lines of code could replace what I was doing in 1,500 lines of code before."
>
> —Senior Data Engineer, Fortune 500 Aerospace & Defense Company

In this post, we'll walk through the CDC and SCD patterns data engineers and SQL practitioners encounter every day, why these patterns are painful to implement by hand and how AutoCDC in Lakeflow Spark Declarative Pipelines automates them declaratively — while also delivering meaningful improvements in price and performance.

CDC and SCD are still hard for data engineers

Even for teams that understand these patterns well, getting them correct and keeping them correct over time is where things break down. As data volumes grow and use cases expand, pipelines become fragile; correctness issues surface late; and even small changes require careful rewrites to avoid corrupting downstream tables.

MAINTAINING SCD TYPE 1 TABLES

SCD Type 1 tables overwrite existing rows to reflect the latest state. Even this "simple" case quickly runs into challenges:

- Updates arrive out of order
- Duplicate events must be consistently deduplicated
- Deletes must be applied correctly
- Logic must remain idempotent across retries and reprocessing

What often starts as a simple MERGE INTO evolves into deeply nested logic with staging tables, window functions, and sequencing assumptions that are hard to reason about (or safely change). Over time, teams become reluctant to touch these pipelines at all.

MAINTAINING SCD TYPE 2 HISTORY

SCD Type 2 introduces additional complexity:

- Tracking row versions and validity windows
- Handling late-arriving updates without corrupting history
- Ensuring exactly one "current" version exists at any time

Mistakes here don't always fail loudly. They often surface weeks later as subtle metric drift, or the need to rebuild historical tables entirely.

EXTRACTING CHANGE DATA FROM DIFFERENT SOURCES

Not all systems emit clean CDC logs. Some systems emit native change data feeds, while others don't — often because the team consuming the data doesn't control the upstream database — forcing teams to reconstruct changes by comparing successive snapshots of a source table.

Supporting both typically means separate ingestion and processing logic; different correctness assumptions; and more code paths to maintain and debug.

OPERATING CDC PIPELINES OVER TIME

Even once a CDC pipeline is correct, it still has to survive reprocessing and backfills, schema evolution, failures and restarts. Hand-rolled CDC logic tends to grow more fragile over time as these realities accumulate, increasing operational risk and maintenance cost.

Automating complex CDC patterns with declarative data engineering

AutoCDC was designed to standardize these common CDC and SCD patterns behind a declarative abstraction. Instead of hand-coding *how* changes should be applied, teams declare *what semantics* they want, and the platform manages ordering, state, and incremental processing.

This gives teams a consistent, repeatable way to implement CDC and SCD across pipelines, rather than reinventing the pattern each time (which is really the core value of declarative programming in general, and Spark Declarative Pipelines specifically). This foundation also enables Genie Code to generate and manage these pipelines using AutoCDC, ensuring AI-assisted development builds on standardized, production-ready semantics.

CDC workload	AutoCDC	Hand-Written MERGE / Snapshot Logic
Maintaining current-state tables (SCD Type 1)	Declarative pipeline definition automatically handles sequencing, deduplication, and deletes	Custom MERGE logic with window functions and sequencing rules
Maintaining historical tables (SCD Type 2)	Automatic version management with built-in history tracking	Multi-step MERGE logic to close and insert record versions
Inferring changes from snapshot sources	Built-in snapshot CDC support	Manual snapshot diff pipelines with joins and comparisons
Operating pipelines reliably over time (late data, retries, reprocessing)	Automatic ordering and idempotent execution	Requires custom safeguards and additional logic
Code footprint and operational complexity	~6–10 lines of declarative pipeline definition	40–200+ lines of custom pipeline logic

When processing change records from a change data feed (CDF), AutoCDC automatically handles out-of-sequence records and applies updates correctly based on a declared sequencing column. To show how this works in practice, let's consider the sample CDC feed below:

userId	name	city	operation	sequenceNum
124	Raul	Oaxaca	INSERT	1
123	Isabel	Monterrey	INSERT	1
125	Mercedes	Tijuana	INSERT	2
126	Lily	Cancun	INSERT	2
123	null	null	DELETE	6
125	Mercedes	Guadalajara	UPDATE	6
125	Mercedes	Mexicali	UPDATE	5
123	Isabel	Chihuahua	UPDATE	5

Remember, you should choose SCD Type 1 to keep only the latest data, or choose SCD Type 2 to keep historical data. Let's start with Type 1.

AUTOMATING SCD TYPE 1 MAINTENANCE (CHANGE DATA FEED SOURCES)

In this example, a change data feed contains inserts, updates, and deletes for a user table. The goal is to maintain a **current view of each record**, where new updates overwrite older values.

Output table for SCD Type 1

id	name	city
124	Raul	Oaxaca
125	Mercedes	Tijuana
126	Lily	Cancun

User 123 (Isabel) was deleted, so it doesn't appear in the output. User 125 (Mercedes) shows only the latest city (Guadalajara) because SCD Type 1 overwrites previous values.

With a traditional approach, this requires custom `MERGE` logic to deduplicate events, enforce ordering, apply deletes, and ensure the pipeline remains correct across retries or late-arriving data. AutoCDC replaces this fragile logic with a declarative pipeline definition that automatically handles sequencing, deduplication, late-arriving data, and incremental processing - eliminating dozens of lines of custom merge logic.

See full code example in appendix starting on page 56

AUTOMATING SCD TYPE 2 HISTORY (CHANGE DATA FEED SOURCES)

In many analytical systems, keeping only the latest state is not enough - teams need a complete **history of how records change over time**. This is the SCD Type 2 pattern, where each version of a record is stored with validity windows indicating when it was active.

Output table for SCD type 2:

userId	name	city	__START_AT	__END_AT
123	Isabel	Monterrey	1	5
123	Isabel	Chihuahua	5	6
124	Raul	Oaxaca	1	NULL
125	Mercedes	Tijuana	2	5
125	Mercedes	Mexicali	5	6
125	Mercedes	Guadalajara	6	NULL
126	Lily	Cancun	2	NULL

The table preserves complete history. User 123 has two versions (ended at sequence 6 when deleted). User 125 has three versions showing city changes. Records with `__END_AT = NULL` are currently active.

Implementing this manually requires multi-step `MERGE` logic to close out previous records, insert new versions, and ensure only one version remains active at a time. AutoCDC automates these transitions declaratively, managing history columns and versioning logic automatically while ensuring correctness even when updates arrive out of order.

See full code example in appendix starting on page 56

INFERRING CDC FROM SNAPSHOT SOURCES

Not all source systems emit change logs. In many cases, teams receive periodic snapshots of a source table and must infer what changed between runs.

Traditionally, this requires manually comparing snapshots to detect inserts, updates, and deletes before applying those changes with MERGE logic. AutoCDC treats snapshot-based CDC as a first-class pattern, automatically detecting row-level changes between snapshots and applying them incrementally without requiring custom diff logic or state management.

Implementing this manually requires detecting row-level changes between snapshots, closing out previously active records, and inserting new versions with updated validity windows. AutoCDC automatically derives these changes and applies SCD Type 2 semantics, maintaining version history without requiring multi-step merge logic or custom snapshot state tracking.

Managing ordering, state, and reprocessing

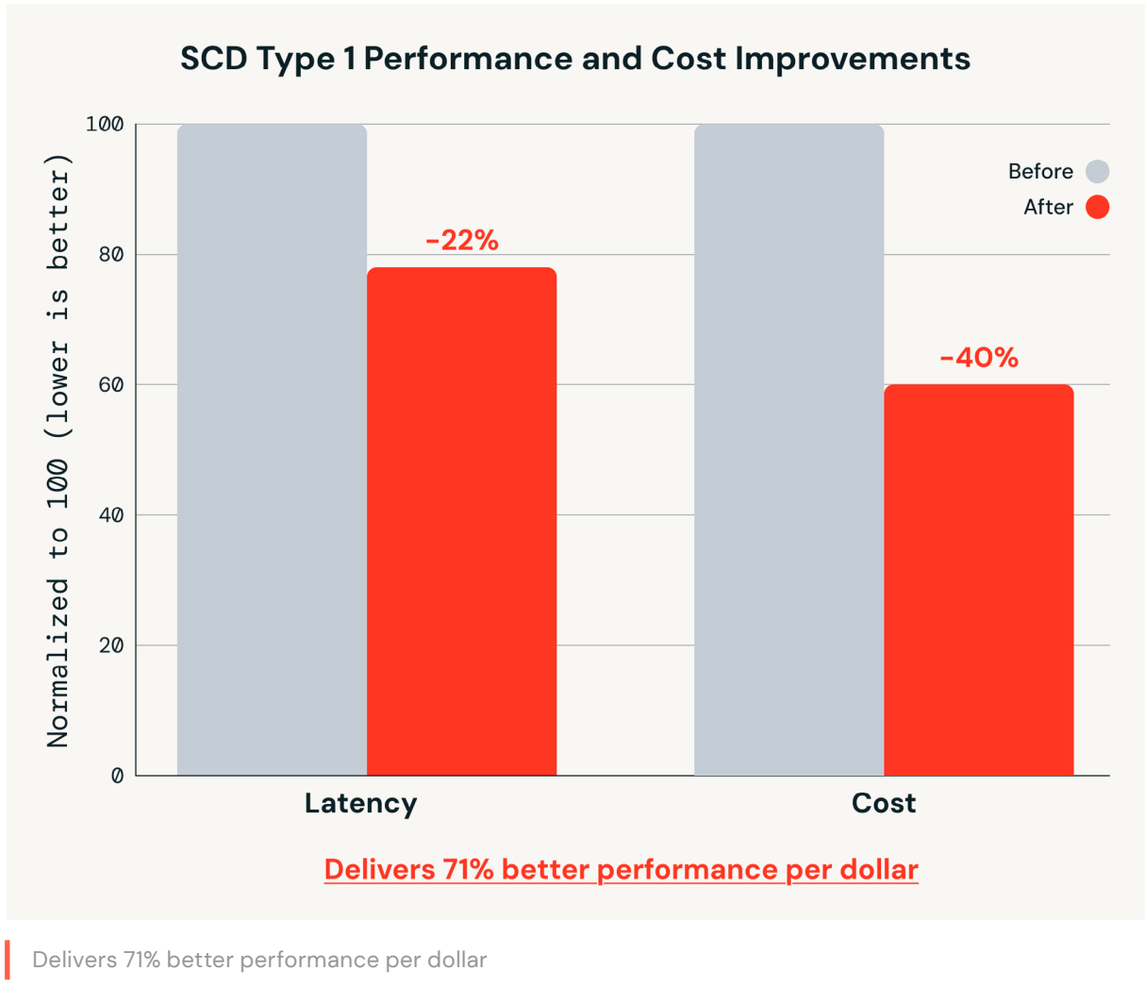
Lakeflow Spark Declarative Pipelines automatically tracks incremental progress and handles out-of-sequence data. Pipelines can recover from failures, reprocess historical data and evolve over time without double-applying or losing changes.

Practically, this removes the need for teams to manage sequencing logic, watermark bookkeeping or reprocessing safety themselves — the platform handles it.

What's new: major price and performance gains

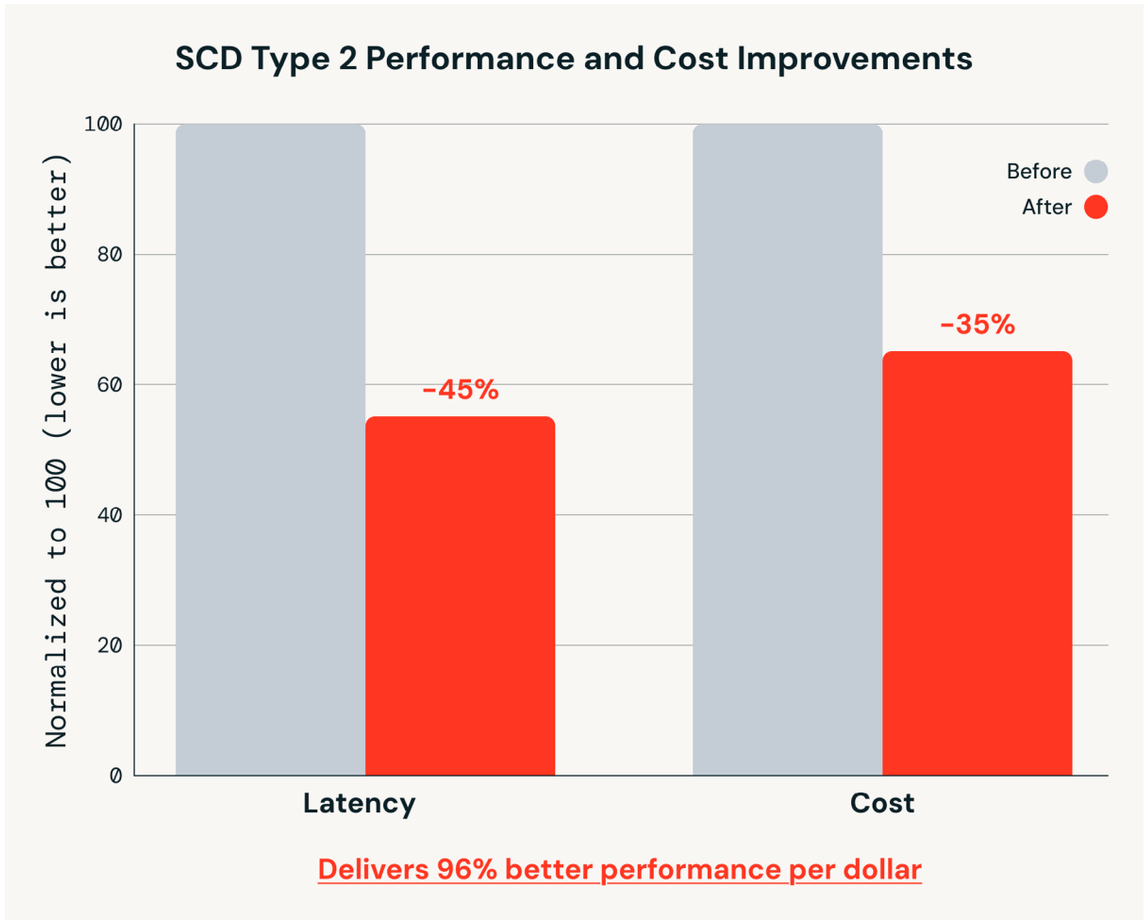
Beyond simplifying pipeline logic, recent Databricks Runtime improvements have delivered substantial gains in both performance and cost efficiency for AutoCDC workloads — just since November 2025:

SCD Type 1 Performance and Cost Improvements



> Delivers 71% better performance per dollar

SCD Type 2 Performance and Cost Improvements



> Delivers 96% better performance per dollar

These gains matter for real-world pipelines that run continuously at scale. While `MERGE INTO` remains a foundational Spark primitive, AutoCDC builds on it to handle out-of-sequence data and incremental processing more efficiently as data volumes grow.

Real-world results with AutoCDC

Teams running CDC and SCD pipelines in production consistently report significant value from AutoCDC:

Navy Federal Credit Union uses AutoCDC in Lakeflow Spark Declarative Pipelines to power large-scale, real-time event processing — handling billions of application events continuously while eliminating custom CDC code and ongoing pipeline maintenance.

NAVY FEDERAL Credit Union

> "The simplicity of the Spark Declarative Pipelines programming model combined with its service capabilities resulted in an incredibly fast turnaround time."
>
> —Jian (Miracle) Zhou, Senior Engineering Manager, Navy Federal Credit Union

Block uses AutoCDC in Lakeflow Spark Declarative Pipelines to simplify change data capture and real-time streaming pipelines on Delta Lake, replacing hand-coded CDC and merge logic with a declarative approach that's fast to implement and easy to operate.

block

"With the adoption of Spark Declarative Pipelines, the time required to define and develop a streaming pipeline has gone from days to hours."

—Yue Zhang, Staff Software Engineer, Data Foundations, Block

Valora Group, a leading Swiss-based "foodvenience" provider, uses AutoCDC in Lakeflow Spark Declarative Pipelines to streamline change data capture for master data and real-time retail analytics, replacing custom CDC code with a declarative approach that's easy to implement, repeat and scale across teams.

valora

"We gained a lot by doing CDC in SDP, because you don't write any code — it's all abstracted in the background. AutoCDC minimizes the number of lines... it's so easy to do."

—Alexane Rose, Data and AI Architect, Valora Holding

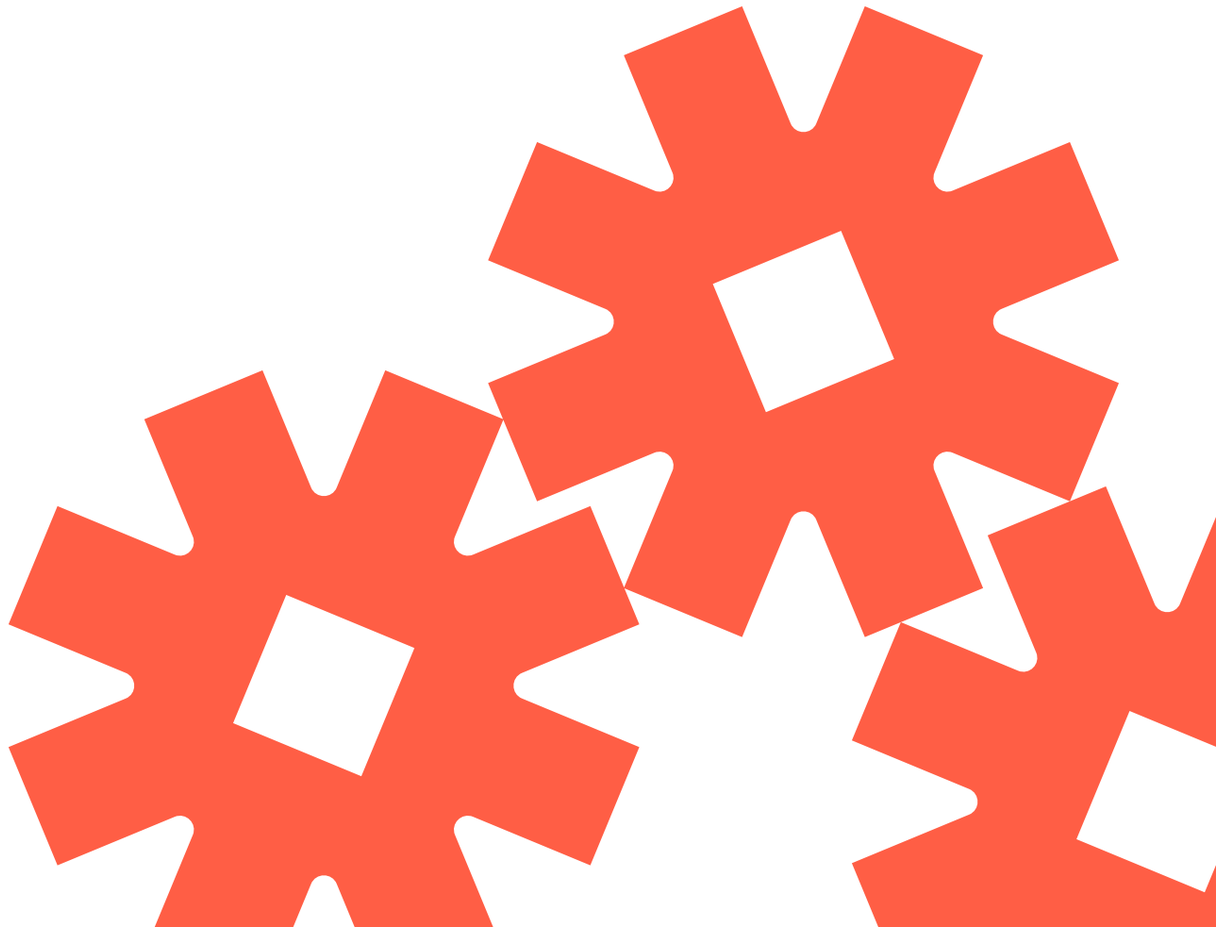
Get started

The AutoCDC APIs are available as part of Lakeflow Spark Declarative Pipelines on Databricks, and can also be created and managed using Databricks Genie Code.

To learn more:

- Review the AutoCDC documentation (SQL and Python)
- Explore examples for SCD Type 1, SCD Type 2, and snapshot-based CDC

Try AutoCDC in your own pipelines and eliminate hand-rolled CDC logic!



Appendix

SCD Type 1 Example

MERGE

```
from delta.tables import DeltaTable
from pyspark.sql.functions import max_by, struct

# Deduplicate: keep latest record per userId
updates = (spark.read.table("cdc_data.users")
    .groupBy("userId")
    .agg(max_by(struct("*"), "sequenceNum").alias("row"))
    .select("row.*"))

# Apply SCD Type 1: upsert updates, delete deletions
(DeltaTable.forName(spark, "target")
    .alias("t")
    .merge(updates.alias("s"), "s.userId = t.userId")

    .whenMatchedDelete(condition="s.operation = 'DELETE'")
    .whenMatchedUpdate(
        condition="s.sequenceNum > t.sequenceNum",
        set={"name": "s.name", "city": "s.city",
"sequenceNum": "s.sequenceNum"}
    )
    .whenNotMatchedInsertAll(condition="s.operation !=
'DELETE'")
    .execute())
```

AutoCDC

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

@dp.view
def users():
    return spark.readStream.table("cdc_data.users")

dp.create_streaming_table("target")

dp.create_auto_cdc_flow(
    target="target",
    source="users",
    keys=["userId"],
    sequence_by=col("sequenceNum"),
    apply_as_deletes=expr("operation = 'DELETE'"),
    stored_as_scd_type=1
)
```

SCD Type 2 Example

MERGE

```
from delta.tables import DeltaTable
from pyspark.sql.functions import col, lit, max_by, struct

# Deduplicate: keep latest record per userId
updates = (spark.read.table("cdc_data.users")
    .groupBy("userId")
    .agg(max_by(struct("*"), "sequenceNum").alias("row"))
    .select("row.*"))

# Step 1: close out active rows for records being updated or
deleted
(DeltaTable.forName(spark, "target")
    .alias("t")
    .merge(
        updates.alias("s"),
        "s.userId = t.userId AND t.__END_AT IS NULL AND
s.sequenceNum > t.__START_AT"
    )

    .whenMatchedUpdate(set={"__END_AT": "s.sequenceNum"})
    .execute())

# Step 2: insert new rows for inserts and updates (not
deletes)
new_rows = (updates
    .filter("operation != 'DELETE'")
    .withColumn("__START_AT", col("sequenceNum"))
    .withColumn("__END_AT", lit(None).cast("long"))
    .drop("operation"))

new_rows.write.mode("append").saveAsTable("target")
```

AutoCDC

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

@dp.view
def users():
    return spark.readStream.table("cdc_data.users")

dp.create_streaming_table("target")

dp.create_auto_cdc_flow(
    target="target",
    source="users",
    keys=["userId"],
    sequence_by=col("sequenceNum"),
    apply_as_deletes=expr("operation = 'DELETE'"),
    stored_as_scd_type=1
)
```

Why Data Engineering Needs to Become End-to-End Declarative

By Bilal Aslam, Ray Zhu and Matei Zaharia

Data engineering teams are under pressure to deliver higher quality data faster, but the work of building and operating pipelines is getting harder, not easier. We interviewed hundreds of data engineers and studied millions of real-world workloads and found something surprising: data engineers spend the majority of their time not on writing code but on the *operational burden generated by stitching together tools*.

The reason is simple: existing data engineering frameworks force data engineers to manually handle orchestration, incremental data processing, data quality and backfills - all common tasks for production pipelines. As data volumes and use cases grow, this operational burden compounds, turning data engineering into a bottleneck for the business rather than an accelerator.meet their needs, which inherited a whole stack to run alongside it.

This isn't the first time the industry has hit this wall. Early data processing required writing a new program for every question, which didn't scale. SQL changed that by making individual queries declarative: you specify what result you want, and the engine figures out **how** to compute it. SQL databases now underpin every business.

But data engineering isn't about running a single query. Pipelines repeatedly update multiple interdependent datasets over time. Because SQL engines stop at the query boundary, everything beyond it — incremental processing, dependency management, backfills, data quality, retries — still has to be hand-assembled. At scale, reasoning about execution order, parallelism and failure modes quickly becomes the dominant source of complexity.

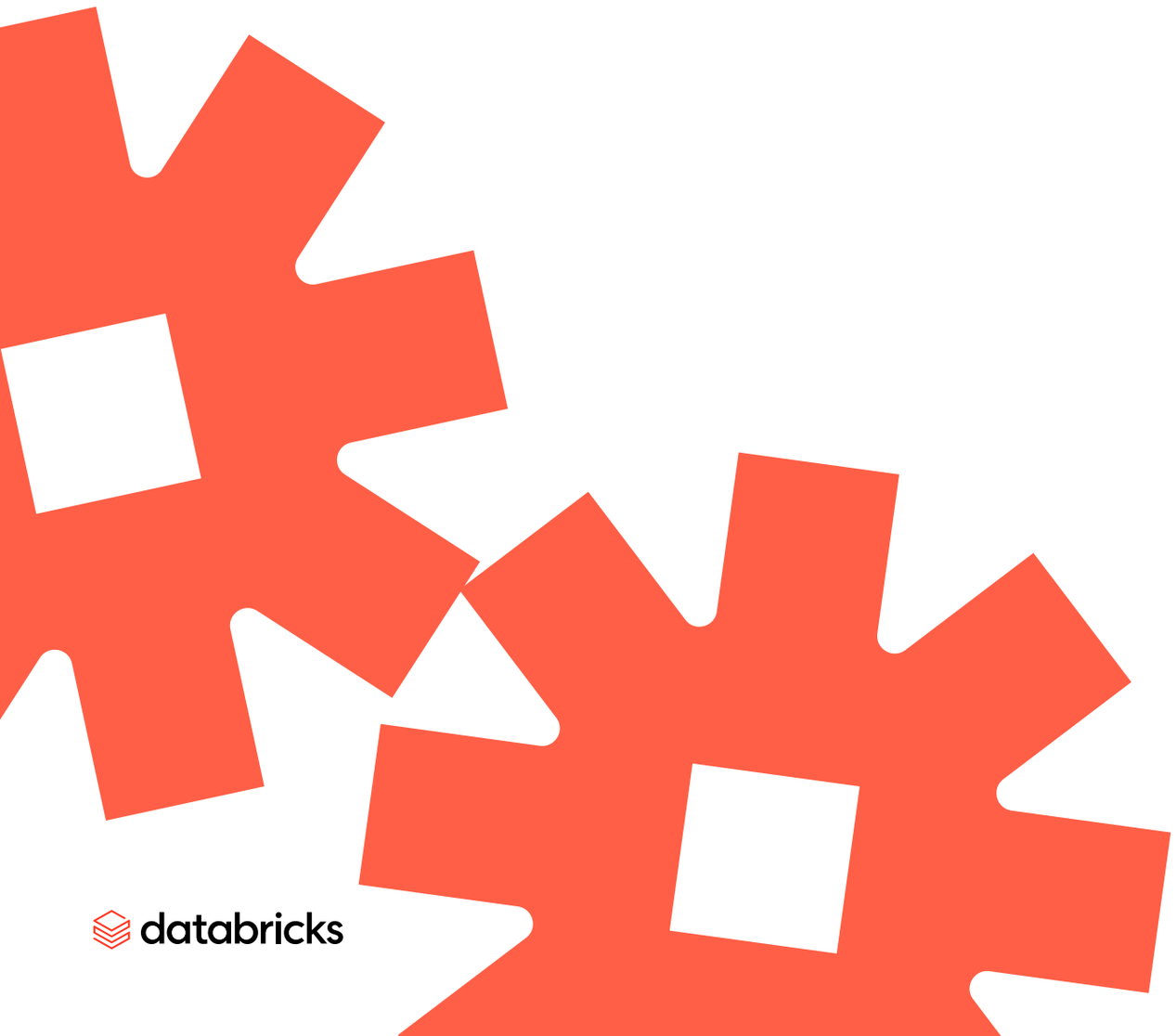
What's missing is a way to *declare the pipeline as a whole*. Spark Declarative Pipelines (SDP) extend declarative data processing from individual queries to entire pipelines, letting Apache Spark plan and execute them end to end. Instead of manually moving data between steps, you declare what datasets you want to exist and SDP is responsible for how to keep them correct over time.

For example, in a pipeline that computes weekly sales, SDP infers dependencies between datasets, builds a single execution plan and updates results in the right order. It automatically processes only new or changed data, expresses data quality rules inline and handles backfills and late-arriving data without manual intervention. Because SDP understands query semantics, it can validate pipelines upfront, execute safely in parallel and recover correctly from failures — capabilities that require first-class, pipeline-aware declarative APIs built directly into Apache Spark.

End-to-end declarative data engineering in SDP brings powerful benefits:

- **Greater productivity:** Data engineers can focus on writing business logic instead of glue code.
- **Lower costs:** The framework automatically handles orchestration and incremental data processing, making it more cost-efficient than hand-written pipelines.
- **Lower operational burden:** Common use cases such as backfills, data quality and retries are integrated and automated.

To illustrate the benefits of end-to-end declarative data engineering, let's start with a weekly sales pipeline written in PySpark. Because PySpark is not end-to-end declarative, we must manually encode execution order, incremental processing, and data quality logic, and rely on an external orchestrator such as Airflow for retries, alerting, and monitoring (omitted here for brevity).



databricks

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from delta.tables import DeltaTable

spark = SparkSession.builder.appName("weekly_sales").getOrCreate()

# Manual incremental processing - find what's already been processed
last_processed = spark.sql("""
    SELECT COALESCE(MAX(week_start), '1970-01-01') as max_week
    FROM sales.weekly_sales
""").collect()[0]["max_week"]

# Read only new data manually
df = spark.read.table("sales.raw_sales") \
    .filter(col("date") > last_processed)

# Manual data quality - filter and quarantine bad records
good_records = df.filter(col("amount") > 0)
bad_records = df.filter(col("amount") <= 0)

# Write bad records to quarantine table
bad_records.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("sales.quarantined_sales")

# Manual aggregation logic
weekly_agg = good_records \
    .withColumn("week_start", date_trunc("week", col("date"))) \
    .groupBy("week_start", "store_id") \
    .agg(sum("amount").alias("total_sales"))

# Manual merge - handle updates and inserts
delta_table = DeltaTable.forName(spark, "sales.weekly_sales")

delta_table.alias("target").merge(
    weekly_agg.alias("source"),
    "target.week_start = source.week_start AND target.store_id =
source.store_id"
).whenMatchedUpdate(
```

```
delta_table.alias("target").merge(
    weekly_agg.alias("source"),
    "target.week_start = source.week_start AND target.store_id =
source.store_id"
).whenMatchedUpdate(
    set = {"total_sales": "target.total_sales + source.total_sales"}
).whenNotMatchedInsertAll() \
 .execute()

# Manual error handling, monitoring, and retries must be handled by
external orchestrator
```

This pipeline expressed as a SQL dbt project suffers from many of the same limitations: we must still manually code incremental data processing, data quality is handled separately and we still have to rely on an orchestrator such as Airflow for retries and failure handling:

SQL

```
-- models/weekly_sales.sql
-- Aggregates daily sales into weekly totals by store
{{
  config(
    materialized='incremental',
    unique_key=['week_start', 'store_id']
  )
}}

WITH source_data AS (
    SELECT *
    FROM {{ source('sales', 'raw_sales') }}

-- manual incremental processing
    {% if is_incremental() %}
    WHERE date > (SELECT MAX(week_start) FROM {{ this }})
    {% endif %}
),

validated AS (
    SELECT *
    FROM source_data
    WHERE amount > 0  -- Data quality filter
),

weekly_aggregates AS (
    SELECT
        DATE_TRUNC('week', date) AS week_start,
        store_id,
        SUM(amount) AS total_sales
    FROM validated
    GROUP BY 1, 2
)

SELECT * FROM weekly_aggregates

-- models/quarantined_sales.sql
-- manual quarantine for bad data
{{
  config(
    materialized='incremental',
    unique_key='transaction_id'
  )
}}

SELECT *
FROM {{ source('sales', 'raw_sales') }}
WHERE amount <= 0

-- manual incremental processing
{% if is_incremental() %}
AND date > (SELECT MAX(date) FROM {{ this }})
{% endif %}
```

Let's rewrite this pipeline in SDP to explore its benefits. First, let's install SDP and create a new pipeline:

SHELL

```
$ pip install pyspark[pipelines]

$ spark-pipelines init
```

Next, define your pipeline with the following code. Note that we comment out the `expect_or_drop` data quality expectation API as we are working with the community to open source it:

PYTHON

```
@sdp.table()
def raw_sales():
    return spark.read.table("sales.raw_sales")

@sdp.table()
# @sdp.expect_or_drop("valid_amount", "amount > 0")

def weekly_sales():
    return (
        spark.read("raw_sales")
        .withColumn("week_start", date_trunc("week", col("date")))
        .groupBy("week_start", "store_id")
        .agg(sum("amount").alias("total_sales"))
    )
```

To run the pipeline, type the following command in your terminal:

SHELL

```
$ spark-pipelines run
```

We can even validate our pipeline upfront without running it first with this command — it's handy for catching syntax errors and schema mismatches:

SHELL

```
$ spark-pipelines dry-run
```

Backfills become much simpler — to backfill the `raw_sales` table, run this command:

SHELL

```
$ spark-pipelines run --full-refresh raw_sales
```

The code is much simpler – just 20 lines that deliver everything the PySpark and dbt versions require external tools to provide. We also get these powerful benefits:

- **Automatic incremental data processing.** The framework tracks which data has been processed and only reads new or changed records. No MAX queries, no checkpoint files, no conditional logic needed.
- **Integrated data quality.** The `@dp.expect_or_drop` decorator quarantines bad records automatically. In PySpark, we manually split and wrote good/bad records to separate tables. In dbt, we needed a separate model and manual handling.
- **Automatic dependency tracking.** The framework detects that `weekly_sales` depends on `raw_sales` and orchestrates execution order automatically. No external orchestrator needed.
- **Integrated retries and monitoring.** The framework handles failures and provides observability through a built-in UI. No external tools required.

SDP in Apache Spark 4.1 has the following capabilities which make it a great choice for data pipelines:

- Python and SQL APIs for defining datasets
- Support for batch and streaming queries
- Automatic dependency tracking between datasets, and efficient parallel updates
- CLI to scaffold, validate, and run pipelines locally or in production

We are excited about SDP's roadmap, which is being developed in the open with the Spark community. Upcoming Spark releases will build on this foundation with support for continuous execution, and more efficient incremental processing. We also plan to bring core capabilities like Change Data Capture (CDC) into SDP, shaped by real-world use cases and community feedback. Our aim is to make SDP a shared, extensible foundation for building reliable batch and streaming pipelines across the Spark ecosystem.

Building a Near Real-Time Application With Zerobus Ingest and Lakebase

Learn how you can simplify data ingestion for IoT, clickstream and telemetry use cases on Databricks

By Grant Doyle and Benjamin Nwokeleme

Event data from IoT, clickstream, and application telemetry powers critical real-time analytics and AI when combined with the Databricks Data + AI Platform. Traditionally, ingesting this data required several data hops (message bus, Spark jobs) between the data source and the lakehouse. This adds operational overhead, data duplication, requires specialized expertise and it's generally inefficient when the lakehouse is the only destination for this data.

Once this data lands in the lakehouse, it is transformed and curated for downstream analytical use cases. However, teams often need to serve this analytical data for operational use cases, and building these custom applications can be a laborious process. They need to provision and maintain essential infrastructure components like a dedicated OLTP database instance (with networking, monitoring, backups and more).

Additionally, they need to manage the reverse ETL process for the analytical data into the database to resurface it in the real-time application. This would require the team to build additional pipelines to push data from the lakehouse into the external operational database. These pipelines add to the infrastructure that developers need to set up and maintain, which altogether diverts their attention from the main goal: building the applications for their business.

So how does Databricks simplify both ingesting data into the lakehouse and serving gold data to support operational workloads?

Enter Zerobus Ingest and Lakebase.

About Zerobus Ingest

Zerobus Ingest is a set of APIs that provide a streamlined way to push event data directly into the lakehouse. Eliminating the single-sink message bus layer entirely, Zerobus Ingest reduces infrastructure, simplifies operations and delivers near real-time ingestion at scale. As such, Zerobus Ingest makes it easier than ever to unlock the value of your data.

The data-producing application must specify a target table to write data to, ensure that the messages map correctly to the table's schema and then initiate a stream to send data to Databricks. On the Databricks side, the API validates the schemas of the message and the table, writes the data to the target table and sends an acknowledgment to the client that the data has been persisted.

KEY BENEFITS OF ZEROBUS INGEST:

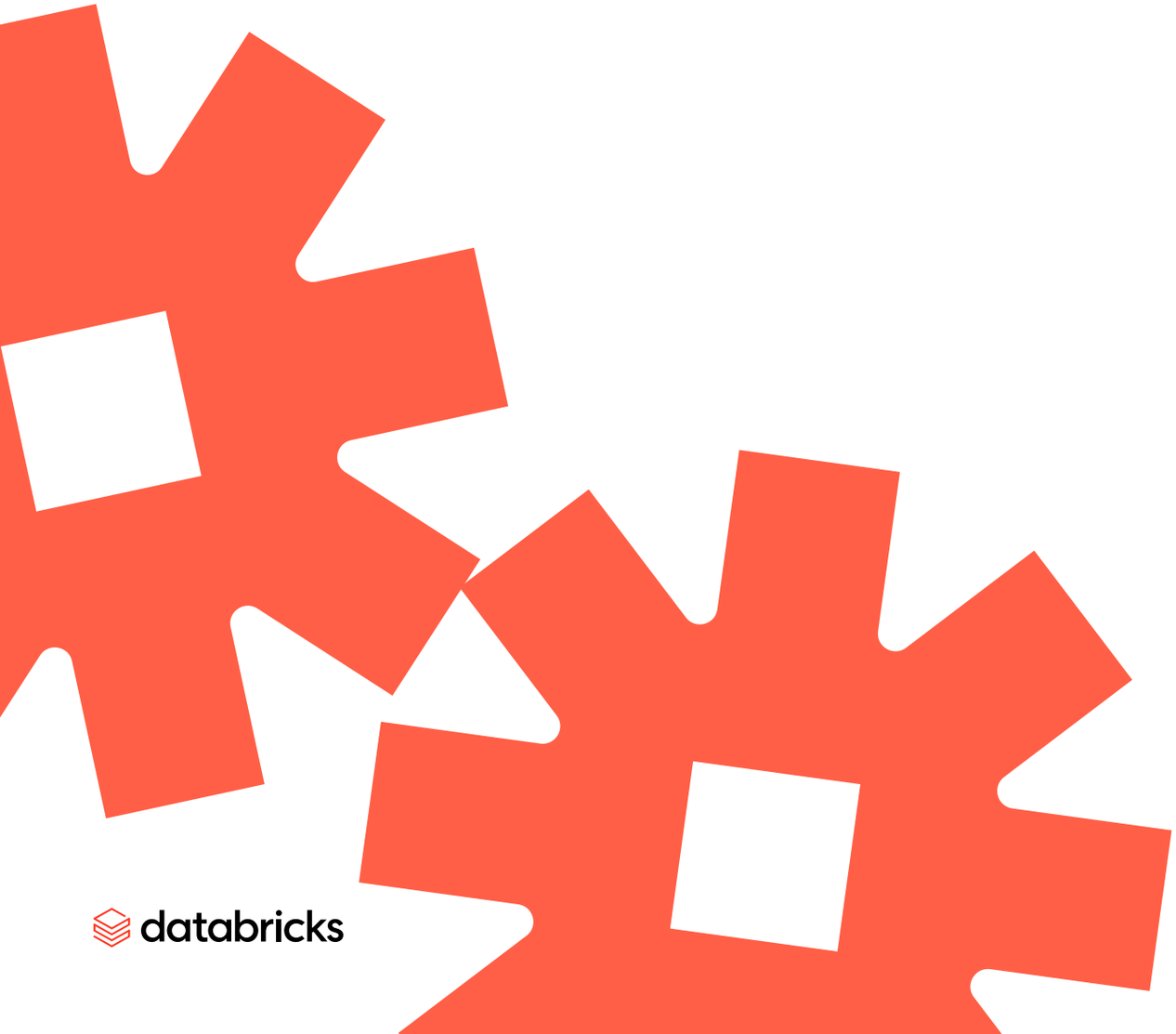
- **Streamlined architecture:** eliminates the need for complex workflows and data duplication
- **Performance at scale:** supports near real-time ingestion (up to 5 secs) and allows thousands of clients writing to the same table (up to 100MB/sec throughput per client)
- **Integration with the Data + AI Platform:** accelerates time to value by enabling teams to apply analytics and AI tools, such as MLflow for fraud detection, directly on their data

Zerobus Ingest Capability	Specifications
Ingestion latency	Near real-time (≤5 seconds)
Max throughput per client	Up to 100 MB/sec
Concurrent clients	Thousands per table
Continuous sync lag (Delta → Lakebase)	10–15 seconds
Real-time foreach writer latency	200–300 milliseconds

About Lakebase

Lakebase is a fully managed, serverless, scalable, Postgres database built into the Databricks Data + AI Platform, designed for low-latency operational and transactional workloads that run directly on the same data powering analytical and AI use cases.

The complete separation of compute and storage delivers rapid provisioning and elastic autoscaling. Lakebase's integration with the Databricks Data + AI Platform is a major differentiator from traditional databases because Lakebase makes Lakehouse data directly available to both real-time applications and AI without the need for complex custom data pipelines. It is built to deliver database creation, query latency and concurrency requirements to power enterprise applications and agentic workloads. Lastly, it allows developers to easily version control and branch databases like code.



KEY BENEFITS OF LAKEBASE:

- **Automatic data synchronization:** ability to easily sync data from the Lakehouse (analytical layer) to Lakebase on a snapshot, scheduled or continuous basis, without the need for complex external pipelines
- **Integration with the Databricks Data + AI Platform:** Lakebase integrates with Unity Catalog, Lakeflow Connect, Spark Declarative Pipelines, Databricks Apps, and more
- **Integrated permissions and governance:** Consistent role and permissions management for operational and analytical data. Native Postgres permissions can still be maintained via the Postgres protocol

Together, these tools allow customers to ingest data from several systems directly into Delta tables and implement reverse ETL use cases at scale.

Next, we will explore how to use these technologies to implement a near real-time application!

How to build a near real-time application

As a practical example, let's help 'Data Diners,' a food delivery company, empower their management staff with an application to monitor driver activity and order deliveries in real-time. Currently, they lack this visibility, which limits their ability to mitigate issues as they arise during deliveries.

Why is a real-time application valuable?

- **Operational awareness:** Management can instantly see where each driver is and how their current deliveries are progressing. That means fewer blind spots with late orders or when a driver needs assistance.
- **Issue mitigation:** Live location and status data enable dispatchers to reroute drivers, adjust priorities or proactively contact customers in the event of delays, reducing failed or late deliveries.

Let's see how to build this with Zerobus Ingest, Lakebase, and Databricks Apps on the Data + AI Platform!

OVERVIEW OF APPLICATION ARCHITECTURE

This end-to-end architecture follows four stages: (1) A data producer uses the Zerobus SDK to write events directly to a Delta table in Databricks Unity Catalog. (2) A continuous sync pipeline pushes updated records from the Delta table to a Lakebase Postgres instance. (3) A FastAPI backend connects to Lakebase via WebSockets to stream real-time updates. (4) A front-end application built on Databricks Apps visualizes the live data for end users.

Starting with our data producer, the data diner app on the driver's phone will emit GPS telemetry data about the driver's location (latitude and longitude coordinates) en route to deliver orders. This data will be sent to an API gateway, which ultimately sends the data to the next service in the ingestion architecture.

With the Zerobus SDK, we can quickly write a client to forward events from the API gateway to our target table. With the target table being updated in near real time, we can then create a continuous sync pipeline to update our lakebase tables. Finally, by leveraging Databricks Apps, we can deploy a FastAPI backend that uses WebSockets to stream real-time updates from Postgres, along with a front-end application to visualize the live data flow.

Before the introduction of the Zerobus SDK, the streaming architecture would have included multiple hops before it landed in the target table. Our API gateway would have needed to offload the data to a staging area like Kafka, and we would need Spark Structured Streaming to write the transactions into the target table. All of this adds unnecessary complexity, especially given that the sole destination is the lakehouse. The architecture above instead demonstrates how the Databricks Data + AI Platform simplifies end-to-end enterprise application development — from data ingestion to real-time analytics and implementation of interactive applications.

Getting Started

Prerequisites: What You Need

- Lakebase: Generally Available on AWS and Azure
- Zerobus Ingest: Generally Available on AWS and Azure
- Databricks Apps: Check that you have the permissions to create Databricks Apps

STEP 1: CREATE A TARGET TABLE IN DATABRICKS UNITY CATALOG

The event data produced by the client applications will live in a Delta table. Use the code below to create that target table in your desired catalog and schema.

SQL

```
CREATE TABLE <catalog>.<schema>.rideshare_events (
<YOUR_FIELDS>
)
TBLPROPERTIES (
   'delta.autoOptimize.optimizeWrite' = 'true',
   'delta.autoOptimize.autoCompact' = 'true',
   'delta.enableRowTracking' = 'false'
);
GRANT USE CATALOG ON CATALOG YOUR_CATALOG TO `<service-principal-
id>`;
GRANT USE SCHEMA ON SCHEMA YOUR_CATALOG.YOUR_SCHEMA TO `<service-
principal-id>`;
GRANT SELECT, MODIFY ON TABLE YOUR_CATALOG.YOUR_SCHEMA.YOUR_TABLE TO
`<service-principal-id>`;
```

STEP 2: AUTHENTICATE USING OAUTH

PYTHON

```
from zerobus.sdk.aio import ZerobusSdk
from zerobus.sdk.shared import StreamConfigurationOptions,
TableProperties

stream = await sdk.create_stream(
CLIENT_ID, CLIENT_SECRET, table_properties, options
)
```

STEP 3: CREATE THE ZEROBUS CLIENT AND INGEST DATA INTO THE TARGET TABLE

The code below pushes the telemetry events data into Databricks using the Zerobus API.

PYTHON

```
await stream.ingest_record(telemetry)
```

Change Data Feed (CDF) limitation and workaround

As of today, Zerobus Ingest does not support CDF. CDF allows Databricks to record change events for new data written to a delta table. These change events could be inserts, deletes, or updates. These change events can then be used to update the synced tables in Lakebase. To sync data to Lakebase and continue with our project, we will write the data in the target table to a new table and enable CDF on that table.

PYTHON

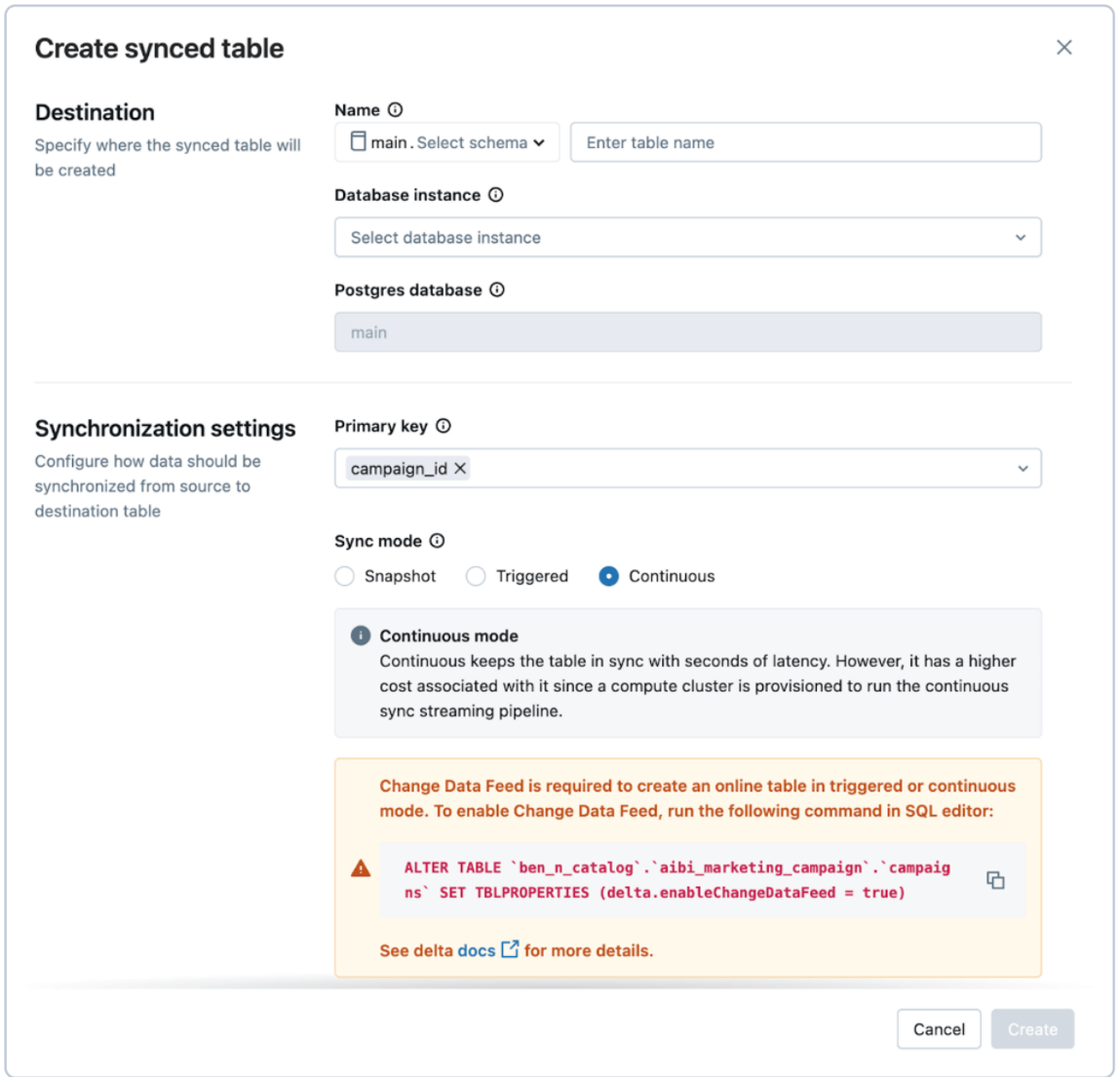
```
# read in target table

df = spark.readStream.table("catalog.schema.rideshare_events")

# write to a new delta table

df.writeStream.format("delta").option("checkpointLocation", "/
Volumes/catalog/schema/streaming_checkpoints").table("catalog.
schema.rideshare_events_processed")
```

SQL

```
# alter table and enable CDF
ALTER TABLE catalog.schema.rideshare_events_processed SET
TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

STEP 4: PROVISION LAKEBASE AND SYNC DATA TO DATABASE INSTANCE

To power the app, we will sync data from this new, CDF-enabled table into a Lakebase instance. We will sync this table continuously to support our near real-time dashboard.



In the UI, we select:

- **Sync Mode:** Continuous for low-latency updates
- **Primary Key:** table_primary_key

This ensures the app reflects the latest data with minimal delay.

Note: You can also create the sync pipeline programmatically using the Databricks SDK.

Real-time mode via `Foreach Writer`

Continuous syncs from Delta to Lakebase has a 10-15-second lag, so if you need lower latency, consider using real-time mode via ForeachWriter writer to sync data directly from a DataFrame to a Lakebase table. This will sync the data within milliseconds.

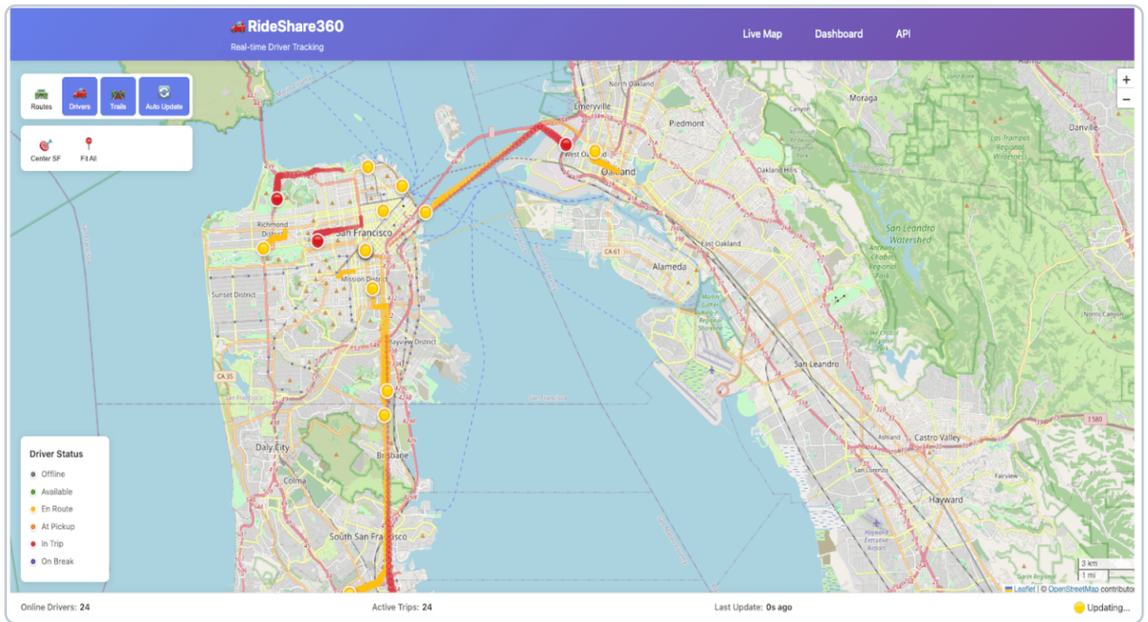
PYTHON

```
from lakebase_foreachwriter import LakebaseForeachWriter

writer = LakebaseForeachWriter(
    username="your-username",
    password="your-password",
    table="your_target_table",
    df=your_dataframe,
    host="your-lakebase.dns.databricks.com",  # Directly provide the
DNS
    mode="insert",  # Optional: defaults to "insert"
    batch_size=1000,  # Optional: defaults to 1000
    batch_interval_ms=100,  # Optional: defaults to 100
)

query = (
    streaming_df.writeStream
    .foreach(writer)

    .start()
)
```

```
query = (
    streaming_df.writeStream
    .foreach(writer)
    .option("checkpointLocation", "/tmp/spark_checkpoints/insert_
example")
    .start()
)
```

Refer to the Lakebase ForeachWriter code on Github.

STEP 5: BUILD THE APP WITH FastAPI OR ANOTHER FRAMEWORK OF CHOICE



With your data synced to Lakebase, you can now deploy your code to build your app. In this example, the app fetches events data from Lakebase and uses it to update a near real-time application to track a driver's activity while en route to making food deliveries. Read the Get Started with Databricks Apps docs to learn more about building apps on Databricks.

Additional resources

Check out more tutorials, demos and solution accelerators to build your own applications for your specific needs.

- **Build an End-to-End Application:** A real-time sailing simulator tracks a fleet of sailboats using Python SDK and the REST API, with Databricks Apps and Declarative Automation Bundles. Read the blog.
- **Build a Digital Twins Solution:** Learn how to maximize operational efficiency, accelerate real-time insight and predictive maintenance with Databricks Apps and Lakebase. Read the blog.

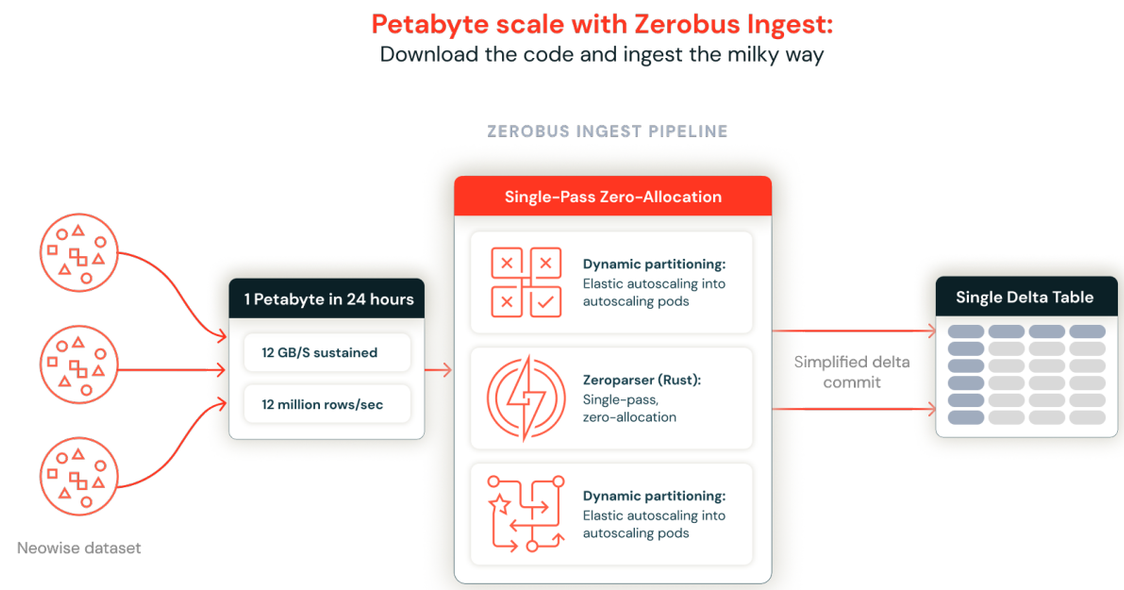
Learn more about Zerobus Ingest, Lakebase, and Databricks Apps in the technical documentation. You can also take a look at the Databricks Apps Cookbook and Cookbook Resource Collection.

Conclusion

IoT, clickstream, telemetry and similar applications generate billions of data points every day, which are used to power critical real-time applications across several industries. As such, simplifying ingestion from these systems is paramount. Zerobus Ingest provides a streamlined way to push event data directly from these systems into the lakehouse while ensuring high performance. It pairs nicely with Lakebase to simplify end-to-end enterprise application development.

Petabyte Scale With Zerobus Ingest: Download the Code and Ingest the Milky Way

By Victoria Bukta and Giselle Goicochea



We just published a deep-dive engineering post on Zerobus Ingest that covers how it works and the design choices that made it possible.

1 petabyte. 24 hours. 12 GB/s sustained. 12 million rows per second. Single table. Zero pre-configuration.

THE THREE ENGINEERING BETS THAT HELPED ZEROBUS INGEST DELIVER PERFORMANCE AT SCALE:

- **Dynamic partitioning:** Ordering is guaranteed at the stream connection level rather than the partition level, which means pods can be added *and* removed dynamically — true elastic autoscaling, not just scale-out.
- Zeroparser **(now open sourced):** A single-pass, zero-allocation protobuf decoder built in Rust that sustains ~1 GB/s per CPU core with fully dynamic descriptors — no compile-time schemas required.
- **Latency-optimized write-ahead log:** The WAL keeps clients lean — push data, get ack, free buffer. This low-latency, durable handoff is why teams prefer Kafka. Zerobus offers the same guarantee.

NASA's NEOWISE dataset is 11 years of Wide-field Infrared Survey Explorer data — 200 billion time-stamped detections of stars, galaxies and asteroids with a wide, complex schema that reflects what the universe actually produces. It's actual scientific data, not a synthetic workload, and it behaves exactly like production telemetry because it is production data.

That's why we picked it.

Run the benchmark yourself

Everything we used to produce the petabyte-run is in our sample repo:

- Dataset prep
- The Locust-based producer
- Instructions for pointing it at your own Zerobus endpoint

github.com/databricks-solutions/zerobus-ingest-examples — NEOWISE benchmark

The NEOWISE run was our test of what the system can do at scale, but your pipeline is the real test. Try taking our benchmark and swapping out the data for yours.

What comes next?

Zerobus Ingest is generally available, with Kafka Producer API support, MQTT, Avro, and more on the roadmap.

Beyond what's on our list, we want to know what's on yours. **Where do you want Zerobus to go next? What do you think the next frontier of streaming looks like?**

This community is our best signal. Drop a comment below or reach out directly to your account team— we're actively shaping what we build next.

GET STARTED:

- Docs: Zerobus Ingest overview
- SDK: github.com/databricks/zerobus-sdk
- Benchmark harness: NEOWISE benchmark on GitHub

databricks

Try the New Zerobus Rescue Column: Choose-Your-Own-Adventure Schema Management (Beta)

By Victoria Bukta and Andrija Stevanović

It is Friday afternoon. An upstream team ships a harmless-looking change: they add a region field to the events your service already emits, and somewhere along the way, a sensor starts reporting temperature as "72F" instead of 72. You did not write that producer. You do not control its release schedule. And yet by Monday, your ingestion is either dropping those records on the floor or flat-out stalling. Every data engineer has lived some version of this, and now has nightmares reliving this data disruption.

The fundamental question when designing these pipelines is: "**How strict should ingestion be about the shape of your data?**"

The Zerobus Rescue Column (Beta) offers a way to answer that question in a way that makes sense for your organization. This is a designated Variant type column used to "catch" any extra or schema-mismatched fields.

Two camps, one spectrum

I have heard both arguments, and honestly, both have merit.

One camp says, **"I don't want to stream garbage into my lakehouse. Gate it. If a record doesn't meet my contract, fail the producer; it's their problem."**

The other camp says, **"Accept everything. I would rather capture a messy record now and clean it up later than lose it forever. I don't control the producers, but I'm responsible for ensuring data lands."**

Most teams live somewhere on this **spectrum**, and where they sit changes based on how responsibilities are delegated across your organization. The design goal for Zerobus was never to pick a side for you. It was to let you choose and to make that choice a property of your table rather than the ingestion service, because we know you (as the data engineer) cannot change the political cards; they are simply dealt to you.

The table is the contract

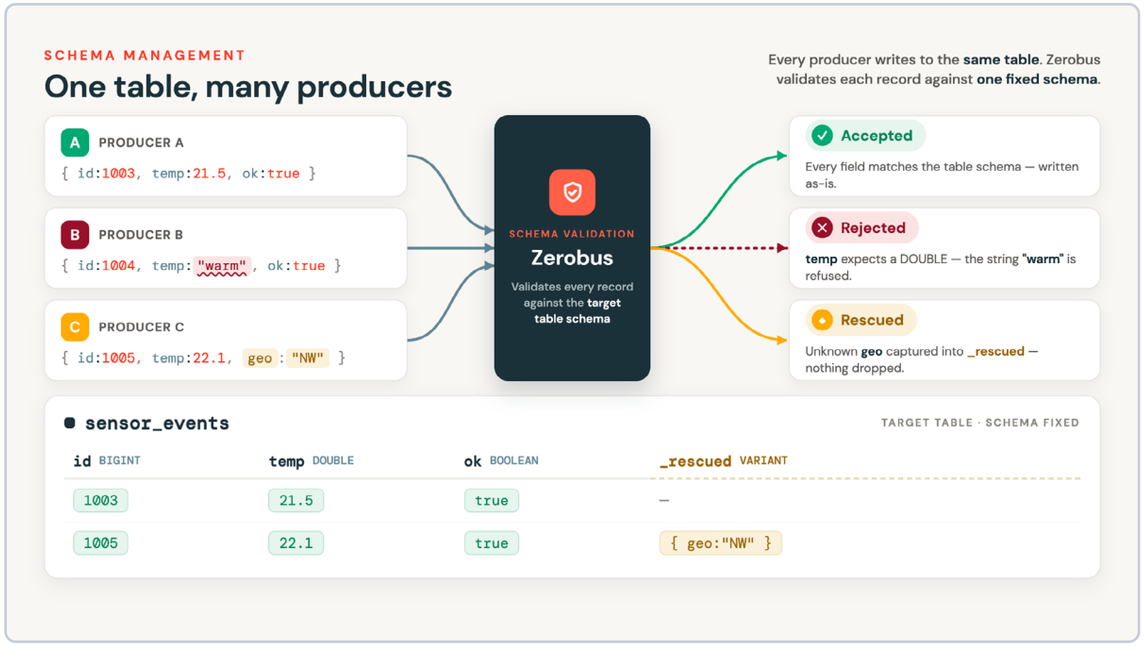
Your Delta table schema is the contract. Zerobus validates every incoming record against that contract, and you decide how strict or how accepting the contract is. Think of it as a **choose-your-own-adventure.** The same service can enforce a rigid schema, accept a flexible subset of columns or catch everything that does not fit, depending entirely on how you define your table.

Zerobus Ingest holds three fundamental principles:

- **Zerobus gates the data.** It validates every record against the target table and rejects anything that does not fit. It never guesses, and it never silently drops a column. It fails fast and loud!
- **You define the contract.** Marking columns as required or nullable and adding a rescue column are how you decide what "fits."
- **Zerobus never augments your table.** It does not add columns, change types or evolve your schema to accommodate a record. You evolve what Zerobus accepts by evolving the table, not the other way around, making your table the data contract.

That last principle is the one people underestimate: **Zerobus is predictable by design.** Its number one job is to land your data durably and predictably in the lakehouse. If it quietly reshaped your table every time a producer sent something unexpected, it would not be predictable for downstream consumers. Everything below is built on top of that guarantee.

Choose your own adventure: Schema edition



SCENARIO 1: ACCEPT A SUBSET (ALL COLUMNS OPTIONAL)

Make every column nullable. Producers can send any subset of the columns, and anything they omit is written as NULL.

```
CREATE TABLE main.default.air_quality (
  device_name STRING,
  temp INT,
  humidity INT);
```

- `{"device_name": "sensor-1", "temp": 22, "humidity": 55}` is **accepted**.
  - All columns present.
- `{"device_name": "sensor-1"}` is `accepted.`
  - Temp and humidity are nullable, so they are written as NULL.
- `{"device_name": "sensor-1", "temp": 22, "region": "us-west"}` is **rejected**.
  - Region does not exist in the table.

This is flexible about what you *omit*, but still strict about what you *add*.

SCENARIO 2: ENFORCE SPECIFIC FIELDS (THE NOT NULL GATE)

Mark columns NOT NULL to require them. Every record must supply those fields, or it is rejected. This is the **strict end** of the spectrum, and NOT NULL is your **gate**.

```
CREATE TABLE main.default.air_quality (
  device_name STRING NOT NULL,
  temp INT NOT NULL,
  humidity INT);
```

- `{"device_name": "sensor-1", "temp": 22, "humidity": 55}` is **accepted**.
  - All required columns present.
- `{"device_name": "sensor-1", "temp": 22}` is **accepted**.
  - Humidity is nullable, so it is written as NULL.
- {"device_name": "sensor-1"} is **rejected**.
  - Temp is required and missing.

This is the answer for the first camp. If you want to guarantee that garbage never lands, NOT NULL is the lever.

SCENARIO 3: CATCH EVERYTHING ELSE (THE RESCUE COLUMN)

Add a **VARIANT rescue column** to capture the fields that do not fit, instead of rejecting the record. Fields that match the table are written to their own columns as usual; any extra or non-conforming fields are grouped into the rescue column as a JSON object. This is the **most accepting** end of the spectrum, with nothing lost to rejection.

This is the answer for the second camp. It is also the new capability we are introducing today.

Rescue to the rescue

Earlier this year, we added Variant type support so you could ingest semi-structured JSON without pinning down every field in advance. **The rescue column is the natural extension of that work.**

The distinction is one of intent:

- **Variant** is for when you *expect* semi-structured data and want to store it natively. You reach for it deliberately.
- **The rescue column** is for when you *have* a schema you believe in, and you want a safety net for the strays.

You can schematize less up front and still keep that insurance policy. Anything extra a producer sends lands in the rescue column instead of being lost, and you keep the durability guarantee that matters most.

Configuring a rescue column

There is no magic column name. You designate a column as the rescue column by giving it three properties:

1. It **allows null values.**
2. It uses the **VARIANT type.**
3. It has the **zerobus-rescue tag** applied in Unity Catalog.

What lands in the rescue column, and what doesn't

Zerobus routes each field in a record based on how it fits the table:

Field in the record	Outcome
Matches a column by **name and type**	Written to that column normally
Not present in the table schema	Captured in the rescue column
Present in the schema, but the **value's type doesn't match** and the target column is nullable	Captured in the rescue column

Here is the whole thing in one example. Consider a table where **rescue** is nullable, uses **VARIANT**, and carries the **zerobus-rescue** tag:

```
CREATE TABLE main.default.air_quality (
  device_name STRING NOT NULL,
  temp INT,
  humidity LONG,
  rescue VARIANT
);
```

Now ingest a record with both a type mismatch and an extra field:

```
{
  "device_name": "sensor-1",
  "temp": "72F",
  "humidity": 87,
  "extra_field": "some value"
}
```

The result:

device_name	temp	humidity	rescue
sensor-1	null	87	{"extra_field": "some value", "temp": "72F"}

Both non-conforming fields, the mistyped **temp** and the unknown **extra_field**, are grouped into the rescue column's JSON object. That Friday-afternoon change from the opening? It lands cleanly, and you still have every byte to reconcile later.

Recommendations

A few things to keep in mind as you design tables around this:

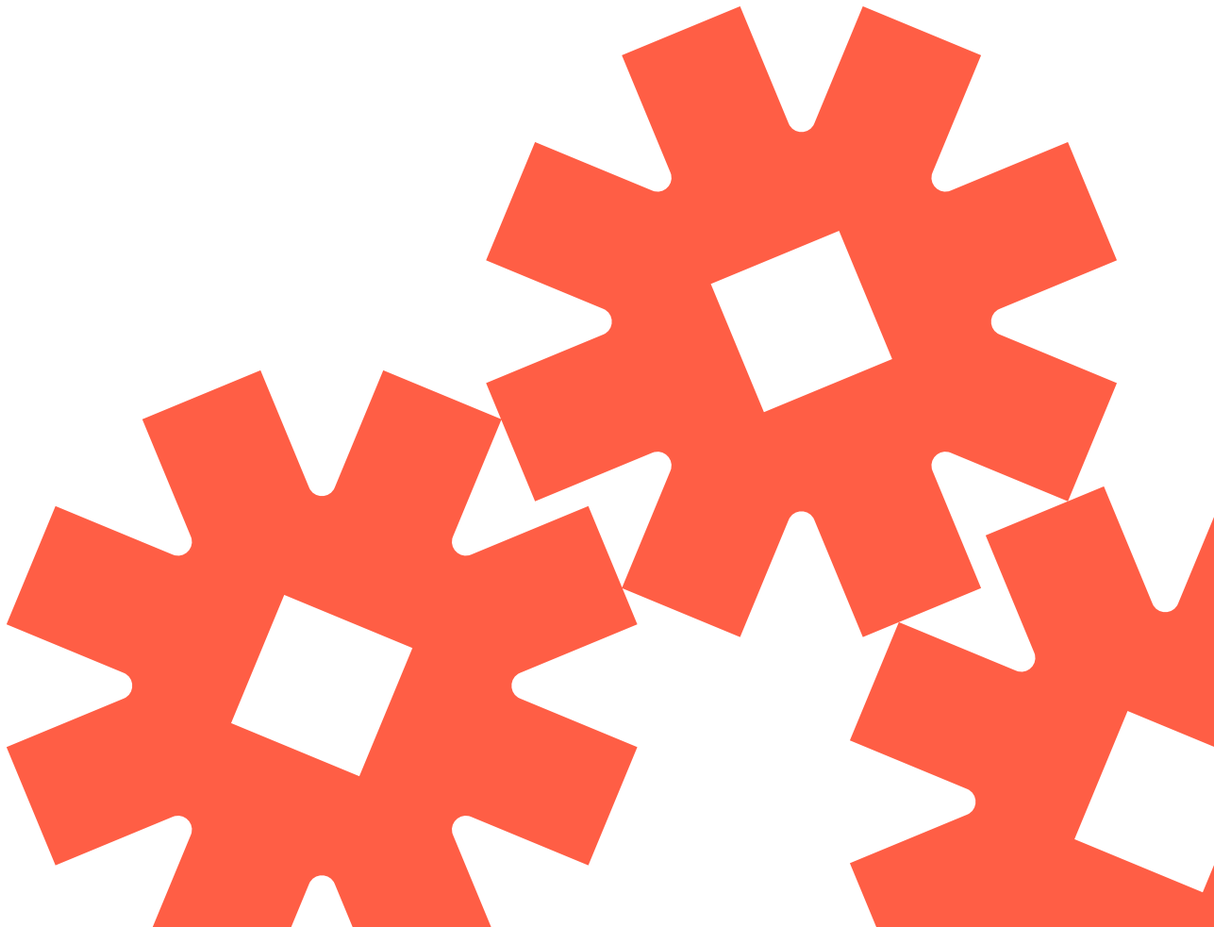
- **Evolve the table, not the ingest.** Zerobus never auto-evolves your schema. When your data shape changes permanently, evolve the table first (for example, with **ALTER TABLE**), then send records against the new schema.
- **Keep your schema backward compatible.** Adding a nullable column is non-breaking, so you can roll out table and producer changes on separate timelines.
- **The rescue column is a landing zone, not a destination.** Once data is durable, you can transform and schematize it downstream. A common pattern is to **fan out**: read the rescue column, promote fields that have become common into real typed columns, and route the rest onward. The rescue column buys you time to make those decisions with the data safely in hand.
- **Combine strictness with rescue deliberately. NOT NULL** on the fields you truly require, plus a rescue column for everything else, gives you a table that both **guarantees** your core contract and **never loses** the extras. That combination is where most teams end up.

Try it out today

Our whole philosophy is the following: **Zerobus makes your data durable and is reliable when it comes to landing your data. You decide how much to schematize along the way.**

Your adventure, your rules.

Ready to try it? Check out the Zerobus rescue column documentation to design your contract. Be sure to enable the Beta feature first.



03

On the Agentic Roadmap

What's new in Genie Code at Data + AI Summit 2026

Manage, govern, and orchestrate longer-running data and ML development workflows on Databricks.

By Julia Powell, Gal Oshri and Weston Hutchins

Genie Code helps data and ML teams build and improve systems faster on Databricks. Over the past year, Databricks' Genie products have grown over 10x and are used by 90% of Databricks customers. Teams are using it to build models and pipelines, debug failures, create dashboards, analyze data in notebooks, and improve production systems.

At Data + AI Summit 2026, we're expanding Genie Code for more complex, agentic data and ML work. We're introducing a new full-page command center, upgrades for production data and ML engineering, and scheduled tasks.

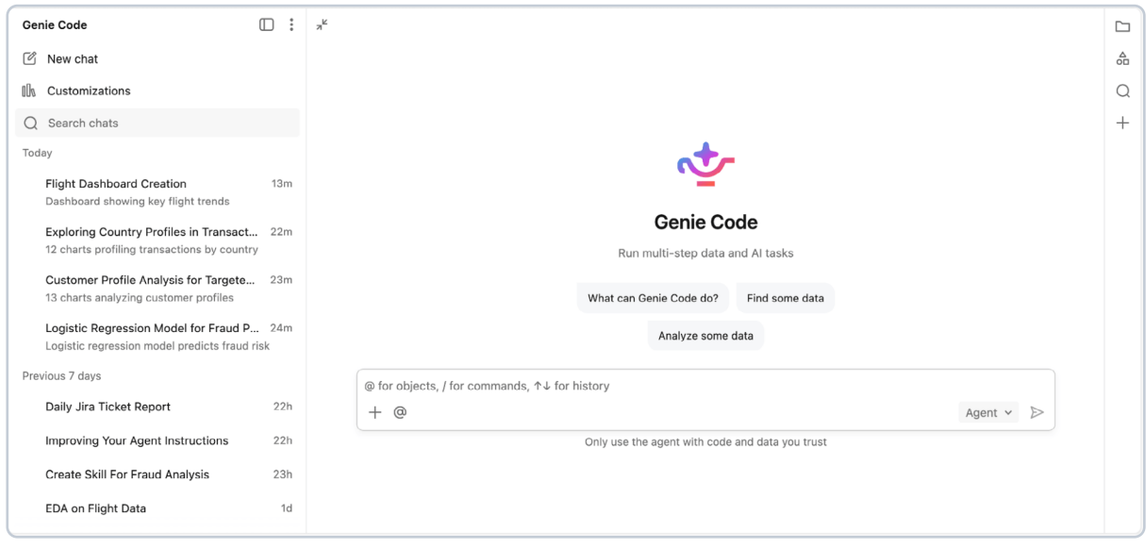
These updates are part of a broader shift across Databricks toward AI-native data and ML workflows. Genie Code helps data teams build, debug, and improve data and ML systems, and we introduced Genie ZeroOps to extend agentic automation to operations. Together, these products help teams move faster across the full lifecycle, from building systems to operating and improving them over time.

Here's what's new:

Manage complex work from a full-page command center

Data and ML development rarely happens in a single prompt. A user may need to inspect existing logic, update multiple assets, run code, review outputs, and refine the next step based on results. That work can span notebooks, SQL, Lakeflow pipelines, dashboards, jobs, models, serving endpoints, and Unity Catalog assets.

We've redesigned the Genie Code experience to give teams a dedicated command center for this kind of complex data and ML work. Instead of managing longer tasks in a smaller side panel, users can use a full-page experience to describe a task, track progress, review outputs, and continue iterating.



Teams can manage multiple Genie Code threads, see when a thread is executing or waiting for input, and return to each one when new results are ready. They can rename threads, search previous conversations, and stay oriented as projects evolve.

BOSCH

"Genie Code has completely changed how I work. I run upwards of 15 parallel threads scoped to different notebooks and assets every day, and managing all of that across tabs is one of the biggest sources of friction in my workflow. Full page Genie Code with concurrent sessions would give me a true workspace for running everything in parallel without constantly losing context.

—Moritz Schiek, Solution Consultant, Bosch

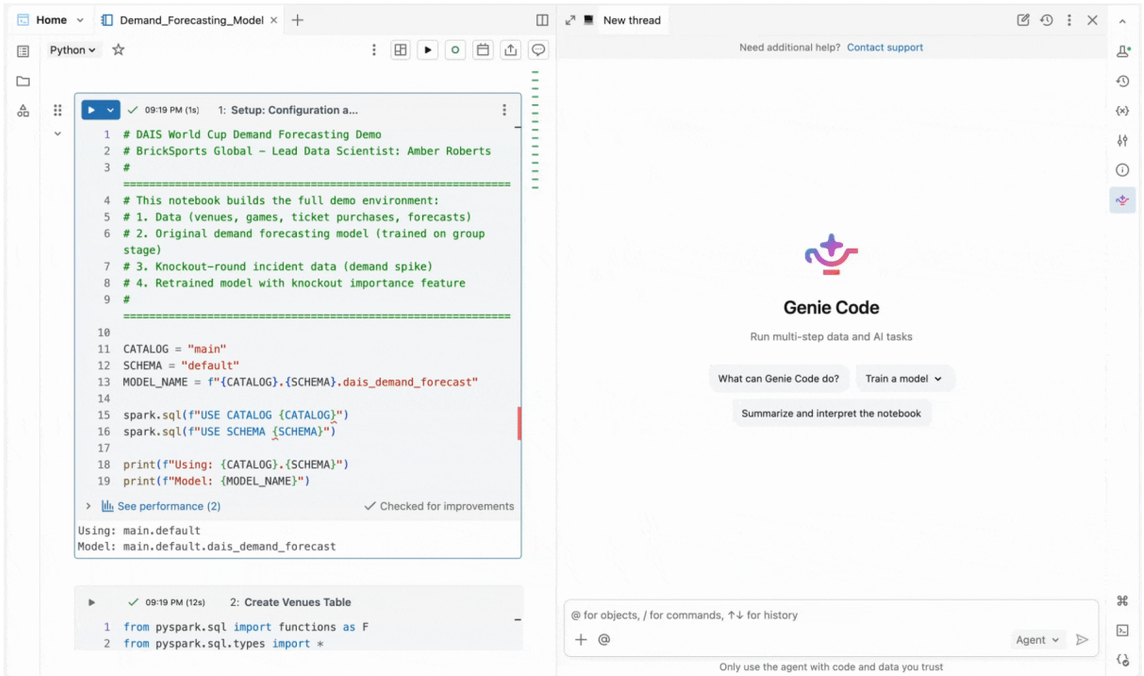
The command center also makes customization easier to discover. Instructions, skills, and connectors are more visible, so teams can guide Genie Code with the right standards, tools, and workspace knowledge.

To access the full-page command center, open the Genie Code side panel and click the maximize button in the top left.

Bringing agentic development to ML workflows

In an ML project, the model is the small part. Most of the time goes to the engineering around it: turning raw data into features, running experiments, getting a candidate to production, and keeping it healthy once real traffic arrives. That work is slow and expensive, and it is why most teams run far fewer models than they have use cases for.

Genie Code for machine learning goes after that engineering. It is a set of capability and intelligence upgrades built into Genie Code, so you do not adopt a new tool. The same agent you already use becomes a specialist for production ML engineering across your existing Databricks ML stack.



databricks

Genie Code's expertise comes from two places. The first is Databricks. We have run production ML with customers for more than a decade, and we have seen where models break, where teams lose time, and what separates a model that works from one that looks right and fails without warning. Genie Code applies those lessons as it works, handling the details a seasoned practitioner would, like correcting for class imbalance and checking feature quality.

The second is your team. A generic coding agent has not seen your past experiments, your business metrics, your evaluation sets, or how you weigh one objective against another, so it guesses. Genie Ontology closes that gap. It learns how your team builds features, trains models, and evaluates candidates, and Genie Code follows those patterns instead of falling back on irrelevant defaults.

With both kinds of knowledge, Genie Code is a stronger partner for day-to-day model development. It writes features in your team's patterns, makes coordinated edits across the many files involved, runs and debugs your code, and compares candidates with your own evaluation scripts. You stay in the loop and decide what to keep.

Genie Code is now natively integrated with the entire Databricks ML stack. The latest upgrades:

- **MLflow.** Genie Code reads your experimentation and observability data: runs, artifacts, model lineage, quality metrics, and system metrics. Ask it "How do I improve GPU utilization during training?" or "What other metrics should I track for this model?" and get answers grounded in your own runs.
- **Model Serving.** Genie Code inspects endpoint health and performance, diagnoses serving issues, and finds ways to optimize a running endpoint.
- **Compute awareness.** Genie Code moves to AI Runtime when a job needs a GPU for training, and uses workspace environment features to set up the environment, so you skip the infrastructure setup.

The result is an agent that finishes real-world data science tasks far more often than a generic coding agent.

"With Genie Code, we moved from raw data to a governed, production-ready ML workflow in 90 minutes. Because it uniquely understands production ML workflows on Databricks, it helped us create Delta tables, explore the data, train and compare models, register them with MLflow and Unity Catalog, and deploy the champion model to a serving endpoint, with time left to optimize for the business outcome that mattered most."

—Radu Dragusin, Principal Engineer, Data & AI, Danfoss

Let Genie Code work autonomously with scheduled tasks

Until now, Genie Code has been primarily interactive: you ask, it responds, and you stay in the loop as the work progresses. Scheduled tasks change that.

Coming soon, scheduled tasks will let Genie Code do work on your behalf even when you are not at your laptop, then hand you the results to review when you return. A scheduled task starts with a prompt and, optionally, a relevant asset such as a notebook, workflow, or dashboard. When it runs, Genie Code creates a thread with the results, which users can review, refine, or continue interactively.

For example, a data team could ask Genie Code to check overnight job results, summarize pipeline runs, explain a change in a dashboard metric, prepare a weekly analysis, or review model performance before a team meeting. The user does not need to rerun the prompt manually or stay in an active chat while the work happens.

Scheduled tasks move Genie Code from interactive assistance toward autonomous work. They help teams keep important workflows moving while results stay visible, reviewable, and grounded in Databricks context.

Genie ZeroOps extends this approach into production operations. It watches live systems, investigates issues, and prepares fixes for teams to review and approve. For ML systems, that can include model drift, serving errors, and upstream pipeline problems. For data engineering systems, it can help teams move from monitoring and diagnosis toward repair and optimization.

Try Genie Code

If you have a Databricks workspace, you already have Genie Code. Open it in your workspace to try the full-page experience today. To see how the Genie family extends into production operations, continue on to the next page to read the Genie ZeroOps launch blog.

Introducing Genie ZeroOps: Put your data and AI operations on autopilot

An AI background agent that monitors your production workloads, investigates issues, and suggests fixes you can verify

By Bilal Aslam, Lennart Kats, Ray Zhu, Mike Del Balso and Ori Zohar

Data and AI work has always had a maintenance problem. Data pipelines break all the time due to not only code issues but also data problems such as upstream schema changes or late-arriving data. ML models drift, and degrading models keep serving confident, wrong answers long before anything throws an error. The burden of keeping data and AI assets running in production is falling on data teams, and it's only growing. The rise of LLMs and agentic tools has made it faster than ever to build pipelines and ship models. As a result, data teams report spending most of their time fighting fires rather than building.

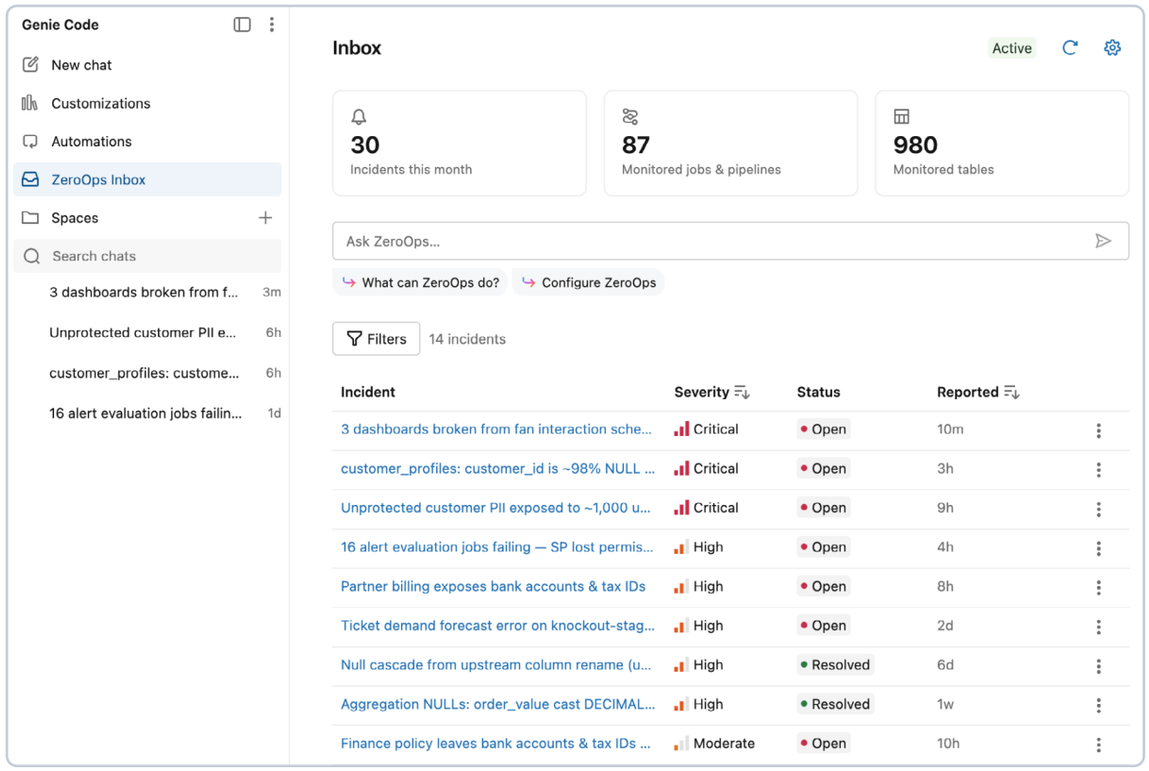
Agentic operations with Genie ZeroOps

To help data teams with this operational burden, we've built Genie ZeroOps: an autonomous background agent that monitors your data and AI assets (such as pipelines, jobs, tables and ML models) and takes action before or when things go wrong. Because it runs inside Databricks, it has secure and easy access to:

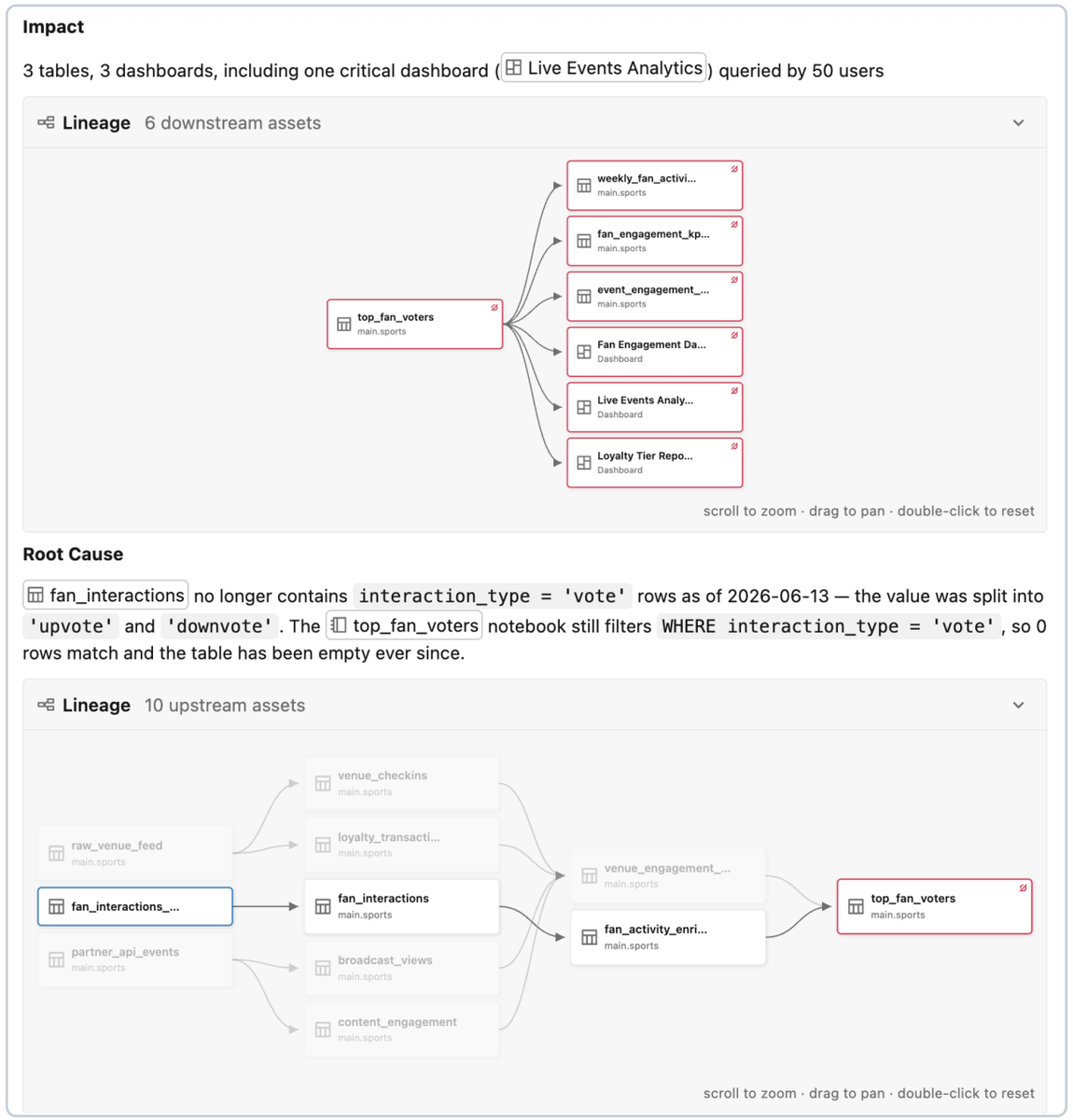
- **Full observability:** metrics, events, logs, and run history from the platform's observability layer.
- **Data lineage through Unity Catalog:** the complete dependency graph of every asset, so it can trace failures to their true root cause.
- **Sandbox environments:** Genie ZeroOps shallow clones production data (creating a table clone using metadata without duplicating the underlying data) into an isolated environment, applies permission guardrails and network isolation, and validates a proposed fix against real data without touching production.

Here's the process it runs for every failure:

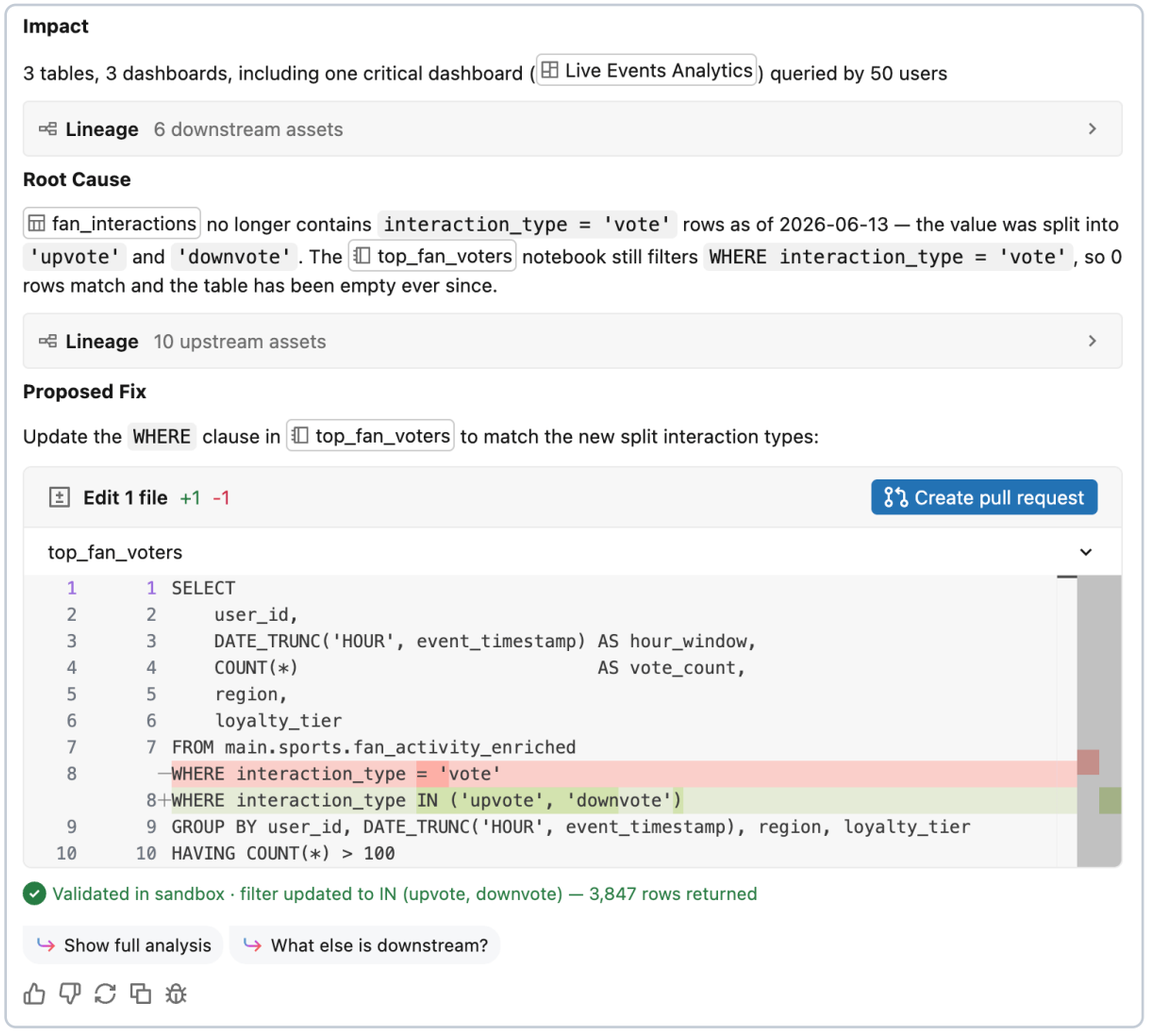
- **Detect:** Continuous monitoring with access to platform observability, including silent failures that show up in data quality metrics before they throw any errors.
- **Assess:** Unity Catalog lineage gives Genie ZeroOps the full dependency graph. It can trace a failure to a code bug, a schema change three tables upstream, or bad data introduced by another pipeline.
- **Remediate:** Agentic code generation produces the fix, with your development workflow (GitHub PRs, Jira tickets) as context.
- **Verify:** Genie ZeroOps runs a secure sandbox with zero-copy clones of your data, scoped permissions, and network isolation. The proposed fix runs against real data there, never against production, and nothing is applied until you approve it.



Genie ZeroOps inbox UI showing incidents ordered by severity



Genie ZeroOps shows you a visualization of impacted assets and the root cause analysis it performed using lineage data



Suggested fixes are provided with an indication of sandbox validation

Why coding agents can't solve data and AI operations

Why do you need a purpose-built agent for data and AI operations? Can't you use the same coding agent that helps you build software and get the same results? The answer is – "no, not really".

Coding agents were built for software engineering, but data engineering and AI are fundamentally different:

- **The context includes data, not just code.** Pipeline failures are often caused by schema changes upstream, bad data propagating through a dependency chain, or silent corruption. None of which code alone can tell you about.
- **Failures can be silent and permanent.** A data bug can sit quietly in a production table for weeks, poisoning downstream consumers. By the time you find it, the business implications have materialized.
- **Production data is sensitive and governed.** Unlike code, it can't be freely copied, shared, or handed to an outside tool.

When something breaks, you need to: detect it, assess root cause, remediate with a fix, and verify it works without side effects.

Examine each step, and you'll find coding agents typically fall short. For detection, they can lack context, such as telemetry or choke on extremely large context, like Apache Spark™ logs. For assessment, finding the root cause and its impact, they often lack access to lineage data. They also don't have a purpose-built harness for data and AI work, which makes the process more costly and time-consuming. Coding agents can write code for remediation, but they often lack the context to do it right and can't fix issues that are data-related. But the step that is most challenging for coding agents is verification.

Verification requires testing code fixes against real production data in an isolated environment. You can't give an external agent access to production data, and even if you did, running code against it risks side effects that can have devastating consequences.

For an agent to safely handle the verify step, it needs to be part of the data platform itself. Genie ZeroOps is part of the Databricks Data + AI Platform, and that's what makes it succeed where coding agents fail.

Machine learning workloads in particular showcase the benefits of a purpose-built agent for operations work.

Genie ZeroOps for machine learning

Production ML introduces some additional challenges to data engineering. A model can have no pipeline errors and still be producing bad predictions, which means keeping pipelines running isn't enough, you need to watch whether the model's outputs are still trustworthy.

When they aren't, Genie ZeroOps diagnoses the cause, builds a corrected candidate, and validates it before it touches live traffic. For a pipeline fix, it validates against a shallow clone of a table. For a model, it trains a candidate on corrected features and evaluates it against the same eval suite and criteria the production model was held to -- not a generic benchmark. It surfaces the candidate only if it's measurably better, and lets you ramp it on live traffic before it takes over.

What makes those fixes trustworthy is context. Genie ZeroOps for ML is built on the same foundation as Genie Code, Genie Ontology and native integration with the Databricks ML stack (Feature Store, MLflow, model serving, notebooks). It knows which features your model uses, how your team evaluates it, and what 'good' means for your business, so it reasons the way your senior ML engineers would.

You stay in control

You configure which assets Genie ZeroOps monitors and what it's authorized to do. Everything runs under Unity Catalog governance, so it can only access data your own credentials allow. Issues surface in an inbox-style UI, prioritized by severity, each with a root cause analysis and a proposed fix. Nothing gets applied to production without your approval.

The sandbox is the technical trust layer. Shallow cloning means the fix is tested with real data but production is never touched. Scoped permissions and network isolation mean the sandboxed environment can't reach outside its boundaries. What was tested is exactly what gets applied.

This is the value of Genie ZeroOps - it lets you scale your operations safely. It does the heavy lifting while you stay in control.

Genie ZeroOps is coming soon

Genie ZeroOps is entering private preview in the coming weeks, starting with support for jobs, pipelines, tables and ML workloads. Apps, and Lakebase databases are on the roadmap.

Talk to your Databricks account team to request early access. In the meantime, explore other members of the Genie family like Genie One and Genie Code.

04 Case Studies

Panasonic | CUSTOMER STORY

Panasonic accelerates enterprise BI with Lakeflow

INDUSTRY Manufacturing

CLOUD Azure

PRODUCT Connect

2 minute
SAP data ingestion

50%
Reduction in load times

Zero
Reliance on legacy ETL

databricks

> By moving from a rigid legacy ETL stack to the Databricks Data + AI Platform, our BI teams can now easily discover and access critical data, cutting Power BI refresh times by 50%. We're turning external, inconsistent data into trusted, production-grade assets that unlock new business insights and strengthen Panasonic's competitive edge.
>
> **— Jerry Deng, BI Director, Panasonic**

Panasonic US's central data infrastructure team has an ambitious mandate: serve as the data backbone for multiple business units spanning sales, supply chain, HR, and beyond. When legacy ETL pipelines and fragmented data warehouses slowed down daily reporting, sometimes with multi-hour ingestion windows and unpredictable failures, the team made a strategic decision to modernize from the ground up. By standardizing on the Databricks Data + AI Platform and Lakeflow, they transformed a fragile, siloed stack into a reliable, enterprise-wide data foundation. Processes that once took hours now complete in minutes, analysts have direct access to data that was previously out of reach, and the team is already building toward its next frontier: AI.

Fragmented legacy pipelines disrupt cross-functional business operations

Panasonic's central Data and IT infrastructure team drives the overarching data strategy for multiple internal companies and business units. To support daily operations, sales forecasting, and supply chain management, business leaders rely heavily on enterprise systems such as SAP S/4HANA, Workday, and global point-of-sale (POS) systems. However, Panasonic's legacy data stack, composed of disconnected ETL tools and complex data warehouses, struggled to handle the volume and complexity of this data, leading to severe performance and resiliency issues.

The most critical bottleneck was SAP data ingestion. Recognizing the limitations of legacy Change Data Capture (CDC) processes, the engineering team sought a more reliable approach, but the existing architecture forced them to run full data refreshes across over 100 tables every day. Massive transactional tables with hundreds of millions of rows required complex partitions and frequently caused legacy pipelines to fail. These heavy loads took five to six hours to complete and broke roughly 10 times a year, requiring hours or full days of IT-intensive cross-team troubleshooting to fix. For one of these internal business units, which operates under strict early-morning reporting cut-offs, these outages caused costly delays. Upper management was frequently left without the daily sales, inventory, and logistics reports needed to make critical business decisions, effectively disrupting daily operations. Furthermore, valuable data remained locked away in legacy database silos, creating visibility barriers for downstream Business Intelligence (BI) analysts who needed access to raw data for accurate forecasting.

Standardizing enterprise ingestion with Lakeflow Connect

To establish a resilient, centralized data backbone, Panasonic migrated to the Databricks Data + AI Platform, actively using Lakeflow Connect to standardize data ingestion across major data sources in the enterprise.

The most urgent priority was SAP S/4HANA. By integrating with SAP Datasphere to land files in Azure Data Lake Storage (ADLS), Panasonic deployed Auto Loader (part of Lakeflow Connect) to seamlessly handle incremental ingestion for one of their most failure-prone pipelines. This modern architecture provided a stable, automated architecture that required a fraction of the overhead. The results were immediate.

From there, the team extended the same approach across other critical systems. HR and workforce data that had been previously difficult to track historically is now ingested with the Workday connector into structured, relational tables that capture changes like manager transitions and employee rehires over time. In addition, the SFTP connector continuously pulls global supply chain data, including point-of-sale figures, shipping metrics, and manufacturing records from Panasonic's Japan headquarters. This time-sensitive data, formatted in CSV and Excel files, is made available for near real-time supply chain analytics.

Beyond structured and semi-structured data, Panasonic is now tackling the challenge of unstructured PDF document repositories. By connecting SharePoint to the Databricks environment, the team has automated the processing of thousands of complex legal and supplier documents. Using Databricks Document Intelligence (ai_parse_document and ai_query), they were able to process and extract dozens of key fields with high precision into structured outputs, transforming static documents into live, queryable data.

"Processing 10,000 supplier contracts and MSAs used to be a two-week manual ordeal on our legacy stack. By leveraging the Databricks SharePoint connector, serverless compute, and Databricks Document Intelligence, we've compressed that entire workflow—from ingestion to extracting critical expiration dates—into just two to three hours." – Shingo Sakamoto, IT Principal Data Architect, Panasonic

Underpinning all of it is a unified governance layer via Unity Catalog, which enables the team to securely share data across business units without duplication. Across all these sources, the team leverages Databricks serverless compute to execute highly performant ETL notebooks in a fraction of the time.

"In our legacy environment, massive SAP tables required five to six hours to load and failed frequently. By standardizing on Databricks and using Auto Loader, ingestion for our largest table dropped from hours to just two minutes. Our pipelines are completely stable now, guaranteeing our leadership has on-time reporting." – Yuka Kato, Lead Data Engineer, Panasonic

Trusted data, enterprise-wide impact

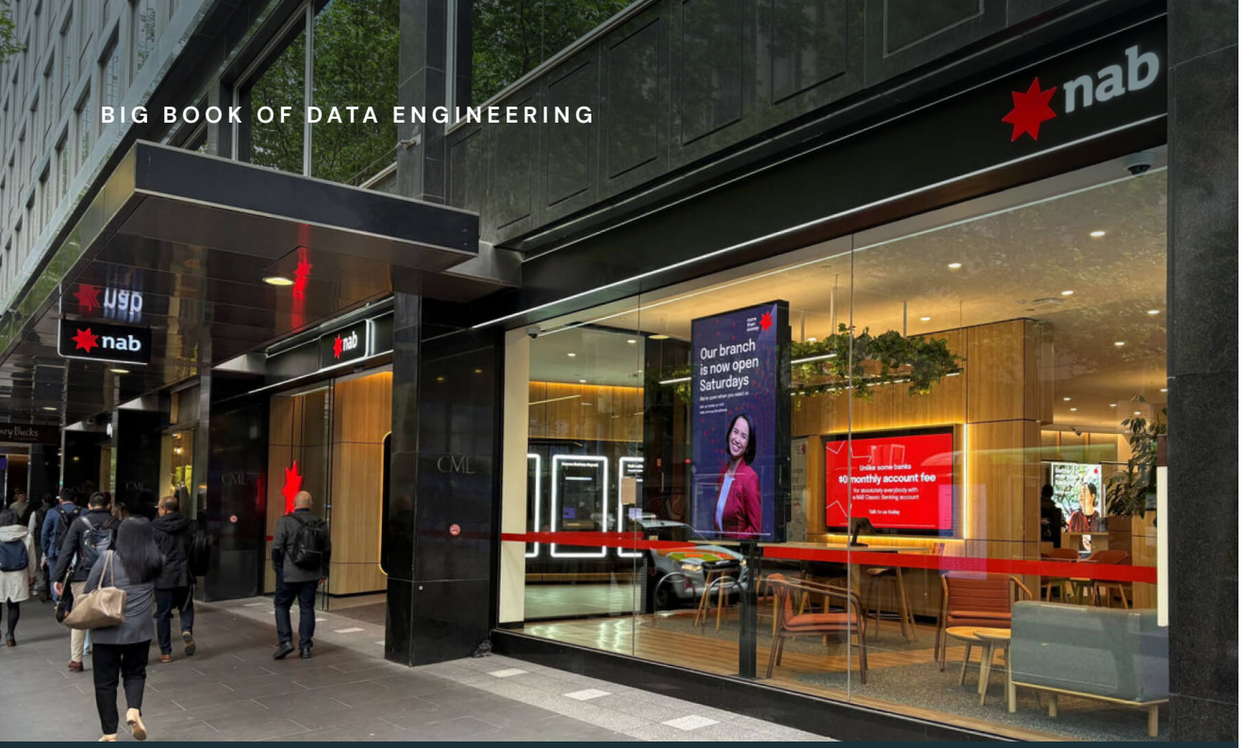
Today, Panasonic's business leaders start every morning with what they need: accurate, on-time reports covering daily sales, billing, and inventory, delivered without fail. End-to-end data processing for all silver tables completes in roughly 30 minutes, and the reliability that once felt out of reach has simply become the new standard.

The impact goes beyond performance. By retiring expensive legacy data warehouse, ETL, and BI licensing, the team achieved a significant reduction in TCO. As a result, this freed up budget and bandwidth, allowing teams to focus on higher-value work. And with a trusted data foundation in place, access has opened up across the organization. BI analysts can now explore data directly, cutting load and refresh times by roughly 50%. Sales representatives and regional managers are building their own views and forecasting models.

"Databricks has empowered our data analysts to do more: they can explore raw data directly, collaborate in shared notebooks, and move faster than ever. As a result of this operational efficiency, our small data science team can tackle solutions at enterprise scale." – Jerry Deng, BI Director, Panasonic

With a stable, unified data foundation in place, that same spirit of access is shaping Panasonic's AI ambitions. The team is implementing a Genie workspace to give its non-technical Quoting Team self-service access to pricing history and predictive insights.

"Our Quoting Team doesn't think in SQL; they think in customers and products. Genie meets them where they are, turning pricing questions into instant answers, enabling a small data team to deliver enterprise-wide impact." - Elena Gusakova, Senior Data Scientist, Panasonic



nab | CUSTOMER STORY

How NAB's journey to 100% declarative pipelines is helping data flow like electricity

INDUSTRY Financial Services

CLOUD AWS, Azure, Google Cloud

PRODUCT SDP

1,800
Spark Declarative Pipelines running on Databricks Lakeflow

99.6%
Job success rate

80%
Less transformation complexity

databricks

> Lakeflow helps our team level up. I don't need to spend another three months training and coaching people – everything is unified in one framework.
>
> **— Dheeraj Puli, Head of Data Reliability Engineering, National Australia Bank**

National Australia Bank (NAB), Australia's largest business bank, is modernizing its enterprise data platform by standardizing on Spark Declarative Pipelines across more than 1,600 data pipelines used by 300–400 engineers.

After years of managing complex, hand-written Spark and legacy data warehouse ETL, NAB is moving to a fully declarative, streaming-first architecture - reducing latency, improving data quality, and dramatically simplifying how data pipelines are built and operated at scale.

Overhauling a data strategy with aggressive goals

Leading this transformation was the key priority for Dheeraj Puli, Head of Data Reliability Engineering. When he joined three years ago, Puli immediately recognized NAB's advantage.

"I was delighted to see that everything was already in the cloud, while other banks were still trying to find ways to get there," he recalls.

Despite being standardized on Databricks, NAB faced a familiar challenge at scale: hundreds of engineers building Spark pipelines in different ways, with limited consistency, guardrails, or reuse. The bank needed a single way to run, standardize, and productionize Spark workloads - from ingestion to transformation - without slowing teams down.

Lakeflow became that unifying layer, providing a consistent foundation for how data engineers access data, build pipelines, and operationalize Spark across the platform.

Puli's mandate was to build and scale ADA, NAB's central data platform. His team of roughly 20 engineers supports hundreds of data engineers while enforcing a simple design principle: latency should never exceed processing time.

"If processing takes 20 minutes, the data should be available in 20 minutes," says Puli.

From custom Spark to Declarative Pipelines

To help hundreds of engineers converge on a single, consistent way of building pipelines on Lakeflow, NAB adopted Spark Declarative Pipelines - a set of declarative APIs in Apache Spark - and used Lakeflow to run and operationalize those pipelines with built-in orchestration, scaling, and reliability on Databricks.

Previously, NAB relied heavily on hand-written Spark and legacy SQL. Bronze ingestion was standardized, but Silver and Gold transformations used a custom Spark framework, creating significant complexity.

"In our Spark world, we had DMLs with 5,000 to 8,000 lines of code," Puli says. "Some pipelines had 64 union-all to handle complex business logic.

"I call Spark Declarative Pipelines hyper-standardization. There's only one way to do things - and that consistency is what we were missing."

With declarative pipelines, engineers define what the data should look like - target tables, merge keys and business logic - while Lakeflow handles orchestration, state management, and incremental processing automatically. Using this approach, NAB reduced some pipelines from 64 union-all down to just three, dramatically lowering operational risk and cognitive load.

Building an end-to-end streaming architecture

NAB began by standardizing Bronze ingestion on Spark Declarative Pipelines, onboarding 120 data sources in the first year. Today, 100% of pipelines run declaratively at the Bronze layer, with roughly 50% migrated at Silver.

The long-term goal is a fully declarative, end-to-end streaming architecture-from Bronze to Gold.

Declarative pipelines also enabled incremental processing and built-in change data capture. Instead of reprocessing entire datasets, pipelines process only new or changed records-reducing cost and improving performance and reliability. Lakeflow's built-in CDC capabilities, including AutoCDC, further eliminated the need to hand-code SCD logic that previously spanned thousands of lines of Spark and SQL.

Tangible operational and business impact

While NAB is still early in its journey, the results are already clear:

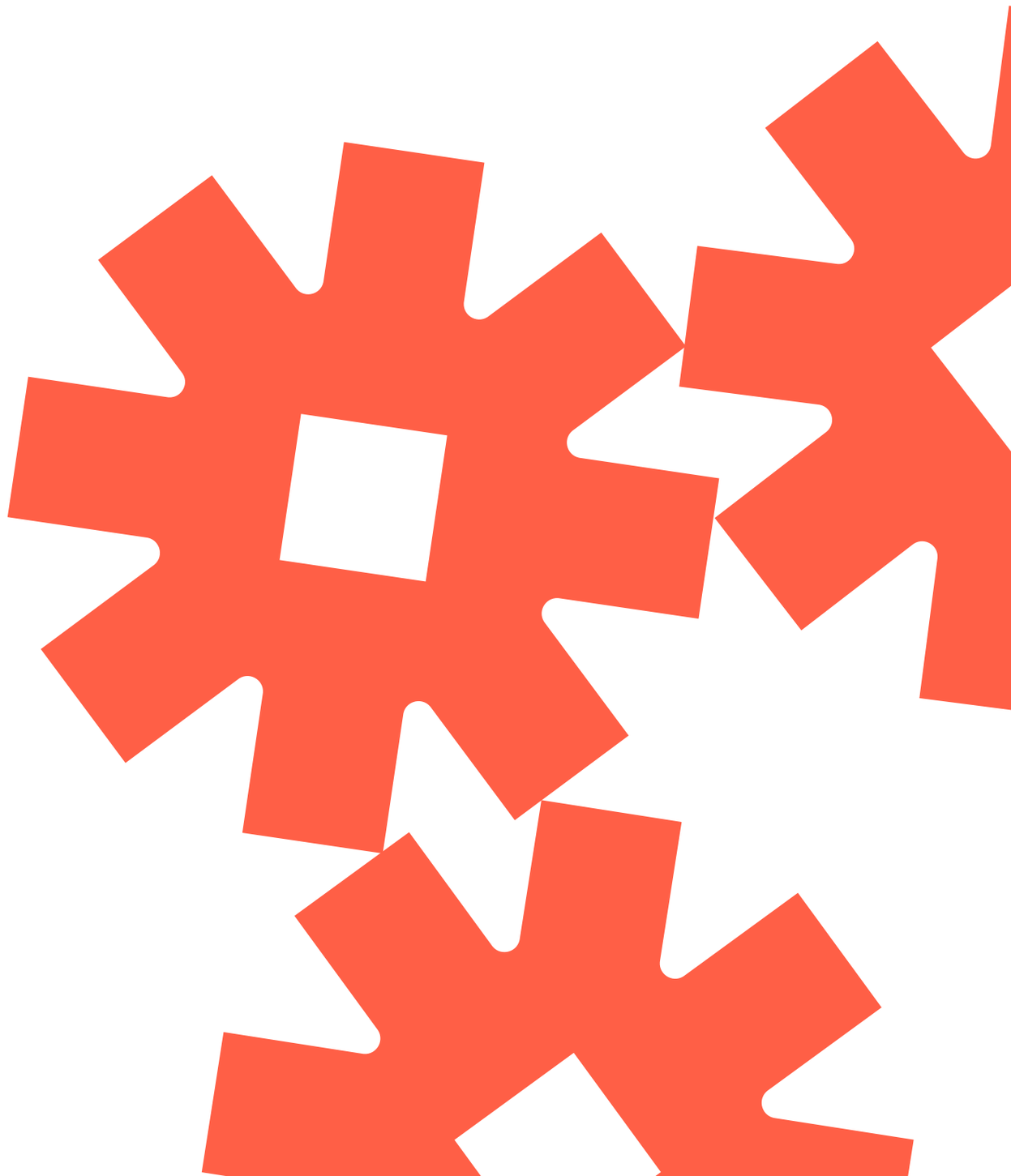
- Latency has dropped dramatically, now supporting under 15 minutes for end to end use cases
- Pipeline success rates have increased to 99.6%
- Data quality has improved by 38%
- Onboarding is faster, replacing months of custom training with a single framework
- Significant cost reductions are expected as adoption expands

Unlocking self-service and what's next

As adoption grows, NAB is preparing to extend Spark Declarative Pipelines fully into the Gold layer, unlocking new levels of self-service for business users.

Looking ahead, NAB aims to become the first bank to run 100% of its pipelines on Spark Declarative Pipelines, setting a new standard for how large financial institutions build, operate, and scale data platforms.

"This is a rapidly evolving product," Puli concludes. "We're going to keep being curious - there's a lot more we haven't explored yet."



coinbase | CUSTOMER STORY

Databricks powers real-time fraud detection at Coinbase

INDUSTRY Financial Services

CLOUD AWS

PRODUCT RTM

<100ms
P99 latency achieved at massive scale

99%
Online/offline feature consistency across models

51%
Estimated annual reduction in compute costs

databricks

> By leveraging Real-Time Mode in Spark Structured Streaming, we've achieved an 80%+ reduction in end-to-end latencies, hitting sub-100ms P99s, and streamlining our real-time ML strategy at massive scale. This performance allows us to compute over 250 ML features all powered by a unified Spark engine.
>
> **— Daniel Zhou, Senior Staff Machine Learning Platform Engineer, Coinbase**

Coinbase's mission is to increase economic freedom in the world by providing a trusted platform for crypto assets, including trading, staking, safekeeping, spending, and fast, free global transfers. To protect users from fraud and power personalized recommendations, Coinbase requires sub-second precision for its machine learning models. However, microbatch architectures primarily intended for ETL use cases, instead introduced latency that impacted model accuracy and drove up compute costs. By migrating to Spark Structured Streaming Real-Time Mode on Databricks, Coinbase transformed its data infrastructure—reducing feature computation latency to milliseconds, achieving 99% feature consistency, and saving hundreds of thousands in infrastructure costs to power more accurate real-time fraud detection at scale.

Stale data left fraud models a step behind

Coinbase leverages machine learning to power primary use cases such as fraud detection, catching suspicious transactions and mitigating anti-money laundering risks. Delivering these capabilities requires highly accurate ML models functioning in near real-time.

Prior to adopting Real-Time Mode (RTM), the platform team at Coinbase had optimized Spark Structured Streaming in microbatch mode (MBM) as far as the architecture would allow. Specifically, the team built innovative solutions to maximize each millisecond of MBM, eventually reaching sub-second freshness (~800-900ms), but at a costly operational burden. When delays happened, it negatively impacted the online and offline feature consistency of their models, hurting accuracy across several risk models.

Powering sub-second precision with Spark Real-Time Mode

To overcome these latency and cost hurdles, Coinbase transitioned its critical risk models to Spark Real-Time Mode (RTM) on Databricks. Adopting RTM was straightforward: the engineering team only needed to update their trigger type, allowing their core business logic to remain completely unchanged. This seamless shift to RTM enabled a dramatic improvement in performance, moving from microbatch processing to real-time streaming, jumping from 800+ms to 100-250ms at massive scale.

This transition immediately improved the freshness of the data feeding into their ML pipelines, driving consistency to produce models that accurately reflect operational systems synced in real time. To ensure smooth adoption, the platform team implemented Continuous Integration (CI) guardrails and created AI agents to automate the setup of streaming features, seamlessly integrating RTM into their existing feature store.

"Our machine learning engineers didn't need to learn the intricacies of Real-Time Mode," noted Kamila Wickramarachchi, Software Engineer at Coinbase. "We simply delivered the massive improvements in data freshness and consistency, and they immediately saw the value in the results."

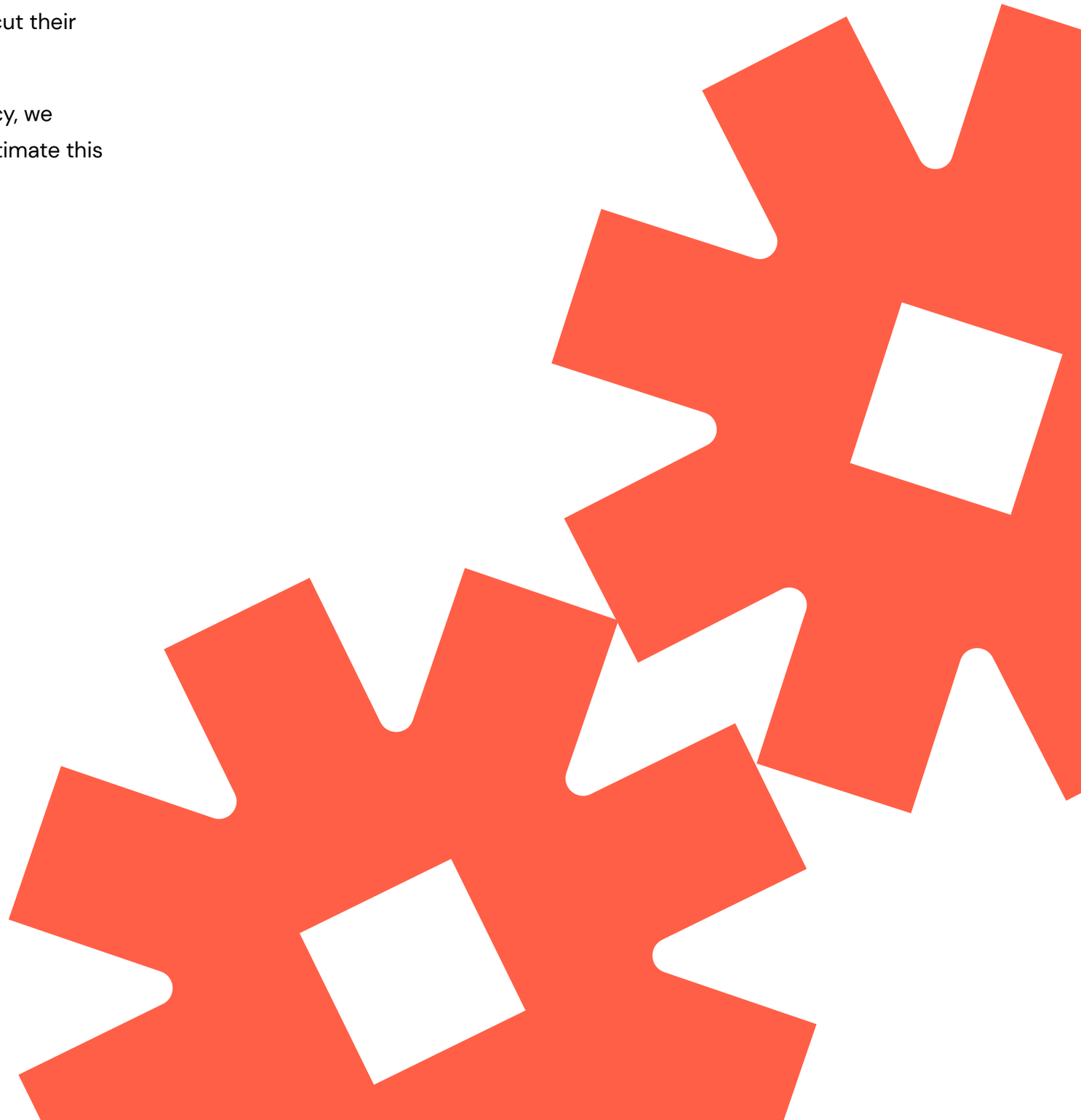
Faster insights at a fraction of the cost

Since implementing RTM, Coinbase has improved its ability to mitigate fraud by ensuring risk models act on the most up-to-date transaction data. Latency dropped to sub-second freshness, achieving 150ms for stateless and 250ms for stateful streaming feature aggregations. Online and offline feature consistency saw up to 98% improvement.

This architectural shift empowered the team to achieve remarkable scale and speed. As Daniel Zhou, Senior Staff Machine Learning Platform Engineer at Coinbase, explained, "By leveraging Real-Time Mode in Spark Structured Streaming, we've achieved an 80%+ reduction in end-to-end latencies, hitting sub-100ms P99s, and streamlining our real-time ML strategy at massive scale. This performance allows us to compute over 250 ML features all powered by a unified Spark engine."

Beyond performance gains, RTM enabled Coinbase to decommission their previously specialized, heavily provisioned clusters required by microbatch mode. This fundamentally changed their cost structure, and the team cut their compute costs in half.

"On top of the massive improvements in data freshness and consistency, we realized an incredible cost reduction," added Wickramarachchi. "We estimate this architectural shift will save us 51% in compute costs this year alone."



AccuWeather | CUSTOMER STORY

Transforming weather forecasting with Lakeflow Jobs

INDUSTRY Technology and Software

PRODUCT Jobs

3x
Faster dataset development

50%
Reduction in unactionable alerts

50%
Cost savings on serverless job usage

databricks

> Lakeflow Jobs saves us at least 10% of hours of work every month, which increases our development lifecycle speed. Instead of spending three months on one data set, we can now do one data set a month.
>
> **— Travis Teague, Data Operations Manager, AccuWeather**

AccuWeather powers global forecasts with Databricks

AccuWeather, recognized and documented as the most accurate and most used source of weather forecasts and warnings in the world, has saved over 12,000 lives, prevented injury to over 100,000 people, and saved companies tens of billions of dollars through better planning and decision-making.

Billions of people around the world rely on AccuWeather's proven Superior Accuracy™ across their consumer digital platforms.

AccuWeather For Business serves more than half of the Fortune 500 companies and thousands of other businesses and government agencies globally who pay for the most accurate weather forecasts than from any other known source.

Faced with the challenge of ingesting, processing and storing immense volumes of diverse weather data at scale, AccuWeather turned to Databricks and Lakeflow Jobs, working with Datadog for observability.

Managing complex weather data at unprecedented scale

AccuWeather has three critical data use cases that power its global forecasting services. The company maintains 30 years of historical weather data for customer analysis, processes real-time observational data for live streaming to customer apps and generates proprietary forecasts by blending multiple weather models with expert meteorologist input—a unique approach that sets them apart from competitors who rely on single data sources. "There are hundreds of parameters that go into forecasting the weather, and when it comes to forecast models, you have hundreds of those that are all weighted differently," explains Travis Teague, Data Operations Manager. "Measuring, explaining and forecasting weather requires continuous collection and processing of all of these models in near real time."

The company faced two primary challenges with its on-premises infrastructure. First, data volume constraints forced them to regularly purge valuable information, preventing historical analysis of model performance. "If we wanted to do analysis on a model run that came in and see if it was worse than what's coming in now, we can't do that because we have to purge that data from our on-prem systems because we just don't have capacity for it," says Teague.

The second challenge involved managing numerous data sources without a single source of truth. Multiple teams were pulling the same data for different purposes, creating expensive data silos and redundant processing costs. The situation was complicated by weather data's unique characteristics, which consist of highly specialized file formats not found in any other industry and that require custom tools for processing.

AccuWeather needed a solution that could automate its complex data workflows, eliminate the manual planning and validation steps that consumed days of effort, and orchestrate seamless job dependencies across their diverse weather datasets.

Unifying complex weather data with Databricks and Lakeflow Jobs

AccuWeather migrated to Databricks to leverage cloud scalability, eliminate on-premises storage constraints and consolidate its fragmented data sources into a unified platform. To address its complex orchestration challenges, AccuWeather selected Lakeflow Jobs as the primary workflow engine to automate the manual planning and validation processes that were consuming days of effort each month. Lakeflow Jobs enabled AccuWeather to orchestrate seamless dependencies across 4,500+ weekly jobs, automatically trigger downstream processes when new weather data arrived and eliminate the cumbersome manual interventions that had previously plagued their data pipelines. The platform's ability to consolidate all weather data formats—from highly structured forecast data to unstructured radar images in PNG or GeoTIFF formats—into a central data lake created the single source of truth that had been missing from their fragmented on-premises setup.

Databricks also helped AccuWeather address its data volume challenges. Because Databricks is a proven platform for enterprise scalability and reliability, AccuWeather can now efficiently distribute large workloads across scalable clusters, which provides them a platform to leverage AI and build their own models. Lakeflow Jobs' serverless capabilities mean the team no longer worries about server patches, software upgrades or managing underlying Spark versions and they can process massive amounts of data at scale quickly and cost efficiently.

The platform's flexibility allows AccuWeather to install specialized weather industry tools on traditional clusters, enabling them to work with highly specialized data formats. "Our data is very unique, and Databricks can handle that," says Teague

Lakeflow Jobs orchestrates the complex multi-step workflows that power AccuWeather's data products, automatically triggering job sequences as soon as new data becomes available. For historical data processing, Lakeflow Jobs manages the intricate pipeline dependencies where observational data feeds into initial historical records, then triggers downstream jobs that enrich and validate the datasets before making them available through Delta Sharing. The orchestration capabilities have been crucial for AccuWeather's advanced forecasting initiative, where Lakeflow Jobs coordinates the ingestion of multiple weather models, triggers machine learning processes that weight and blend different forecasts and manages the complex job dependencies required for reinforcement training workflows. By automating these previously manual orchestration tasks, Lakeflow Jobs enables AccuWeather to deliver real-time weather intelligence to millions of users while building more sophisticated forecasting capabilities

Faster development and enhanced weather intelligence

The migration to Lakeflow Jobs delivered immediate operational improvements that accelerated AccuWeather's dataset development from three months per dataset to one month, enabling faster time-to-market for new weather data products. Lakeflow Jobs eliminated the need for manual intervention, saving an average of one to two days of processing time per job. The team no longer needs to manually validate data before and after processing. Serverless compute handles pre-checks to ensure all source weather data files have arrived from providers and post-checks to validate that processed datasets meet accuracy and completeness requirements before triggering downstream jobs. One critical job converted entirely to serverless Lakeflow Jobs cut usage costs in half, while reducing their reliance on Azure Data Factory has reduced both complexity and licensing expenses.

Development productivity has surged thanks to Declarative Automation Bundles, which provide template repositories for new projects with built-in CI/CD deployment. "Lakeflow Jobs saves us at least 10% of hours of work every month," explains Teague. This time was previously spent on manual orchestration tasks including planning deployment schedules, coordinating job timing across teams, manually triggering dependent jobs when upstream processes completed, and troubleshooting workflow failures that required restarting entire job sequences. "That time saved means it increased our development lifecycle speed. Instead of spending three months on one data set, we can now do one data set a month."

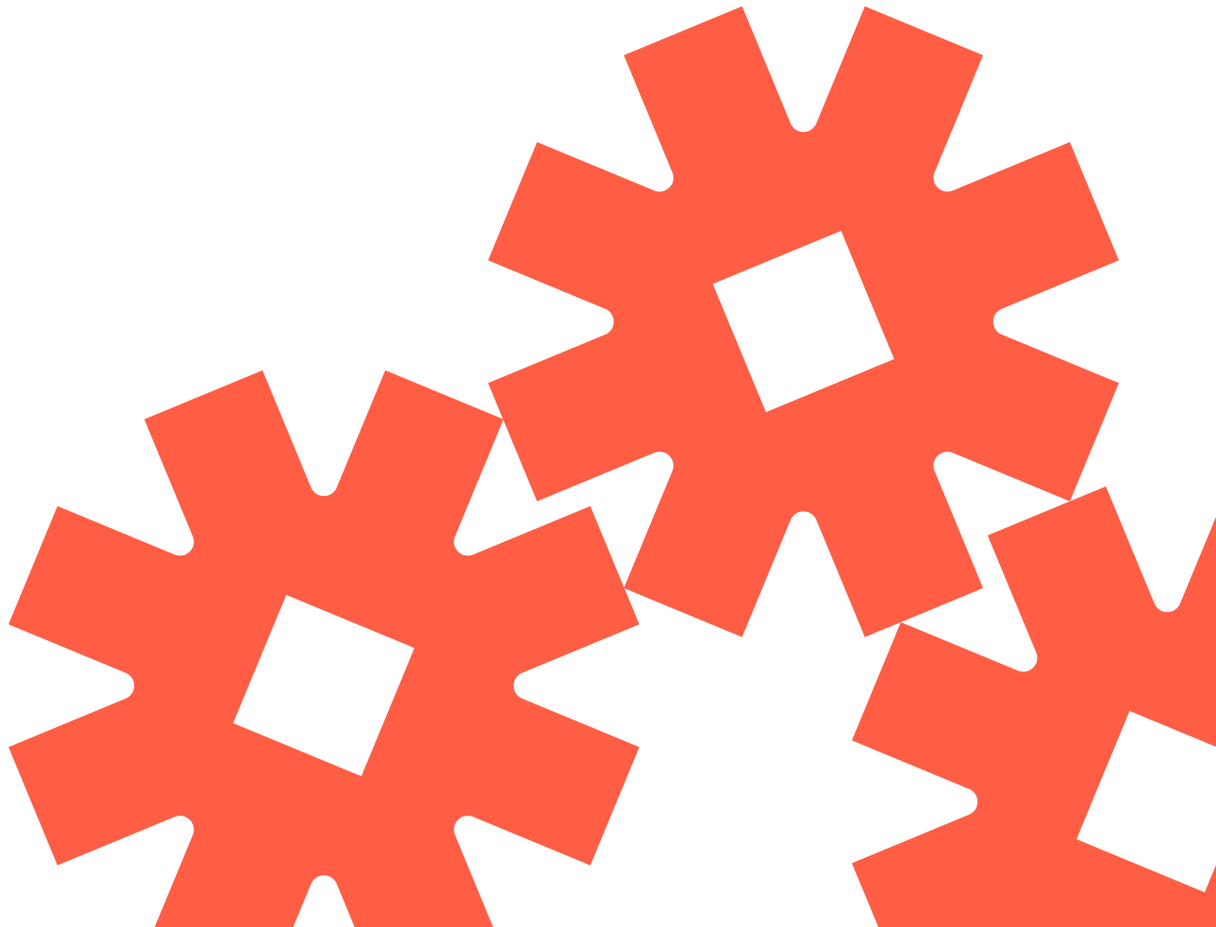
This acceleration has also enabled AccuWeather to expand their team and work on multiple datasets simultaneously instead of focusing the entire team on a single project.

The faster dataset development cycles and automated orchestration directly impact AccuWeather's ability to continue delivering their forecasts with proven Superior Accuracy™ and often providing more advanced warnings than any other known source that save lives, protect property, and help people make the best weather-impacted decisions. The team can now bring in more raw datasets to enrich their historical products and incorporate additional models into their forecasting engine, helping to make their weather intelligence the most accurate and the most comprehensive. Faster iteration through datasets means more achievable deadlines, rapid development cycles, and richer data for customers who rely on AccuWeather's services.

Meanwhile, Databricks' partnership with observability platform provider Datadog has ensured pipeline reliability and performance optimization. Datadog's Data Jobs Monitoring provides unified visibility into Databricks jobs and workflows, helping AccuWeather detect failures and latency spikes faster. "We have reduced unactionable alerts by over 50% through these correlated and aggregated alerting monitors," notes Teague. "Before, our normal incident response time was around an hour and a half. Now it's reduced to just a couple of minutes."

When monitoring Lakeflow Jobs, Datadog applies domain-specific business logic to detect critical issues, such as repeat job failures, and understand the root cause of the failure. When thresholds are reached, alerts are automatically routed to the owning teams enabling faster investigation and resolution. Datadog also helps rightsize AccuWeather's Databricks environment and improve job performance by surfacing idle compute, cluster utilization and Spark execution metrics.

The combined improvements have transformed how AccuWeather delivers weather intelligence to millions of users worldwide. As Teague summarizes, "Databricks and Lakeflow Jobs's scalability, availability and overall performance has helped AccuWeather meet its mission to save lives, protect property, and help people prosper."



TOYOTA | CUSTOMER STORY

Toyota uses Zerobus Ingest for real-time factory data

INDUSTRY Financial Services
CLOUD AWS, Azure, Google Cloud
PRODUCT Connect/Zerobus

98%
Reduction in data latency from 4.5 seconds to 0.1 seconds

Hours to minutes
Overheating conditions detected faster

databricks

> Zerobus Ingest allows us to detect overheating factory conditions in minutes rather than hours, directly supporting our carbon-neutrality strategy and operational efficiency.
>
> **— Kento Izumi, General Manager, Digital Transformation Promotion Division, Toyota Motor Corporation**

Toyota, the world's largest automaker, built its reputation on the Toyota Production System—a methodology that revolutionized manufacturing efficiency. As the company pursues its vision of "Creating Mobility for All" and mission of "Producing Happiness for All," it recognizes that operational excellence extends beyond the assembly line. Toyota turned to Databricks and Zerobus Ingest to monitor and improve manufacturing conditions, ensuring employees work safely and comfortably while maintaining production standards.

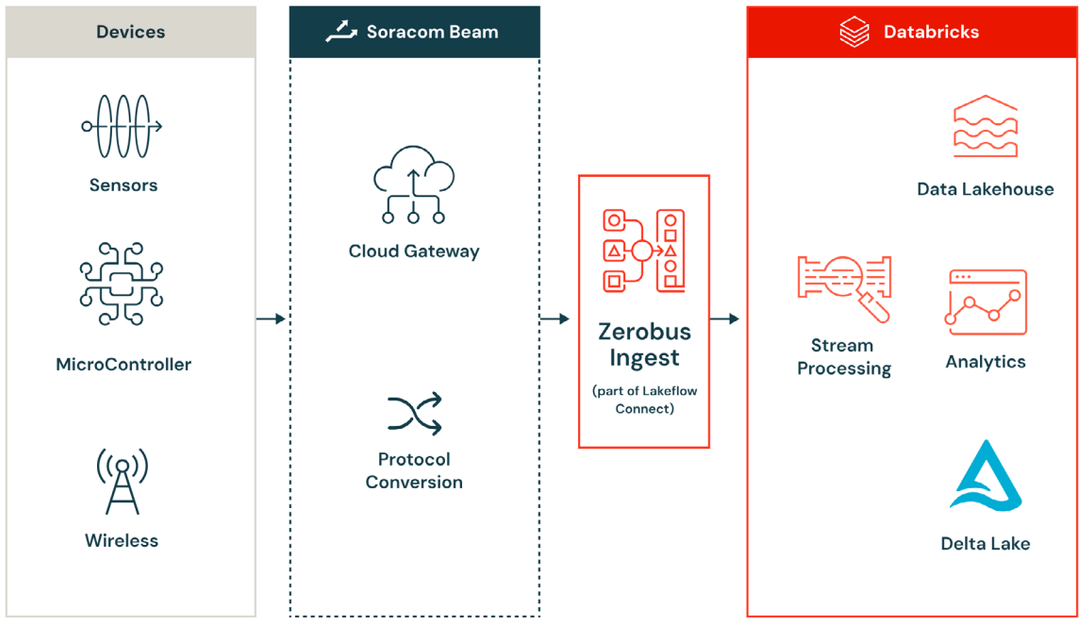
Creating safe working conditions through data

Toyota's mission of "Producing Happiness for All" extends beyond the vehicles it manufactures to the people who build them. Vehicle production demands physically intensive work in challenging environments, where conditions can become particularly harsh during summer months when factory temperatures rise. The company recognized that protecting employees' health and comfort was fundamental to its values and long-term success.

Traditional approaches to monitoring factory conditions provided limited visibility. Without real-time data on temperature, humidity and environmental factors across production facilities, Toyota couldn't proactively identify risk areas or optimize conditions before they affected workers. The company needed a way to capture, analyze and act on manufacturing environment data at scale, leveraging data and AI to create safer, more comfortable workspaces aligned with its commitment to employee wellbeing.

Streaming factory telemetry with minimal latency

Toyota built its solution on Databricks and Zerobus Ingest, creating a direct pipeline from manufacturing equipment to real-time analytics. The architecture leverages Soracom's global IoT connectivity to capture telemetry data from sensors and equipment across multiple factories, streaming it directly to Databricks without the intermediary cloud services that can introduce latency and complexity.



The data flow is straightforward: manufacturing equipment sends telemetry through Soracom Beam to Zerobus Ingest, which ingests and processes the data in Databricks. Toyota tested the integration using both gRPC and REST API protocols to ensure optimal performance and reliability. By eliminating intermediate processing layers, the system delivers near real-time visibility into factory conditions, enabling Toyota to monitor environmental factors as they change throughout the day.

Unity Catalog provides unified governance across the entire data pipeline, ensuring that telemetry data is properly cataloged, secured, and accessible to authorized teams. With data flowing directly into Databricks through Vista—Toyota's unified data and AI platform—the company can apply analytics and AI models for predictive maintenance, anomaly detection and energy optimization across its manufacturing environment.

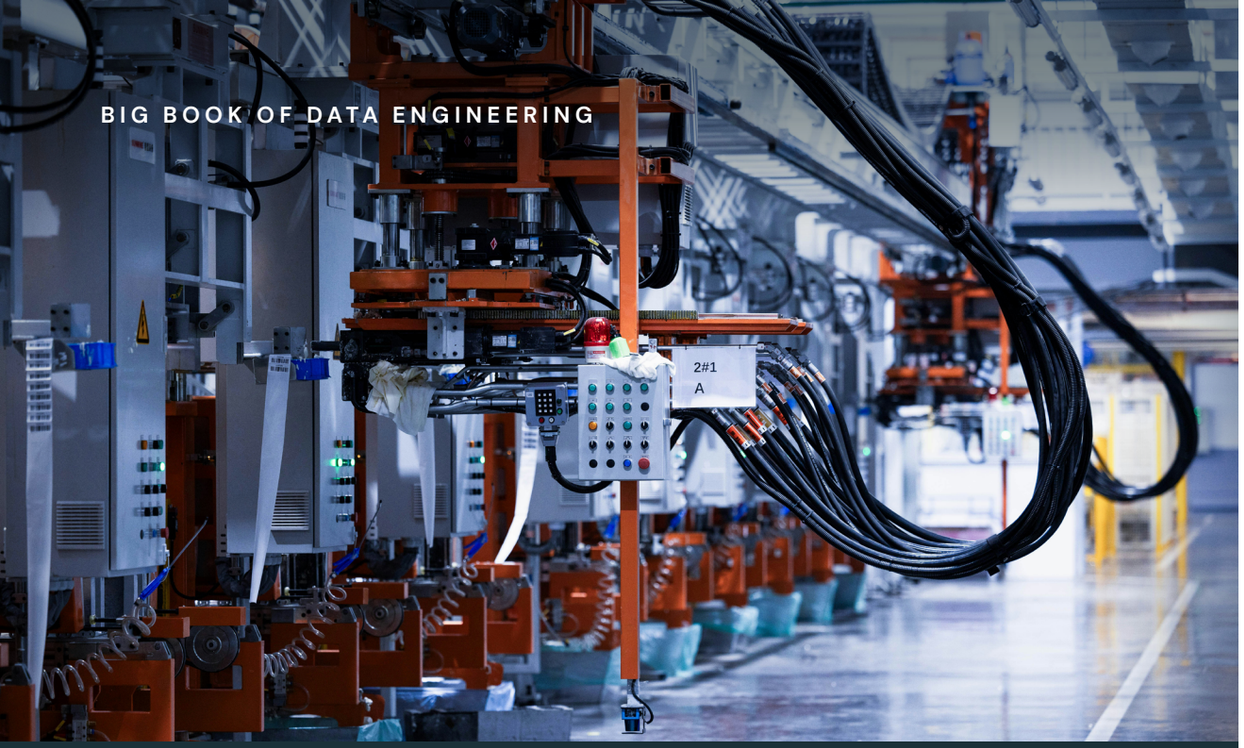
Real-time response and measurable sustainability impact

The shift to Zerobus Ingest transformed how Toyota monitors and responds to factory conditions. Toyota reduced data latency by 98%, from an average of 4.5 seconds on its existing network to just 0.1 seconds using Zerobus Ingest with gRPC protocol. By streaming telemetry data, including equipment temperature readings, in near real time, the company can now detect early signs of overheating. "Zerobus Ingest allows us to detect overheating factory conditions in minutes rather than hours, directly supporting our carbon-neutrality strategy and operational efficiency," said Kento Izumi, General Manager, Digital Transformation Promotion Division at Toyota Motor Corporation.

Human operators receive immediate alerts, enabling them to determine whether cooling actions are needed and execute appropriate countermeasures. While future automation is possible, Toyota has deliberately maintained human decision-making in the loop for this mission-critical system, ensuring experienced personnel validate responses before action is taken.

By dynamically adjusting cooling systems based on live temperature data, Toyota reduces unnecessary energy consumption across its facilities. The ability to respond to conditions as they develop has delivered exponential improvements in energy efficiency across numerous factories.

Unity Catalog's built-in governance ensures consistent data quality and security from ingestion to analysis, while AI/BI Genie enables faster, interactive analysis cycles that support proactive, data-driven operations. The platform has also reduced latency and infrastructure costs by eliminating the need for intermediary cloud services that once sat between data sources and analytics. "Having Zerobus Ingest as an additional data ingestion option gives us the ability to collect diverse factory data in near real-time and trigger immediate countermeasures that have transformed our operations," said Izumi. "When combined with Vista, our unified data and AI platform powered by Databricks, we aren't just collecting data faster; we are optimizing our data operations."



DAIKIN

CUSTOMER STORY

How Daikin Applied Americas builds consistent data pipelines at scale with Genie Code

INDUSTRY Industrials

CLOUD Azure

PRODUCT Genie Code

databricks

Agentic data engineering is changing how pipelines are built

Daikin Applied Americas (DAA) manufactures and services commercial HVAC systems across North America. That means managing large volumes of operational, manufacturing and service data across systems, from equipment telemetry and supply chain data to field service records.

The data team supports analytics and AI use cases across engineering, operations and customer service, all of which depend on reliable, well-structured pipelines.

As those demands grew, so did the pressure on the data team, including more pipelines, more use cases and more coordination across teams. To address this, the team defined a more structured operating model for how pipelines are designed, built and governed, and used Databricks Genie Code to accelerate execution within that model.

The team leveraged Genie Code as an AI-assisted approach to data engineering. Working directly against governed data in Unity Catalog can help plan and generate multi-step pipelines across the workflow. This allows engineers to move from an idea to a working pipeline much faster, without switching tools or manually stitching components together.

That speed fundamentally changed how the team worked. Pipelines that previously took days to prototype could be generated in minutes. Iteration cycles were shortened, and engineers spent less time writing boilerplate and more time refining logic and outcomes.

At the same time, operating in a large, shared data environment requires consistency. Pipelines must follow common architectural patterns, use shared definitions and behave predictably across teams.

Large language models introduce a structural challenge in this context. When teams rely on varied prompts or loosely defined instructions, the same request can yield inconsistent outputs and lead to architectural drift over time.

To address this, the DAA team focused on defining how AI should operate within a governed enterprise environment, rather than relying solely on prompt engineering.

As Trent Lezer, Sr. Director, Data & Analytics at Daikin Applied Americas, puts it: "Genie Code works best when treated like a junior engineer who works fast but must respect the same architectural constraints as everyone else, no special exemptions 'because it's AI.'"

Scaling data engineering through reusable skills

Early usage of Genie Code followed a familiar pattern: long prompts that attempted to encode architecture rules, naming standards, transformation logic and documentation requirements in a single block of text.

This approach did not scale. Instructions varied across teams, prompts became difficult to maintain and similar tasks produced inconsistent outputs.

To address this, the team introduced a MECE (Mutually Exclusive, Collectively Exhaustive) skill framework. As Trent explains: "We implemented a MECE skill framework, each skill defines one coherent competency, skills are non-overlapping and the full set covers the entire lifecycle of data engineering work."

Each skill defines a specific capability in the data engineering lifecycle. Together, the skills are non-overlapping and cover the full workflow. These skills include medallion architecture design, source readiness and grain definition, transformation patterns, canonical alignment and governance standards.

Instead of embedding rules inside prompts, the team structured the environment so Genie Code loads the appropriate skills at runtime and applies them during planning and execution. This shifts behavior from interpreting ad hoc instructions to operating within a defined execution model.

From a governance perspective, this also changes how standards are enforced. As James VanGordon, Solutions Architect at Databricks, notes: "The pattern I keep seeing with Genie Code is pretty simple: prompts get you started, but they are a bad place to enforce team standards. If the same rule matters more than once, it should live in the workspace as a skill, where Genie Code can actually use it."

He also emphasizes embedding standards directly into the execution environment: "That is what makes this real instead of wishful thinking. The skills, Unity Catalog context and Genie Code are working in the same place. The guidance sits where the work is being created, not off to the side in a review process someone has to remember later."

Using the medallion architecture to guide pipeline development

The team also strengthened the role of the medallion architecture as both a governance and reasoning framework. Bronze, Silver and Gold layers already existed, but the shift was making them explicit decision boundaries during pipeline generation, not just storage tiers.

Bronze represents raw source truth. Silver represents cleaned and conformed data. Gold represents business-ready analytics.

To operationalize this structure, the team introduced checkpoints between layers. Before data advances, requirements such as source grain definition, join validation and data stability checks must be satisfied.

These checkpoints are enforced within the development workflow itself, not as downstream review steps. Genie Code operates within these constraints as pipelines are generated and modified.

This ensures consistency across teams while reducing the risk of architectural shortcuts during rapid development.

Connecting pipelines to business concepts

A recurring challenge in enterprise data engineering is aligning technical models with business language.

At DAA, stakeholders think in terms of equipment, customers, service events and contracts, not tables, joins or transformations.

To address this, the team anchored pipeline design in stable business entities. Rather than starting with technical structures, engineers begin by identifying what the data represents and how it behaves over time.

This shift improves downstream efforts and reduces ambiguity when datasets are reused across domains.

Over time, Silver-layer models and Gold datasets become more consistent because they are grounded in shared business concepts rather than isolated technical decisions.

What changed for the team

With this operating model in place and AI embedded, the team saw a clear shift in how work was executed.

Pipeline development accelerated, particularly during early exploration and iteration. Engineers spent less time writing boilerplate code and more time refining business logic.

Outputs also became more consistent across teams. Similar use cases followed similar structural patterns, improving maintainability and reuse.

Importantly, trust in generated outputs increased. Engineers spent less time validating structural correctness and could iterate more quickly.

STANDARDIZING DECISION-MAKING WITHIN THE DEVELOPMENT WORKFLOW

To make these gains repeatable, the team standardized key decisions within the development process.

Rather than relying on implicit knowledge, definitions were made explicit, including what qualifies as Bronze, Silver and Gold data, how source grain is defined, which transformation patterns are reusable and how business entities are represented. This structure was critical for scale. It ensures AI operates within a consistent framework across teams, even as use cases evolve.

The payoff: what this unlocked at scale

The result of this operating model was not just faster pipelines. It was the ability to scale data engineering in a governed enterprise environment.

FASTER DELIVERY WITH FEWER CORRECTIONS

Engineers spend less time fixing structurally incorrect pipelines and more time refining logic and business outcomes.

REDUCED ARCHITECTURAL DRIFT ACROSS TEAMS

Consistent application of skills and governance checkpoints prevents divergence across teams working on similar data challenges.

STRONGER ALIGNMENT BETWEEN ENGINEERING AND BUSINESS

Grounding pipelines in business concepts improves clarity and reduces downstream rework.

SCALABLE GOVERNANCE WITHOUT MANUAL OVERHEAD

Guardrails are embedded directly into the system, reducing reliance on manual enforcement.

INCREASED TRUST IN AI-GENERATED OUTPUTS

Because defined skills and checkpoints constrain outputs, AI operates reliably within production workflows.

As Trent summarizes: “The goal isn’t to make AI follow more rules. It’s to make the right rules impossible to ignore.”

Conclusion

At Daikin Applied Americas, combining a structured operating model with AI-assisted development allowed the data team to scale faster while maintaining consistency, clarity and control.

By defining how pipelines should be built and embedding those rules directly into the development environment, the team created a system in which speed and governance reinforce each other rather than compete.

Learn more about Genie Code.

This blog was co-authored by Databricks and Trent Lezer (Sr. Director of Data & Analytics, Daikin), exploring their implementation of Genie Code.

Unify your stack. Automate your operations.

Databricks Lakeflow is built to eliminate the gaps caused by legacy tool sprawl. By unifying ingestion, transformation, and orchestration on a single platform, Lakeflow empowers data engineers to stop firefighting and start building. With agentic AI putting pipeline maintenance on autopilot, you can finally deliver the trusted, real-time foundation your AI workloads require.

Learn more | **Watch a demo** | **Try Databricks free**

PRO TIP: Ready to master agentic data engineering? Databricks offers comprehensive free training and industry-recognized certifications to help you validate your skills. Learn how to eliminate legacy tool sprawl and build the pipelines of the future.

Explore the Data Engineering Learning Path | **Get Certified in Data Engineering**

About Databricks

Databricks is the data and AI company. More than 20,000 organizations worldwide — including adidas, AT&T, Bayer, Block, Mastercard, Rivian, Unilever and 70% of the Fortune 500 — rely on Databricks to build and scale data and AI apps, analytics and agents. Headquartered in San Francisco, with 30+ offices around the globe, Databricks offers a unified platform that includes Lakebase, Genie, Agent Bricks, Lakeflow, Lakehouse and Unity Catalog. To learn more, follow Databricks on LinkedIn, X and Facebook.

databricks

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark and the Spark logo are trademarks of the Apache Software Foundation. Privacy Policy | Terms of Use